Lecture 22 (11/6)

Overfitting and the bias–variance tradeoff

Today:

1) Polynomial regression: setup, features and fitting
2) Training and test sets (cross-validation)
3) The bias–variance tradeoff
4) Higher-order interactions in the spike protein

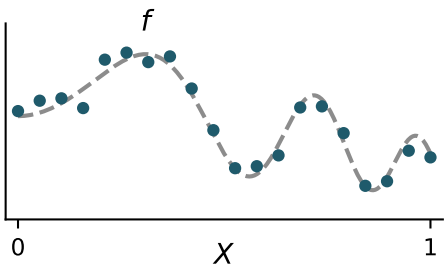
Polynomial regression setup

truth/data generating process

$Y = f(X) + \epsilon$

$\epsilon \sim \text{Normal}(0, \sigma_\epsilon^2)$

f is unknown



data

X	Y
0.00	0.17
0.05	0.53
0.11	0.61
⋮	⋮
1.00	−1.41

$N = 20$ points, equally spaced X in $[0, 1]$

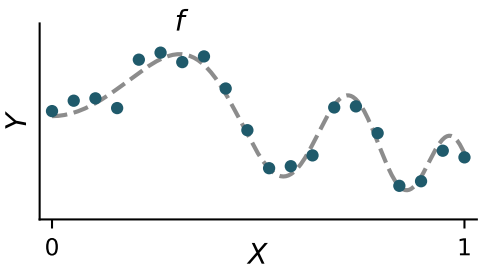
Polynomial regression setup

truth/data generating process

$Y = f(X) + \epsilon$

$\epsilon \sim \text{Normal}(0, \sigma_\epsilon^2)$

f is unknown



data

X	Y
0.00	0.17
0.05	0.53
0.11	0.61
⋮	⋮
1.00	−1.41

$N = 20$ points, equally spaced X in $[0, 1]$

representation in feature space

X^0	X^1	X^2	⋯	Y
1	0.00	0.00	⋯	0.17
1	0.05	0.00	⋯	0.53
1	0.11	0.01	⋯	0.61
⋮	⋮	⋮		⋮
1	1.00	1.00	⋯	−1.41

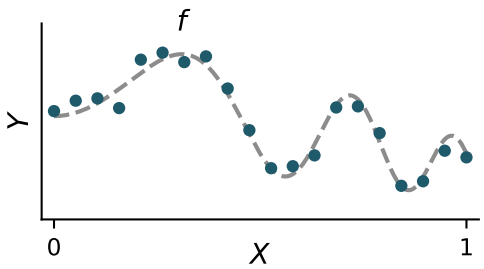
Polynomial regression setup

truth/data generating process

$Y = f(X) + \epsilon$

$\epsilon \sim \text{Normal}(0, \sigma_\epsilon^2)$

f is unknown



data

X	Y
0.00	0.17
0.05	0.53
0.11	0.61
⋮	⋮
1.00	−1.41

$N = 20$ points, equally spaced X in $[0, 1]$

representation in feature space

X^0	X^1	X^2	$\cdots$	Y
1	0.00	0.00	$\cdots$	0.17
1	0.05	0.00	$\cdots$	0.53
1	0.11	0.01	$\cdots$	0.61
⋮	⋮	⋮		⋮
1	1.00	1.00	$\cdots$	−1.41

probability model

$$Y = \sum_{j=0}^{K} \beta_j X^j + \epsilon$$

$\epsilon \sim \text{Normal}(0, \sigma_\epsilon^2)$

Review: Taylor series

We know from calculus that functions can be approximated by their Taylor series around $x = 0$:

$$f(x) = \sum_{j=0}^{\infty} \frac{f^{(j)}(0)}{j!} x^j = f(0) + f'(0)\, x + \frac{f''(0)}{2} x^2 + \cdots$$

If we truncate the Taylor series, we get a polynomial.

Review: Taylor series

We know from calculus that functions can be approximated by their Taylor series around $x = 0$:

$$f(x) = \sum_{j=0}^{\infty} \frac{f^{(j)}(0)}{j!} x^j = f(0) + f'(0)\, x + \frac{f''(0)}{2} x^2 + \cdots$$

If we truncate the Taylor series, we get a polynomial.

The idea of approximating a function by a polynomial can be generalized:

Theorem (Weierstrass approximation)

If f is continuous on $[a, b]$, then for every $\delta > 0$ there is a polynomial

$$p(x) = \sum_j \beta_j x^j$$

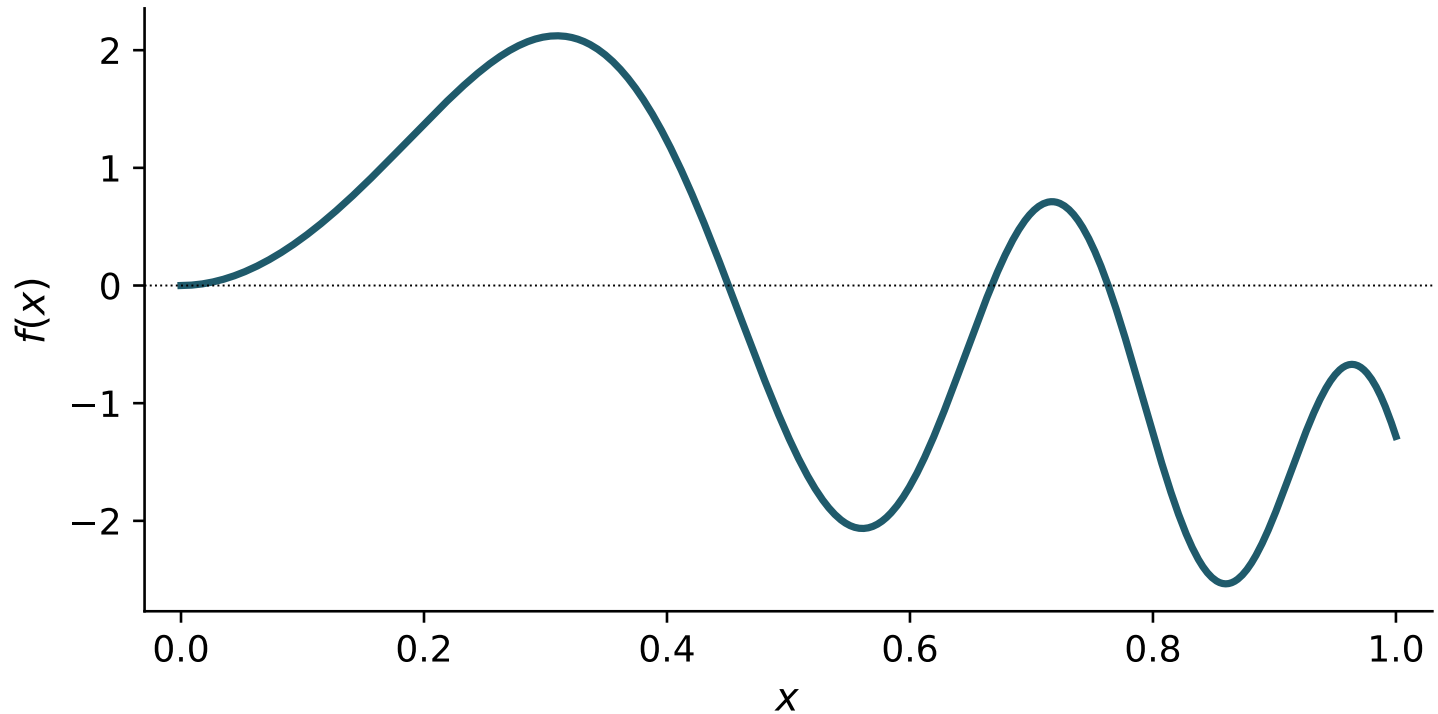
with

$$|f(x) - p(x)| < \delta \quad \text{for all } x \in [a, b].$$

Example

Throughout we will consider the example where the true function is

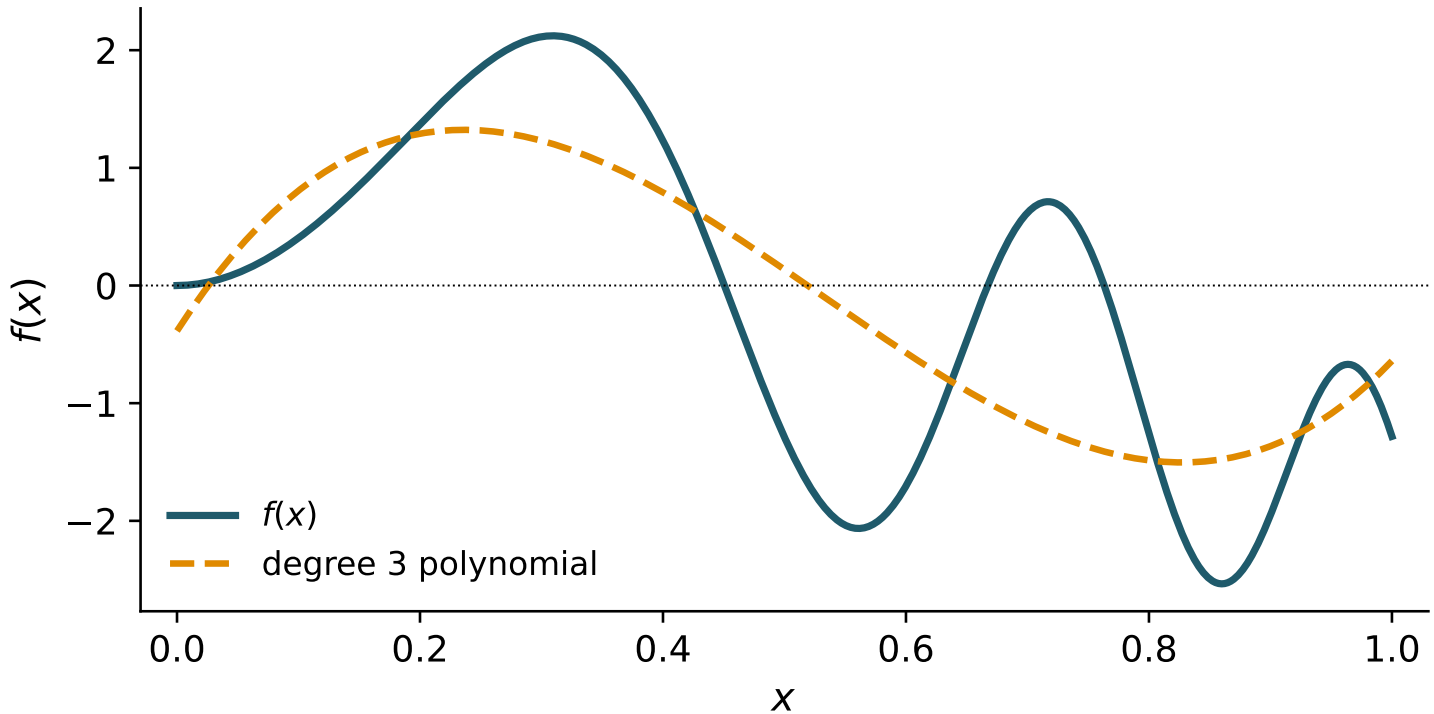
$$f(x) = 3e^{-x}\sin(15x^2) - 2x^3.$$



Example

Throughout we will consider the example where the true function is

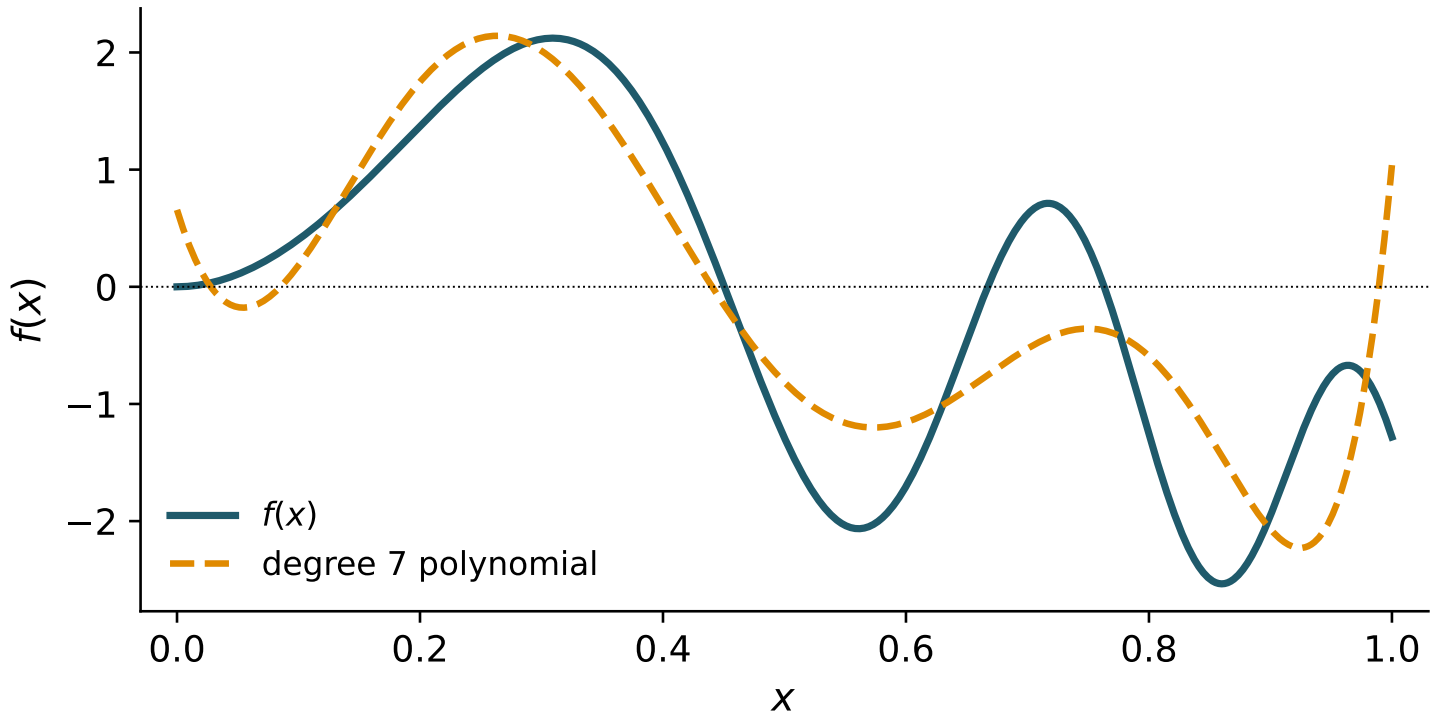
$$f(x) = 3e^{-x}\sin(15x^2) - 2x^3.$$



Example

Throughout we will consider the example where the true function is

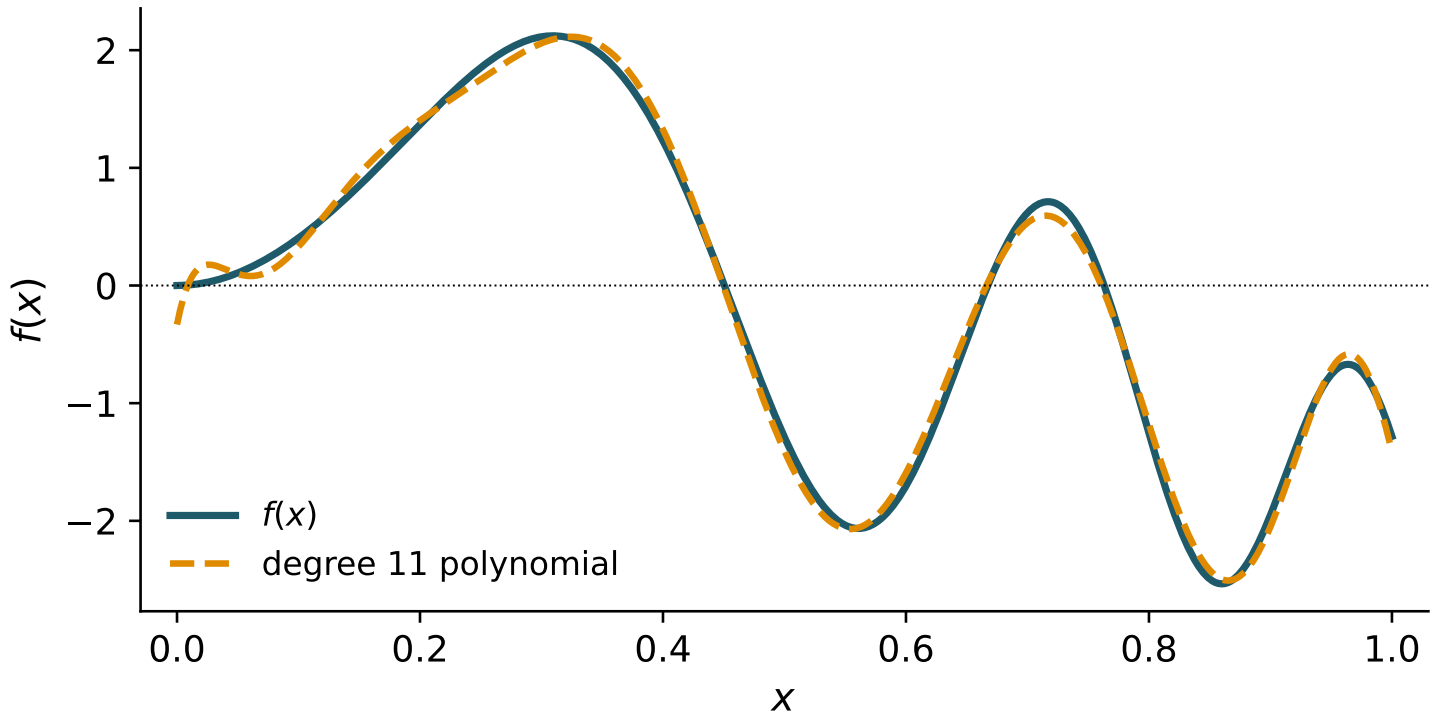
$$f(x) = 3e^{-x}\sin(15x^2) - 2x^3.$$



Example

Throughout we will consider the example where the true function is

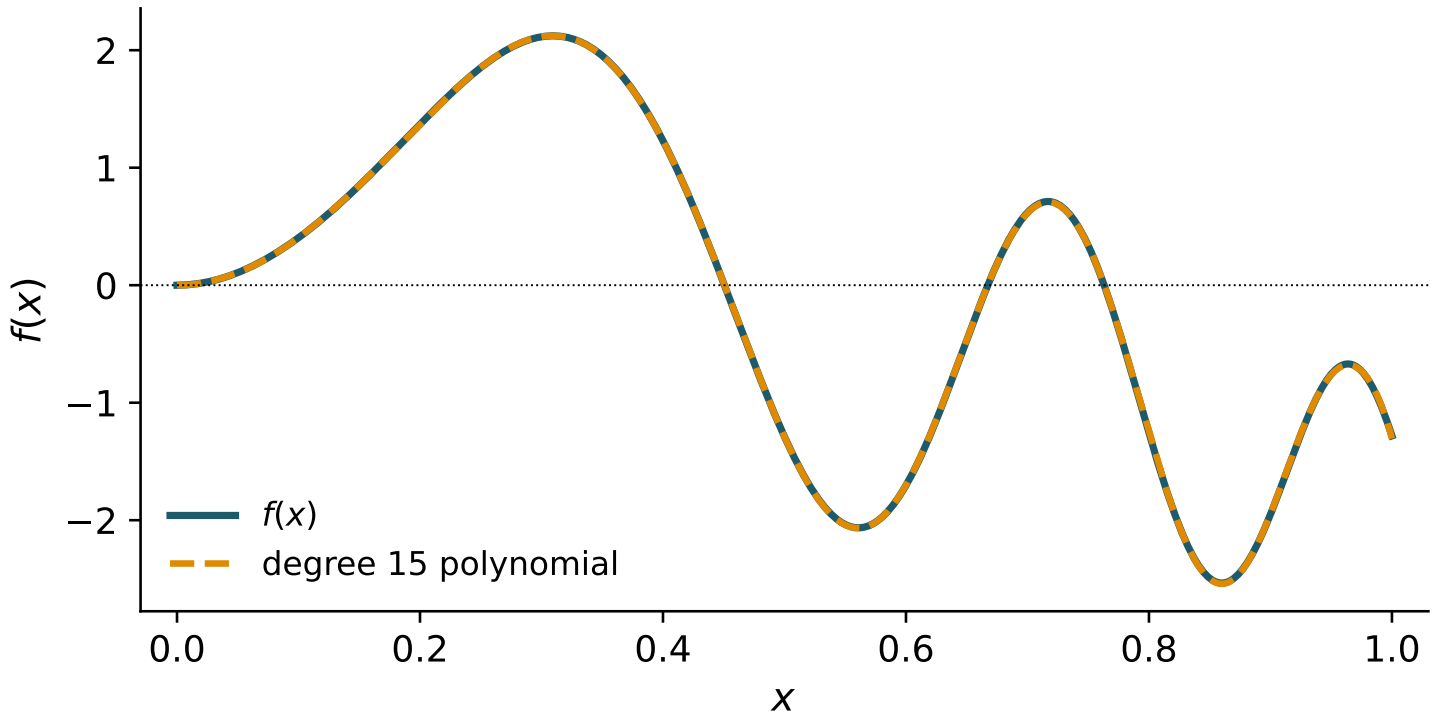
$$f(x) = 3e^{-x}\sin(15x^2) - 2x^3.$$



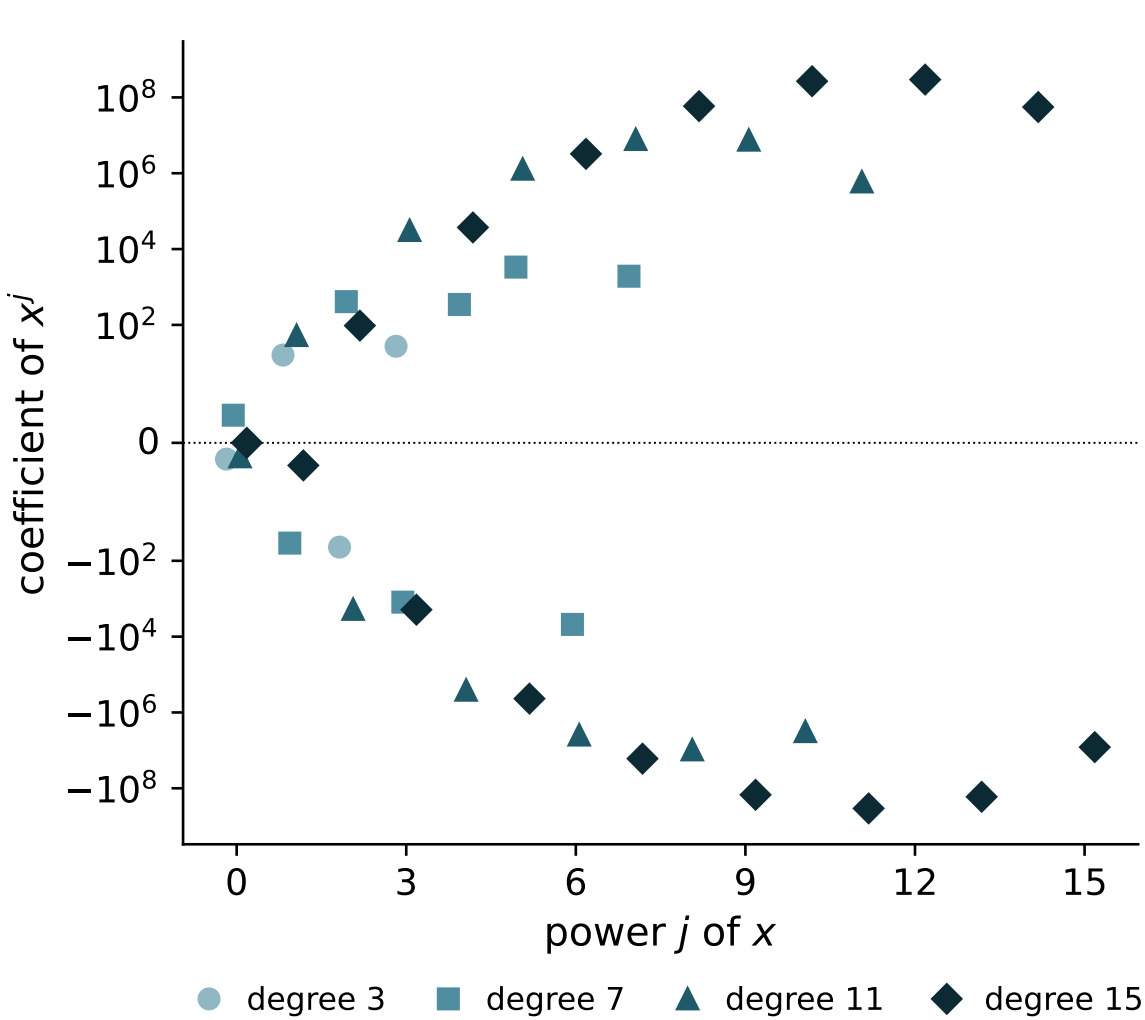
Example

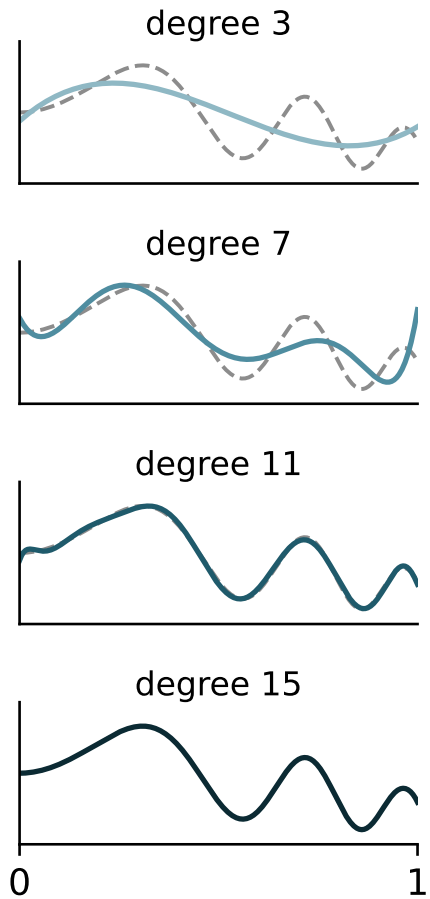
Throughout we will consider the example where the true function is

$$f(x) = 3e^{-x}\sin(15x^2) - 2x^3.$$



Polynomial approximations tend to have large coefficients





Generating the data and fitting the model

```
import numpy as np
import statsmodels.api as sm

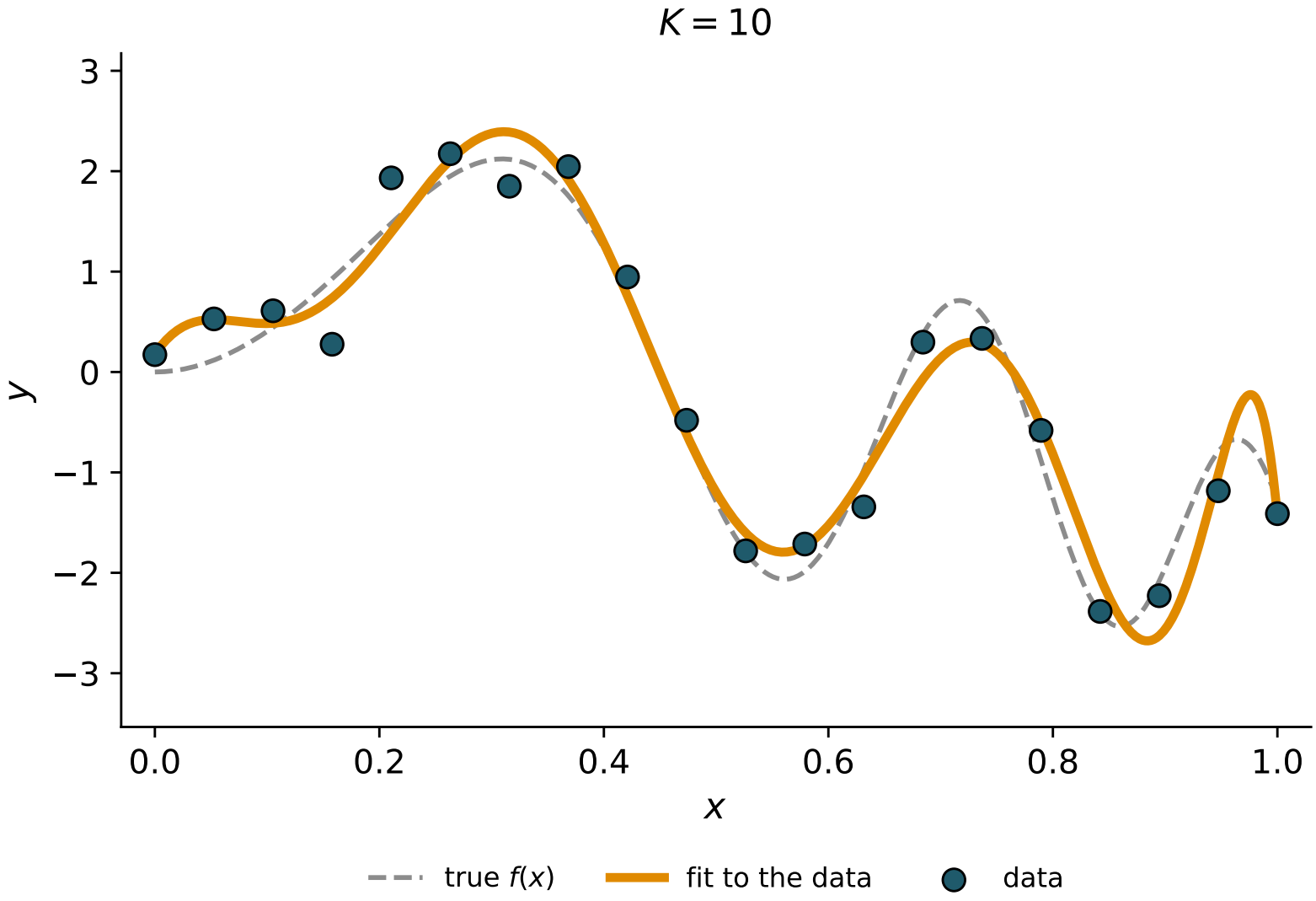
N = 20
x = np.linspace(0, 1, N)
f = 3 * np.exp(-x) * np.sin(15 * x**2) - 2 * x**3
y = f + np.random.normal(0, 0.5, N)

K = 10
X = np.zeros((N, K + 1))
for j in range(K + 1):
    X[:, j] = x**j
results = sm.OLS(y, X).fit()
print(results.summary())
```

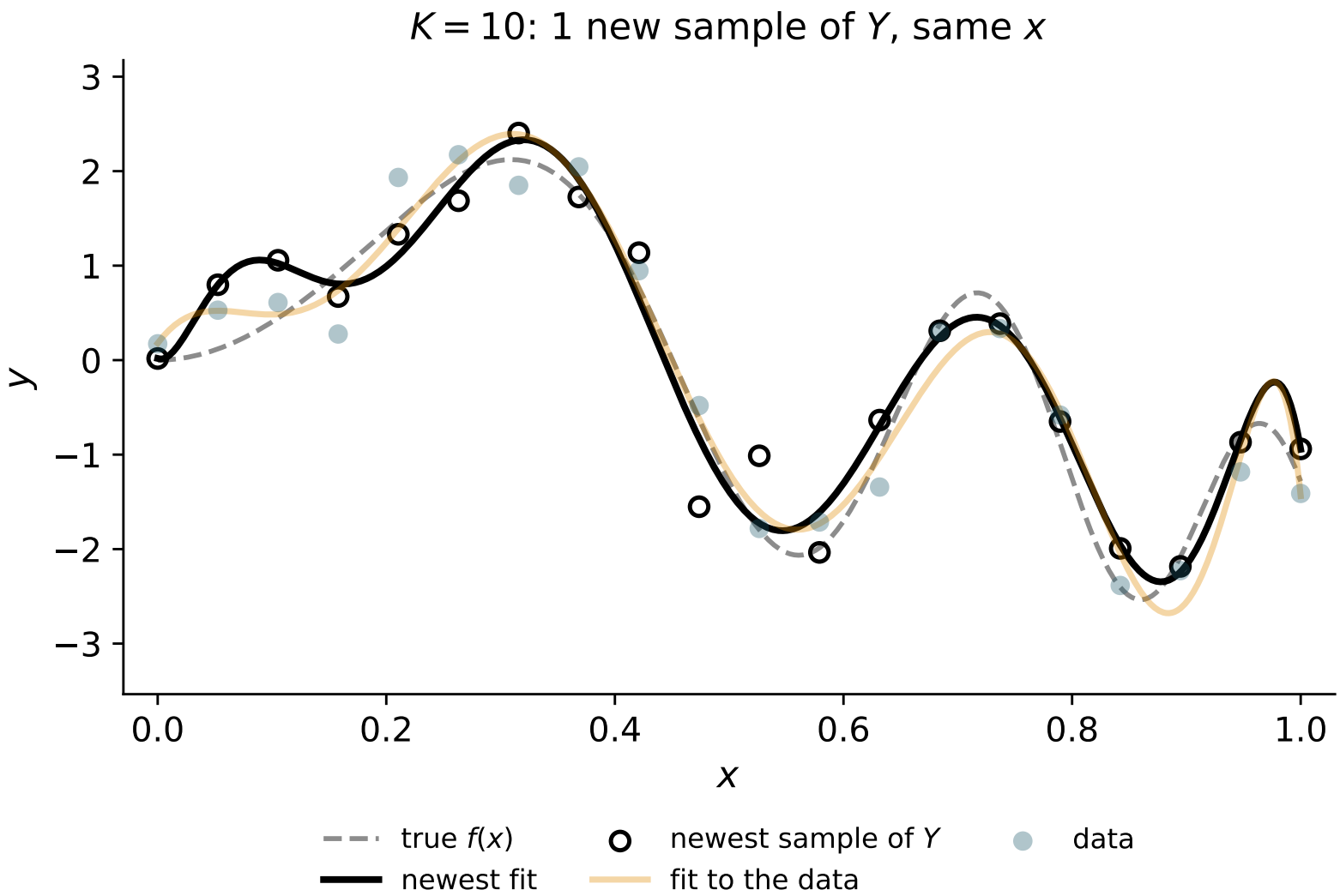
Output with $K = 10$

Dep. Variable:		y	R-squared:		0.964
	coef	std err	t	P>|t|	[0.025	0.975]
const	0.1787	0.403	0.444	0.668	-0.732	1.090
x1	16.3174	45.939	0.355	0.731	-87.604	120.239
x2	-247.9494	1306.197	-0.190	0.854	-3202.773	2706.874
x3	1138.2773	1.5e+04	0.076	0.941	-3.29e+04	3.52e+04
x4	1644.7547	9.1e+04	0.018	0.986	-2.04e+05	2.08e+05
x5	-1.719e+04	3.24e+05	-0.053	0.959	-7.5e+05	7.16e+05
x6	1.618e+04	7.14e+05	0.023	0.982	-1.6e+06	1.63e+06
x7	6.031e+04	9.84e+05	0.061	0.952	-2.17e+06	2.29e+06
x8	-1.509e+05	8.27e+05	-0.182	0.859	-2.02e+06	1.72e+06
x9	1.263e+05	3.87e+05	0.326	0.752	-7.5e+05	1e+06
x10	-3.72e+04	7.74e+04	-0.481	0.642	-2.12e+05	1.38e+05

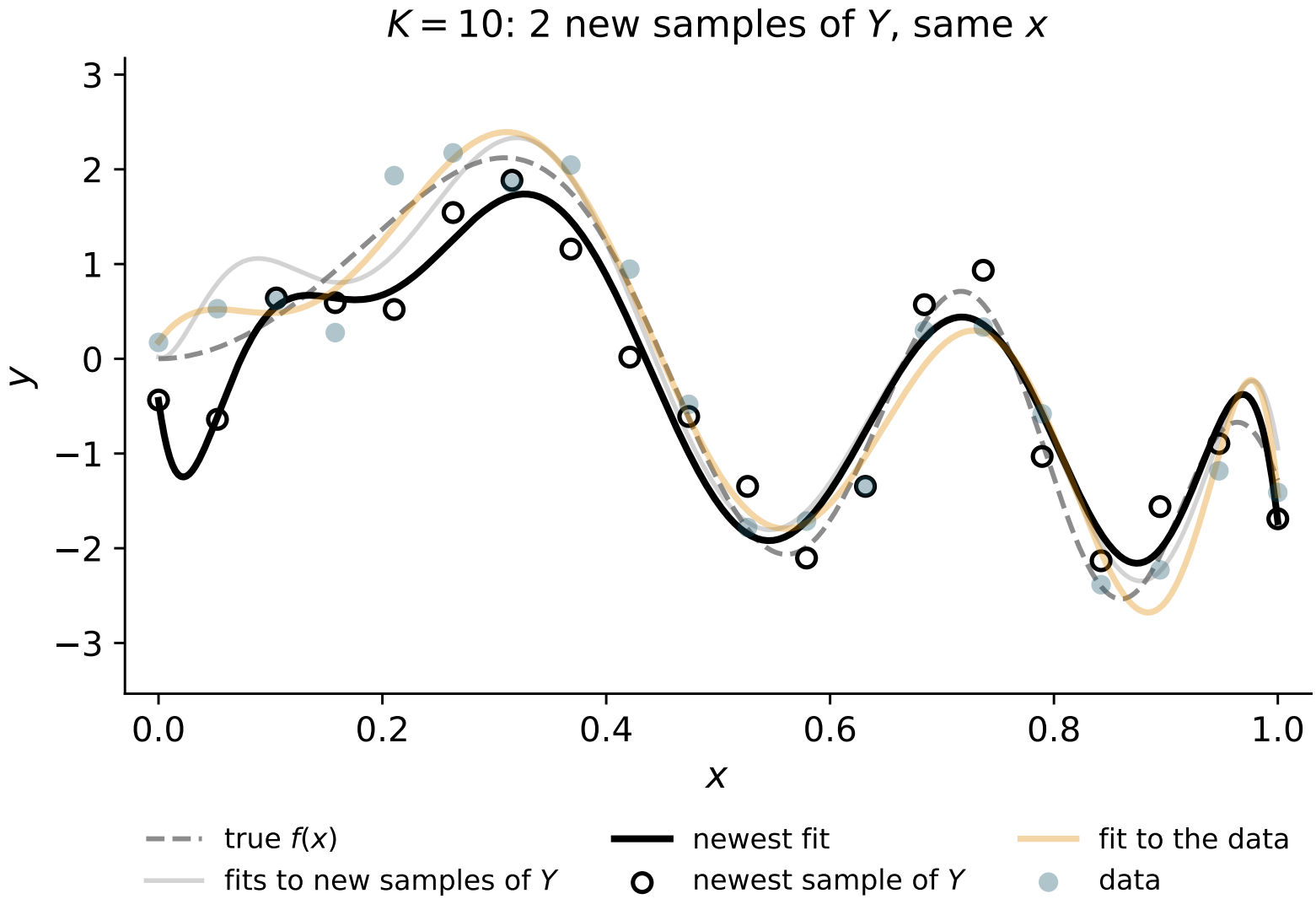
The $K = 10$ fit



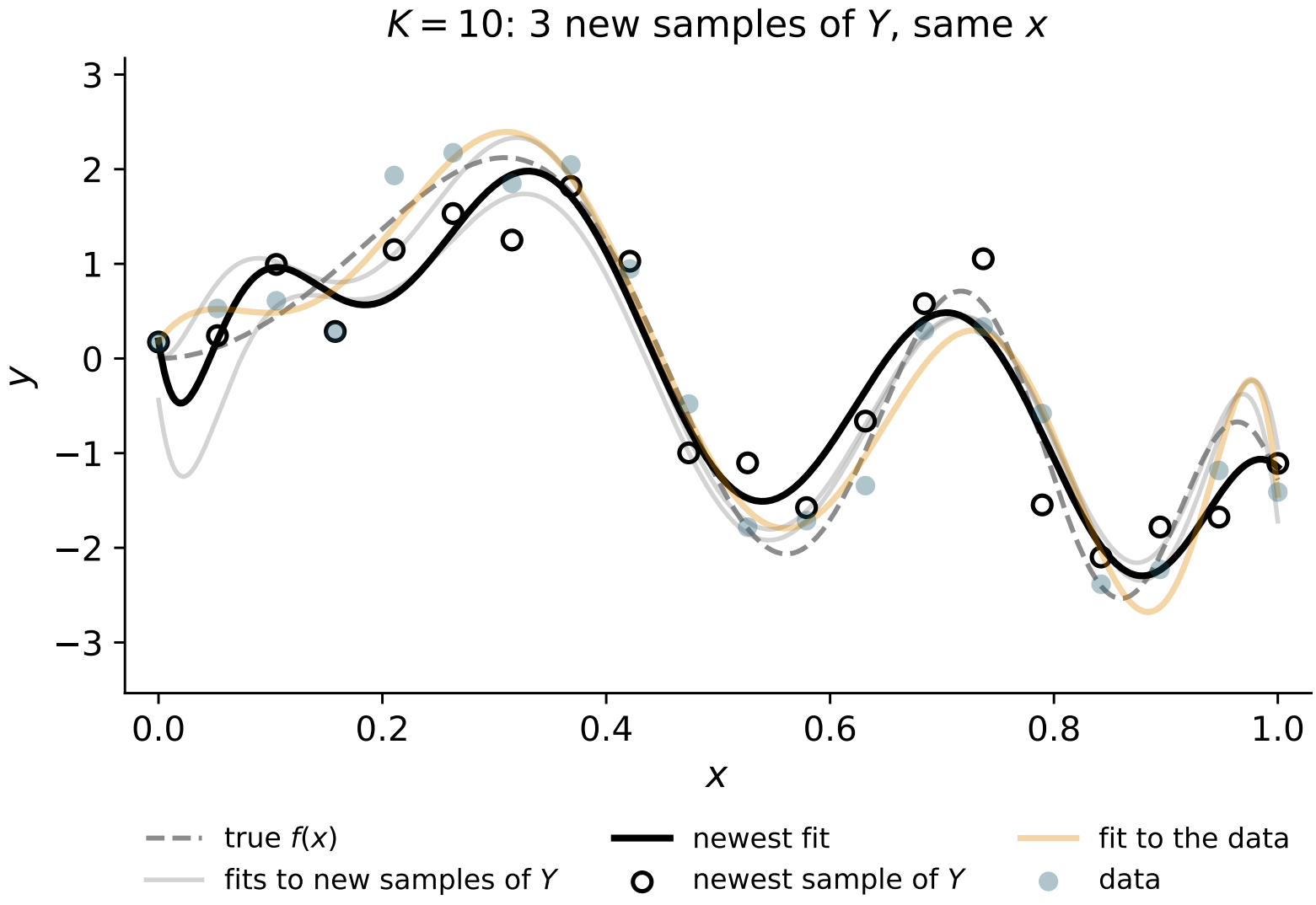
The $K = 10$ fit


$K = 10$: 1 new sample of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
newest fit
newest sample of Y
fit to the data
data

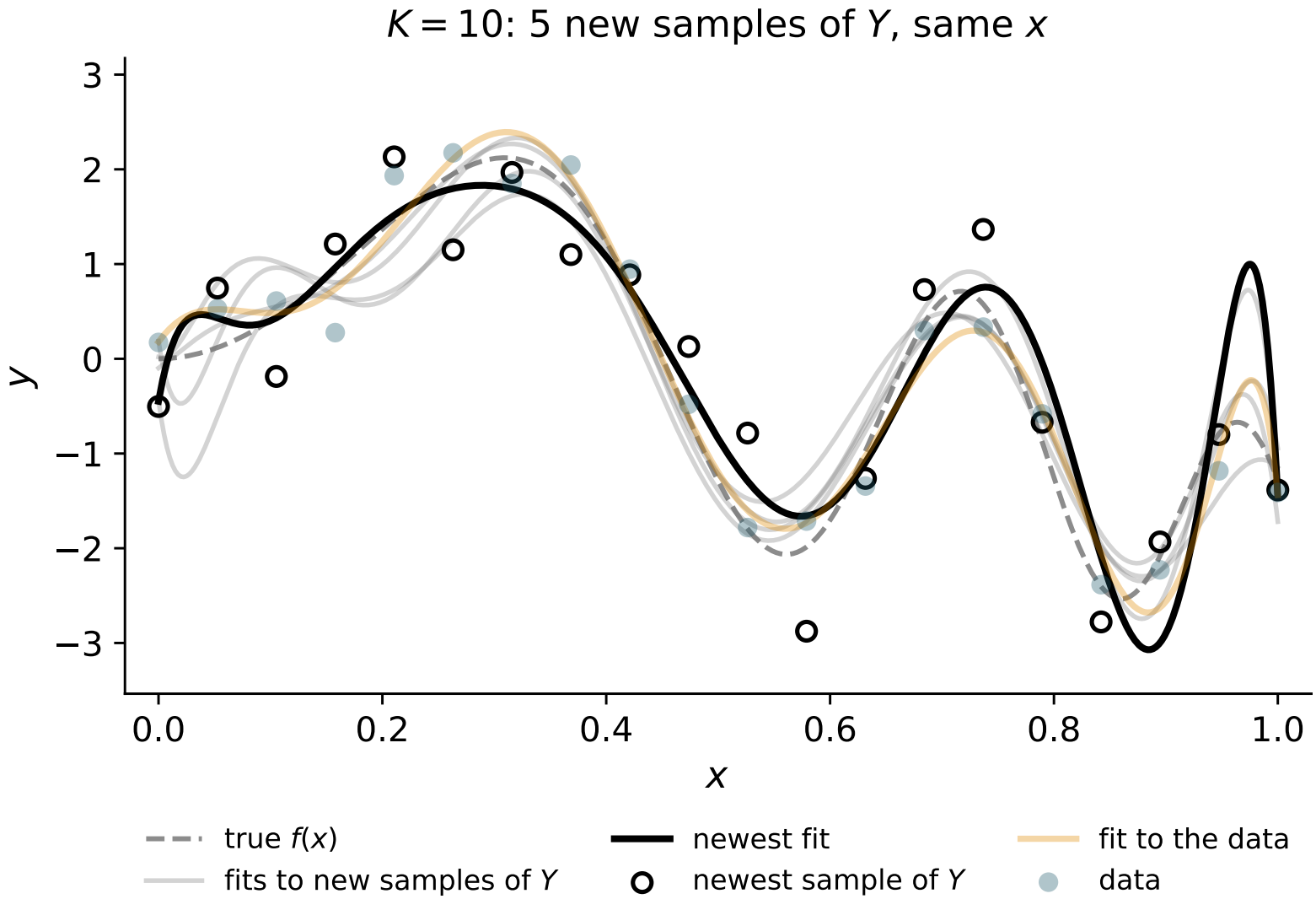
The $K = 10$ fit


$K = 10$: 2 new samples of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
fits to new samples of Y
newest fit
newest sample of Y
fit to the data
data

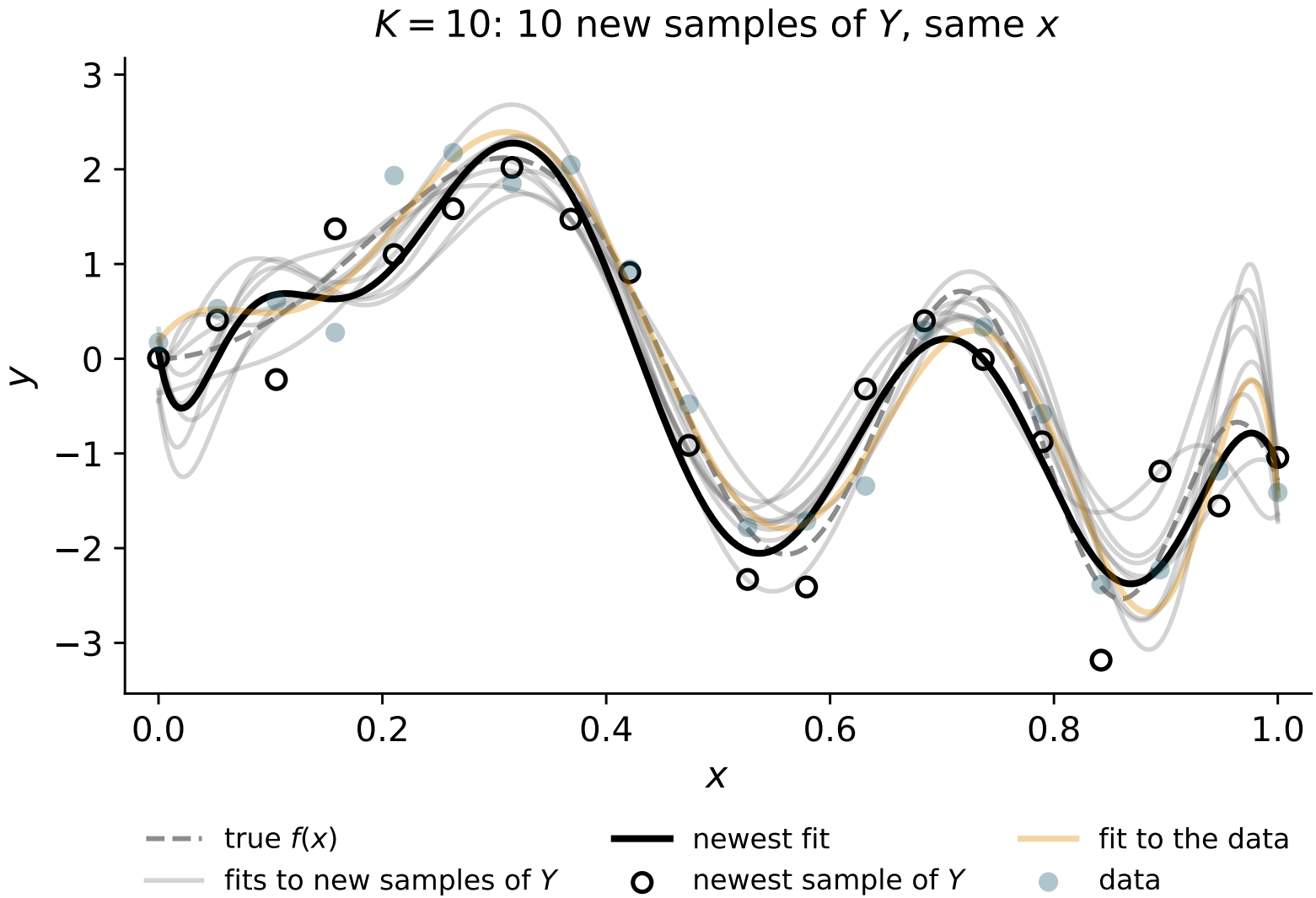
The $K = 10$ fit


$K = 10$: 3 new samples of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
fits to new samples of Y
newest fit
newest sample of Y
fit to the data
data

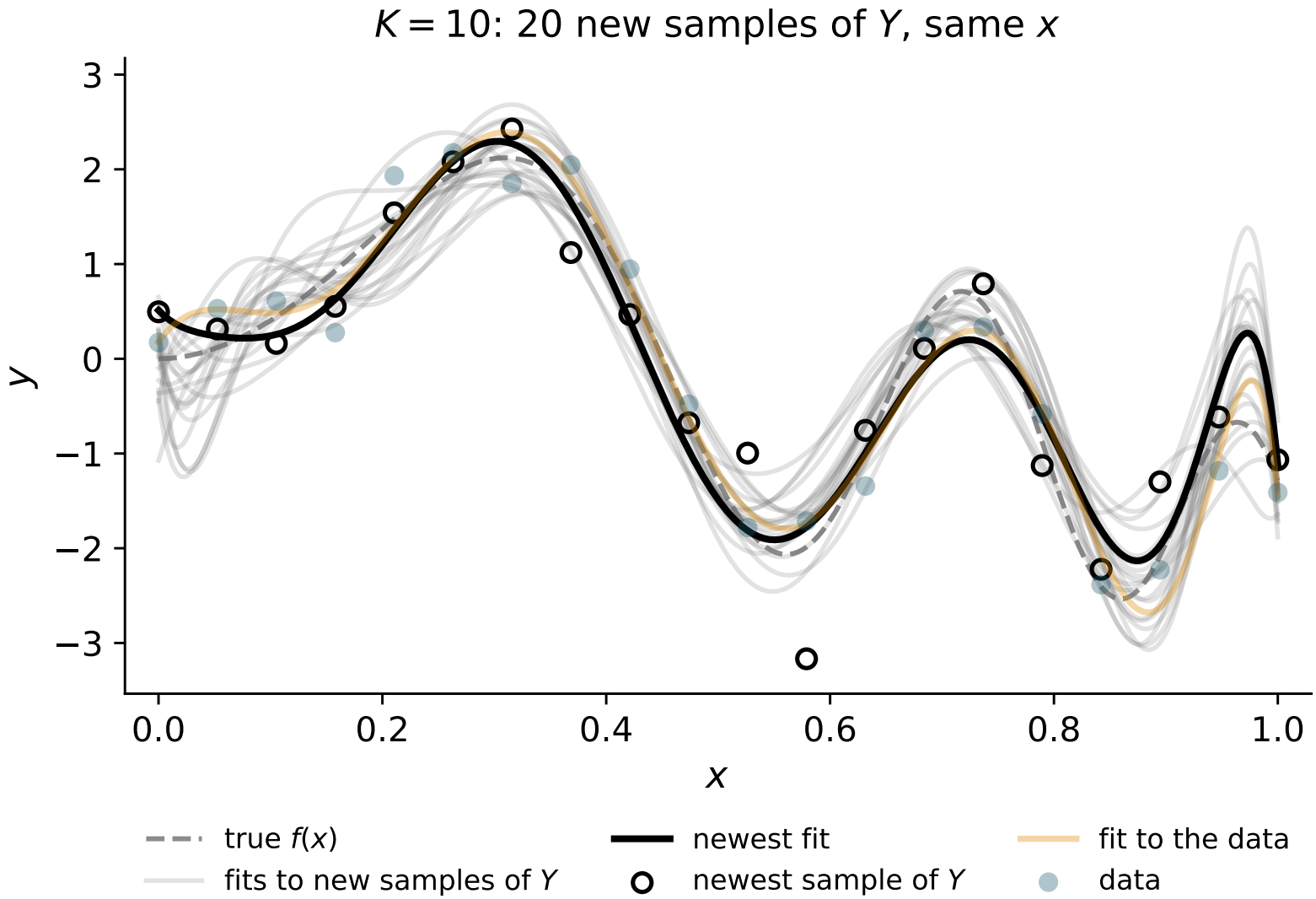
The $K = 10$ fit


$K = 10$: 5 new samples of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
fits to new samples of Y
newest fit
newest sample of Y
fit to the data
data

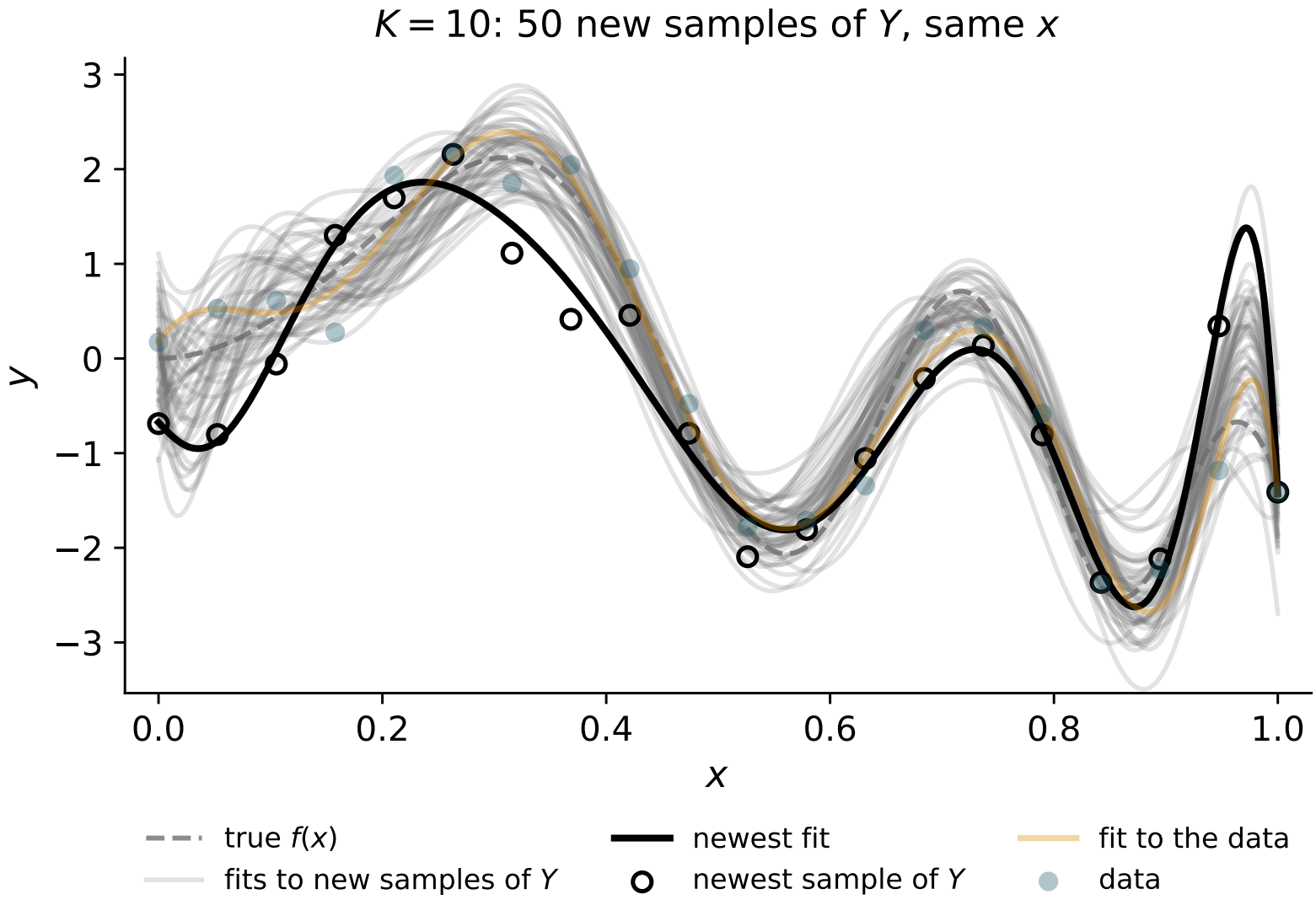
The $K = 10$ fit


$K = 10$: 10 new samples of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
fits to new samples of Y
newest fit
newest sample of Y
fit to the data
data

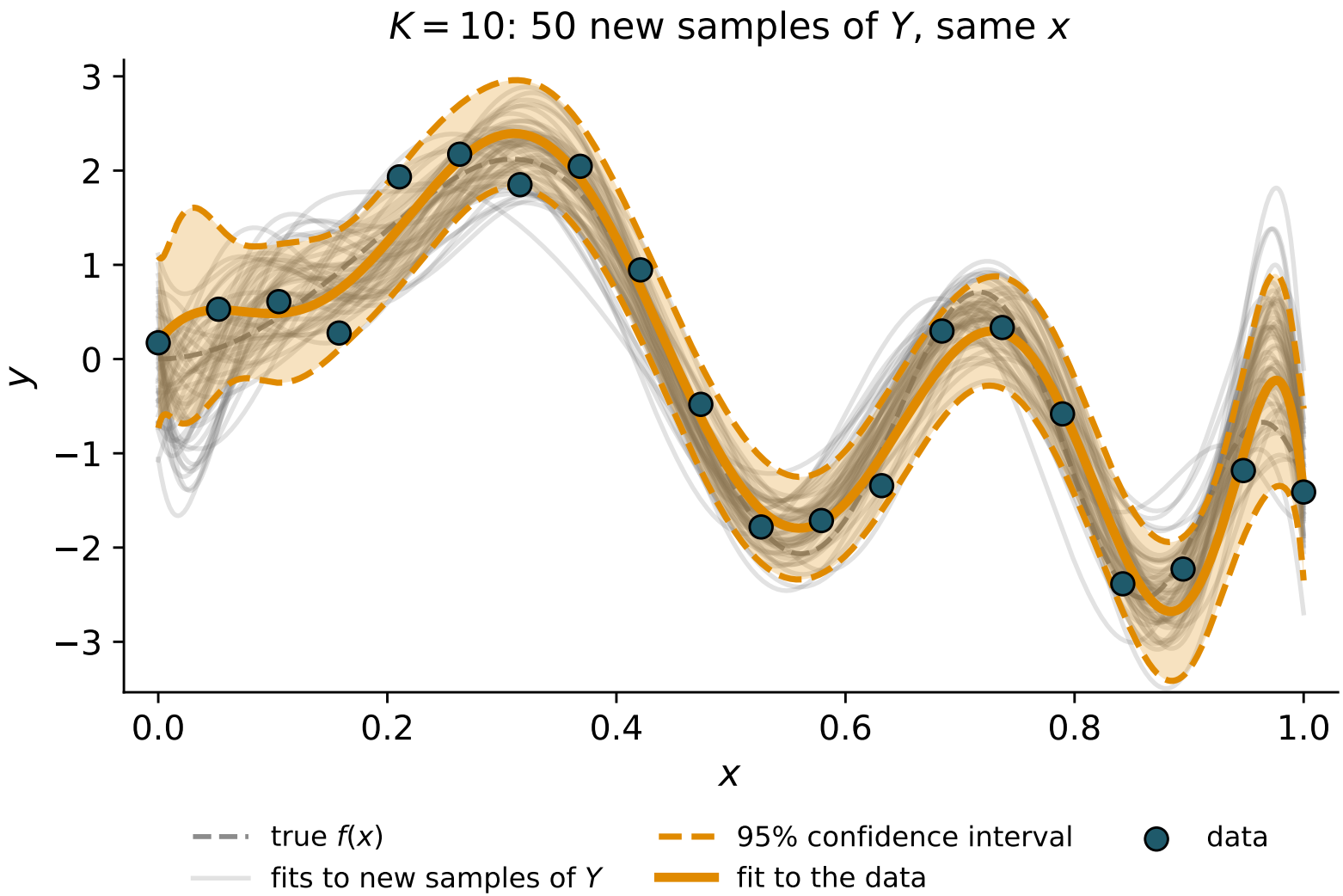
The $K = 10$ fit


$K = 10$: 20 new samples of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
fits to new samples of Y
newest fit
newest sample of Y
fit to the data
data

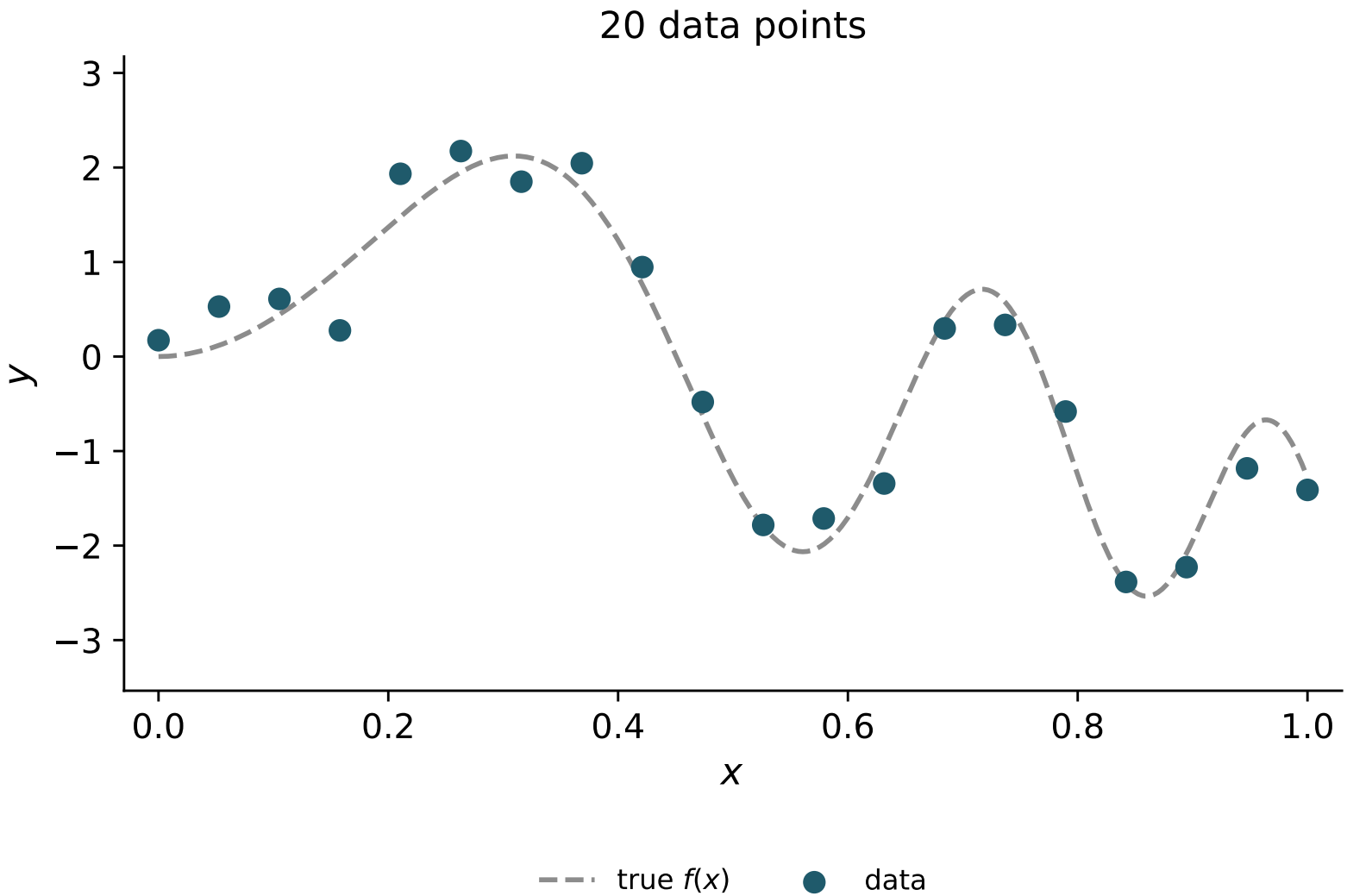
The $K = 10$ fit


$K = 10$: 50 new samples of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
fits to new samples of Y
newest fit
newest sample of Y
fit to the data
data

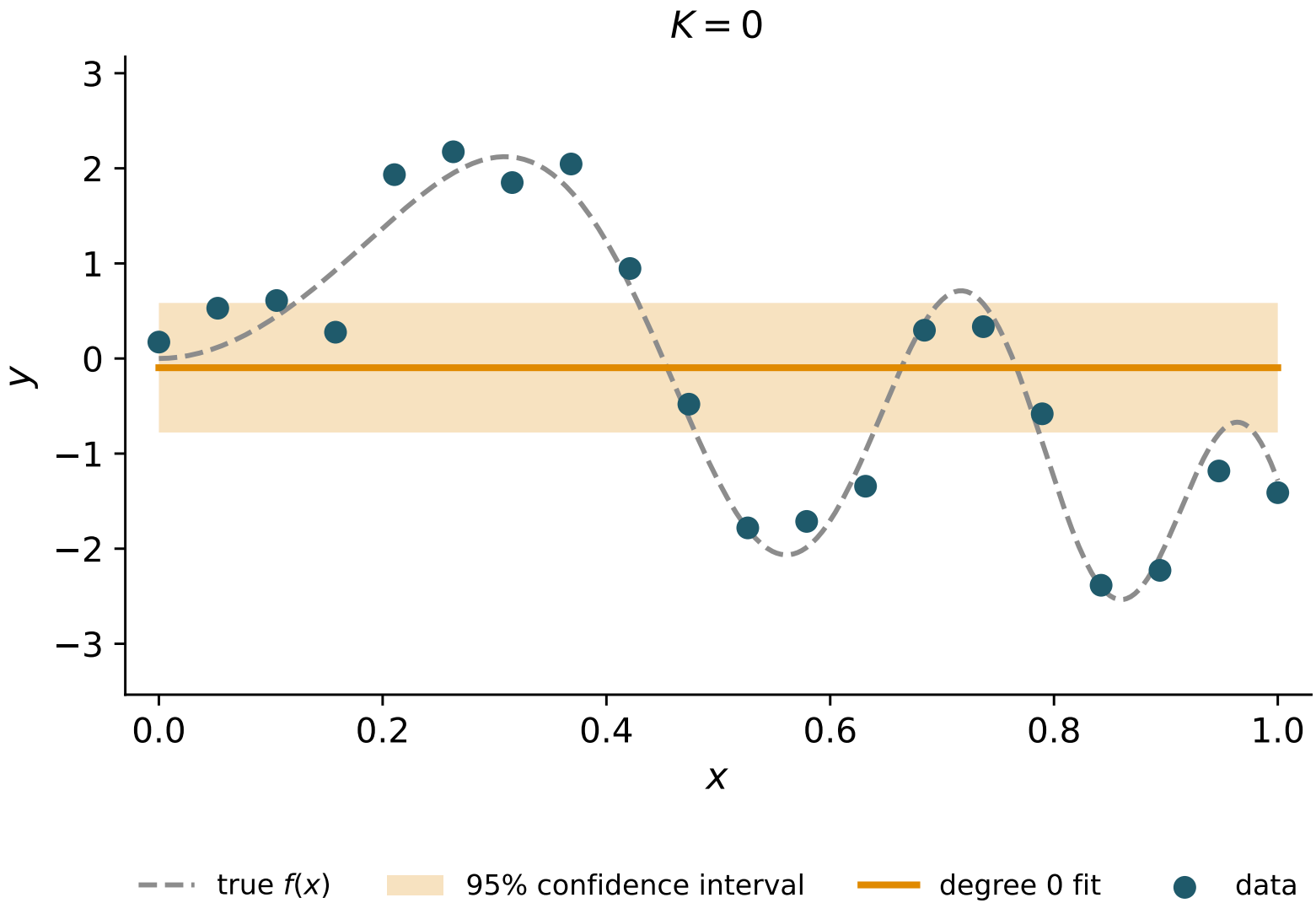
The $K = 10$ fit


$K = 10$: 50 new samples of Y, same x
3
2
1
0
−1
−2
−3
y
0.0
0.2
0.4
0.6
0.8
1.0
x
true $f(x)$
fits to new samples of Y
95% confidence interval
fit to the data
data

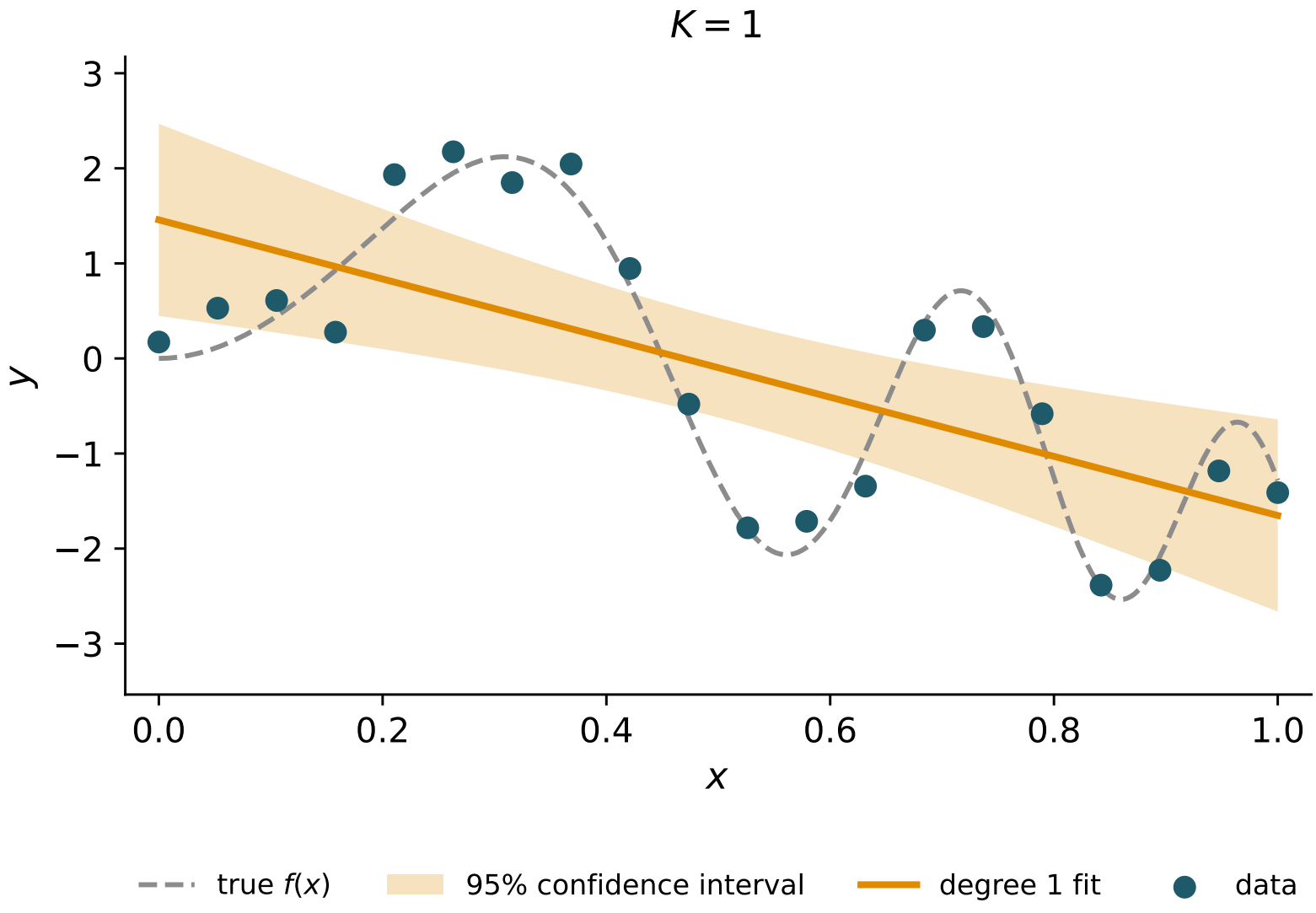
Polynomial fits for different K



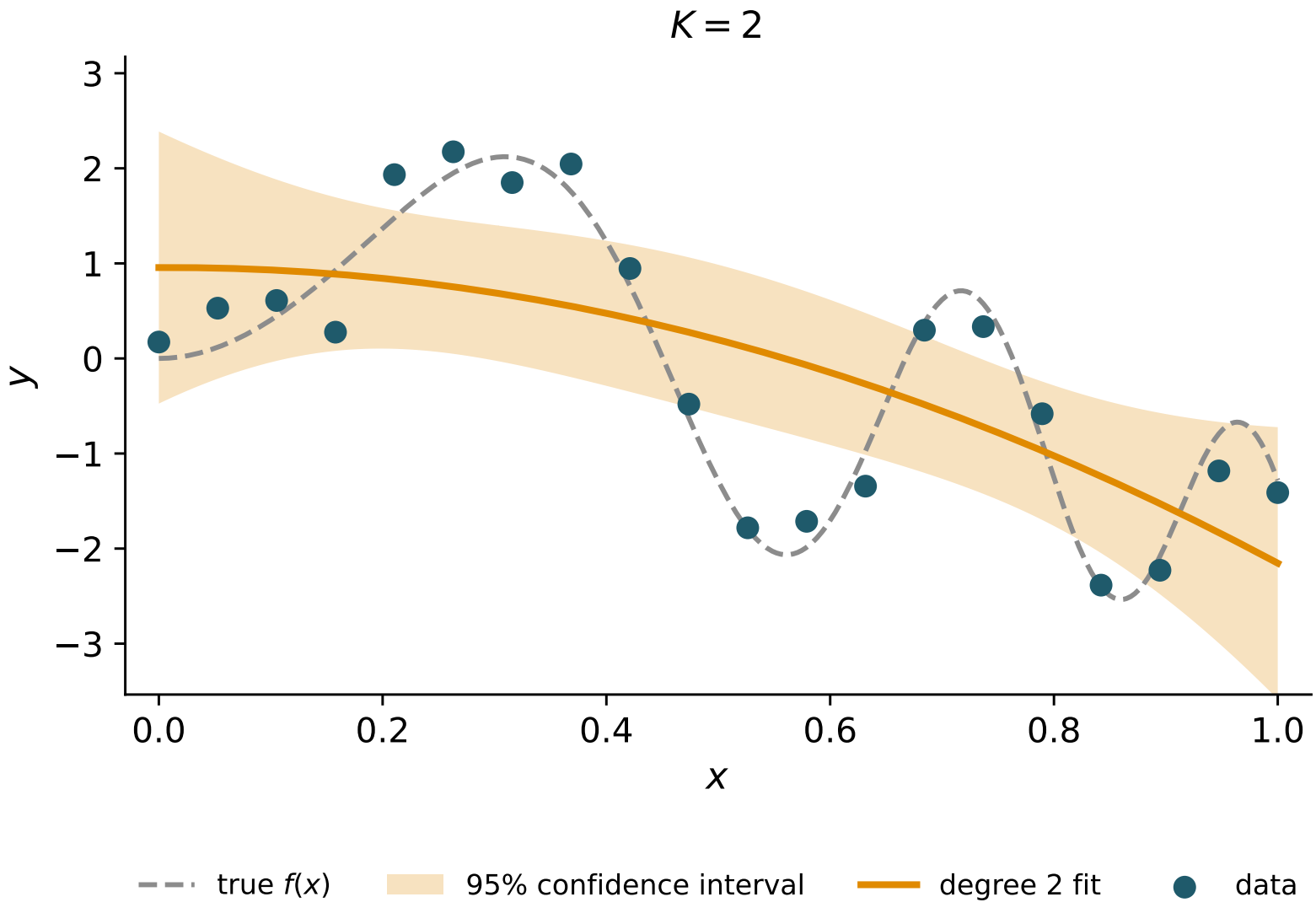
Polynomial fits for different K



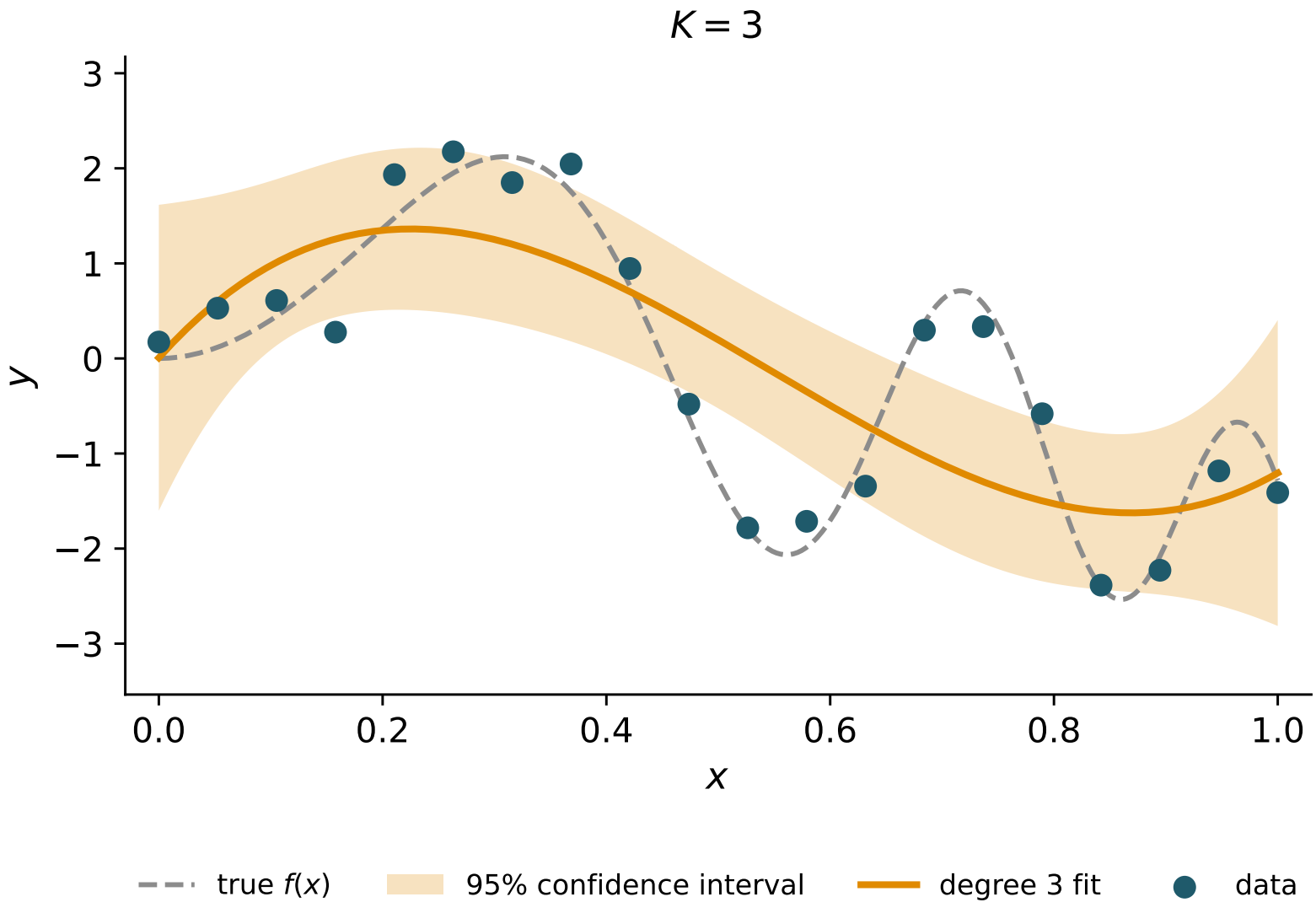
Polynomial fits for different K



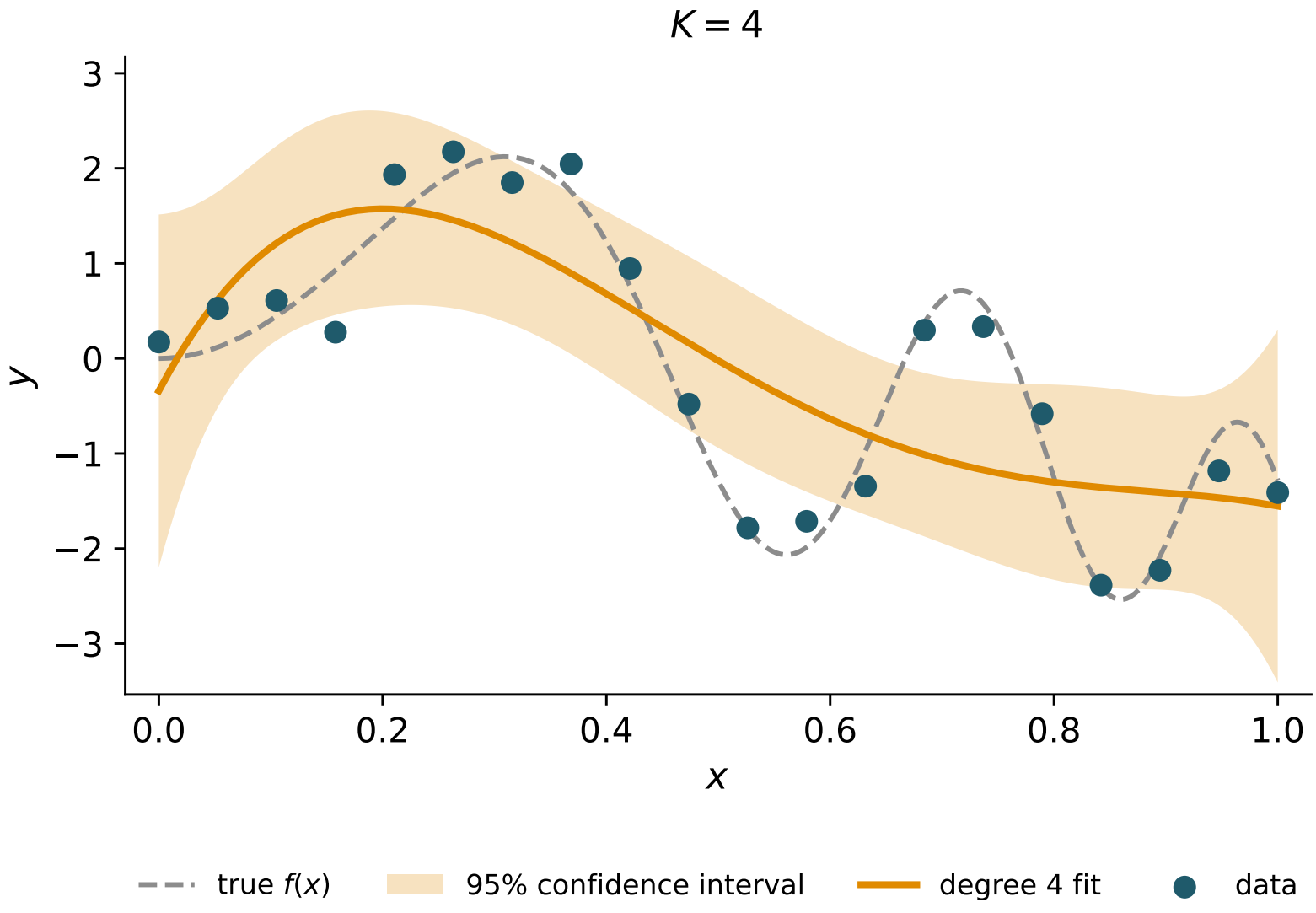
Polynomial fits for different K



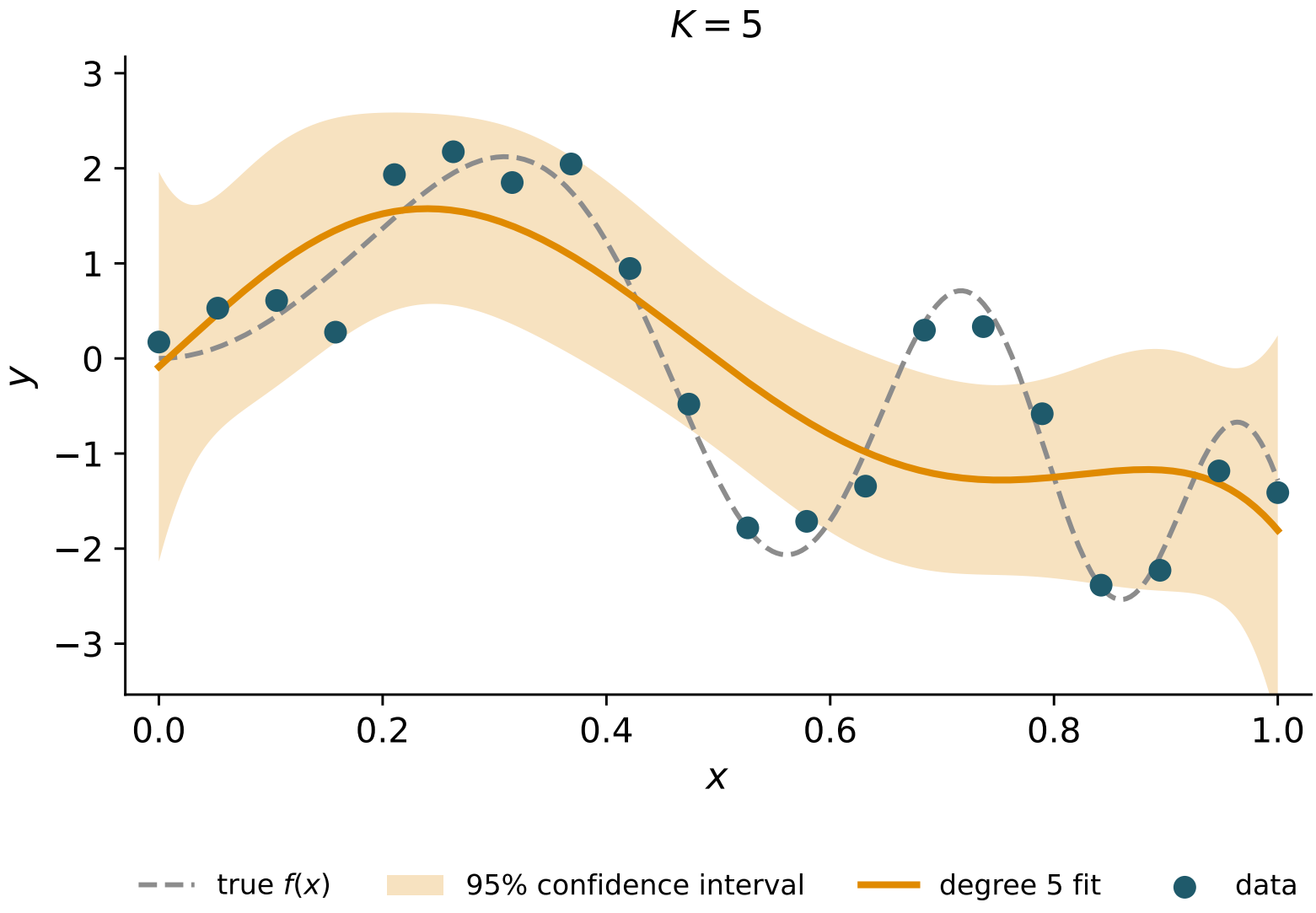
Polynomial fits for different K



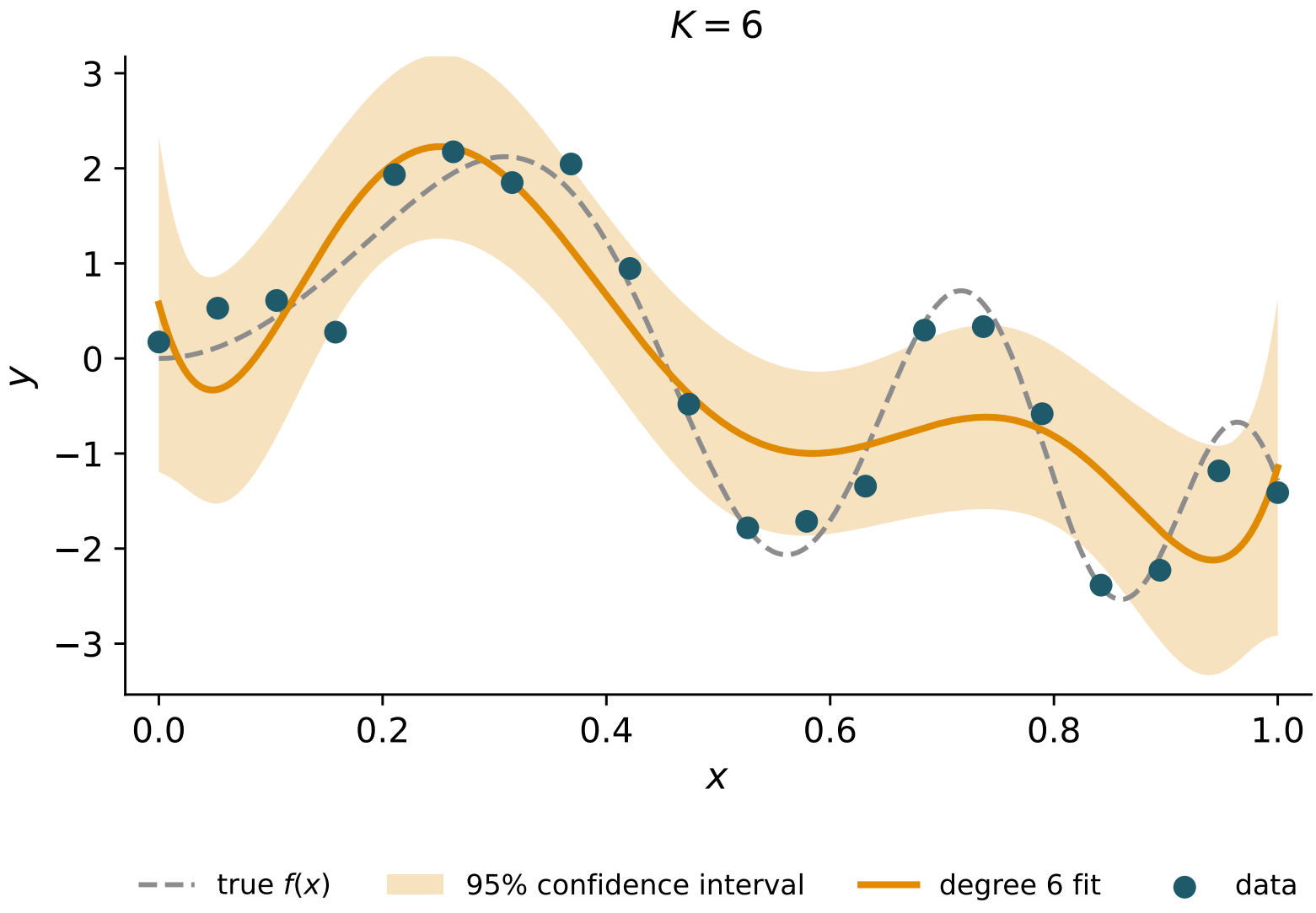
Polynomial fits for different K



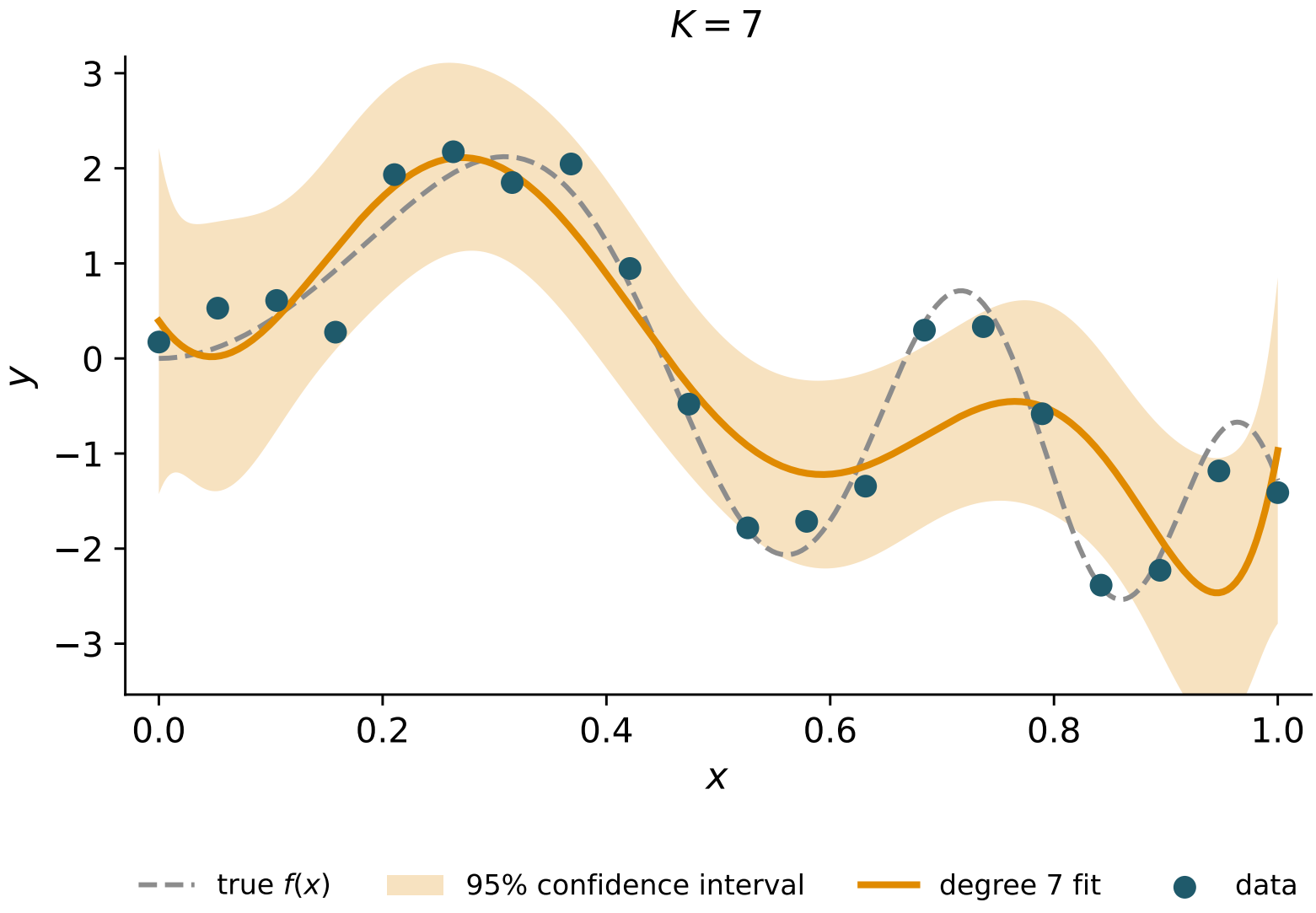
Polynomial fits for different K



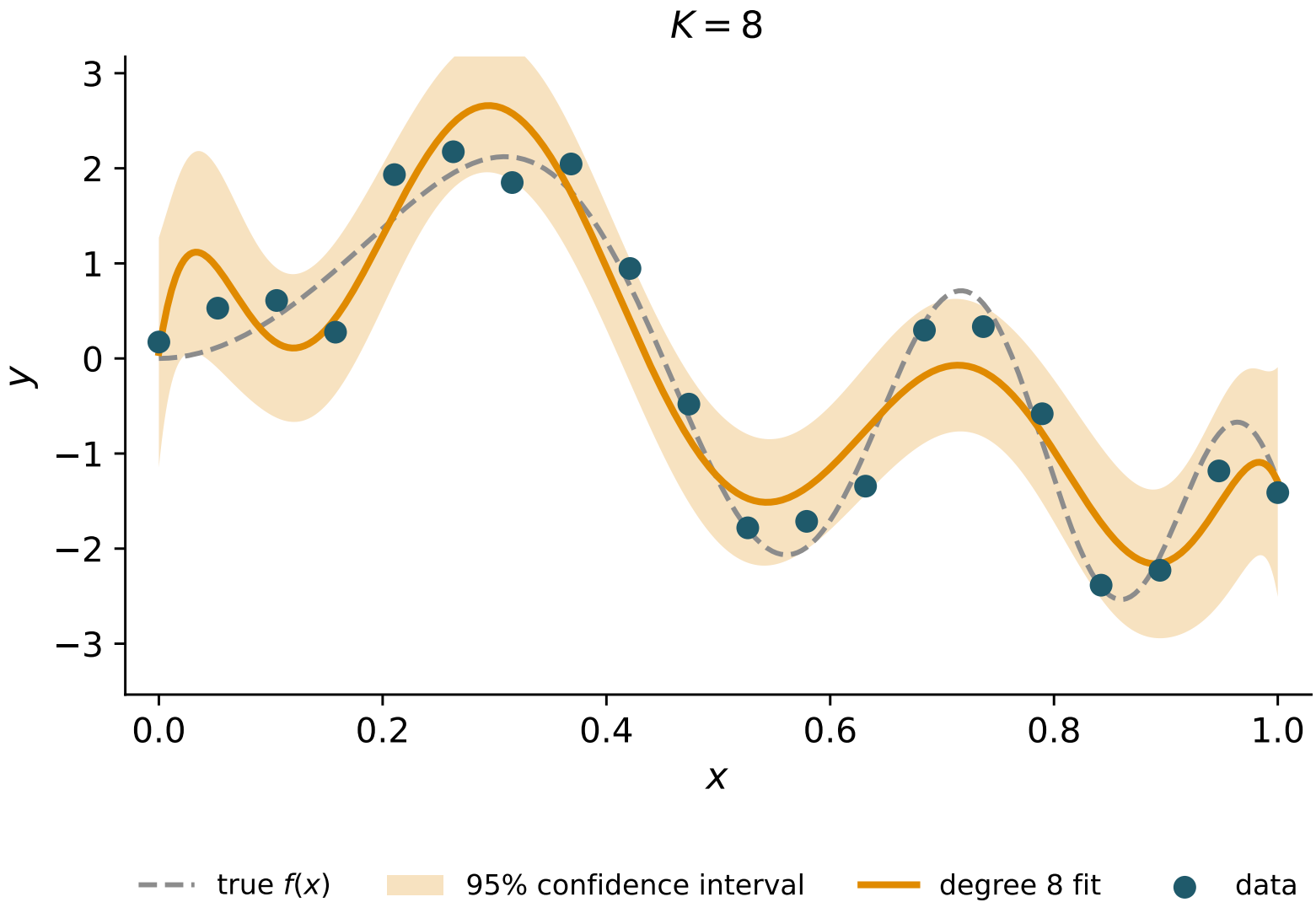
Polynomial fits for different K



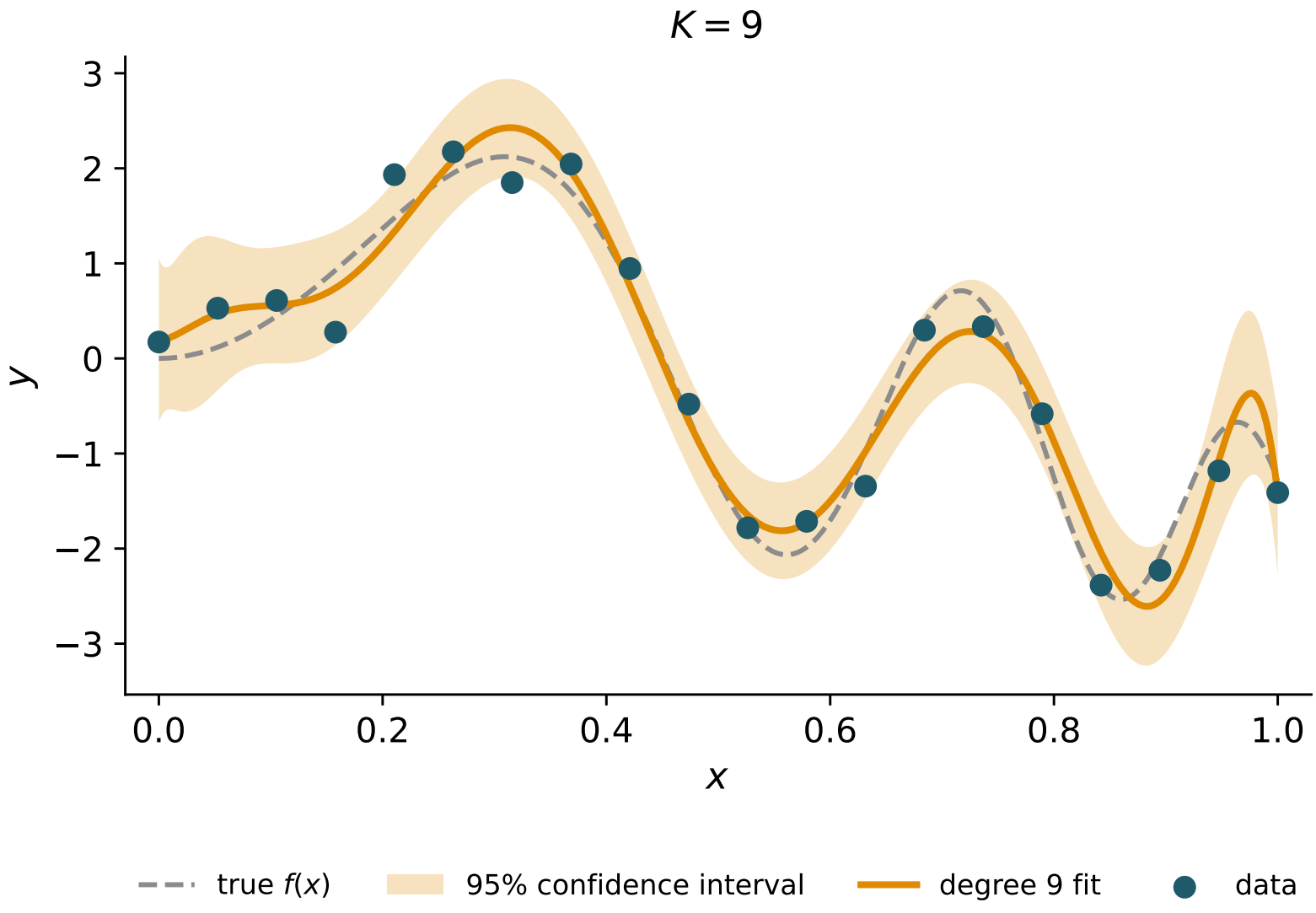
Polynomial fits for different K



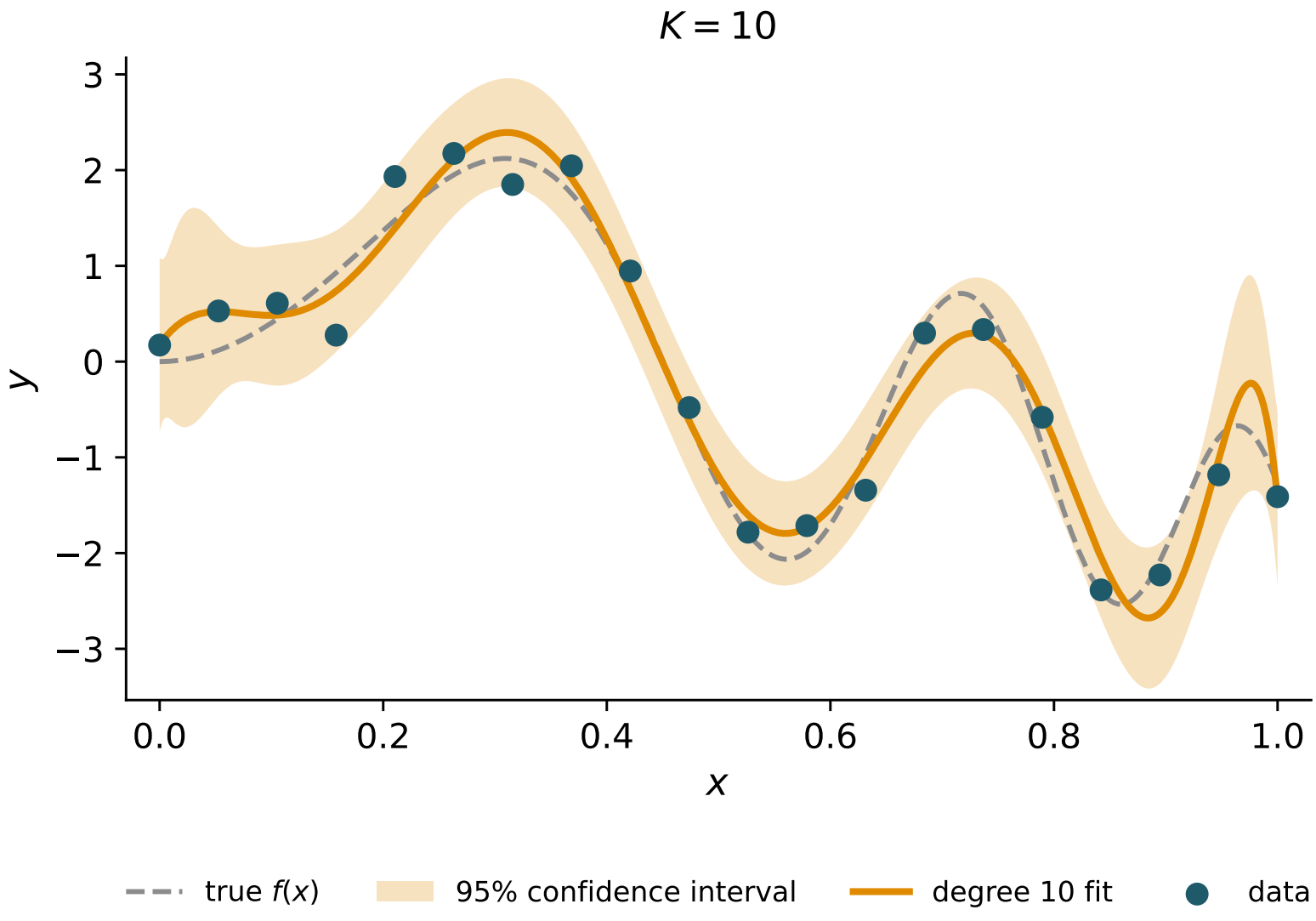
Polynomial fits for different K



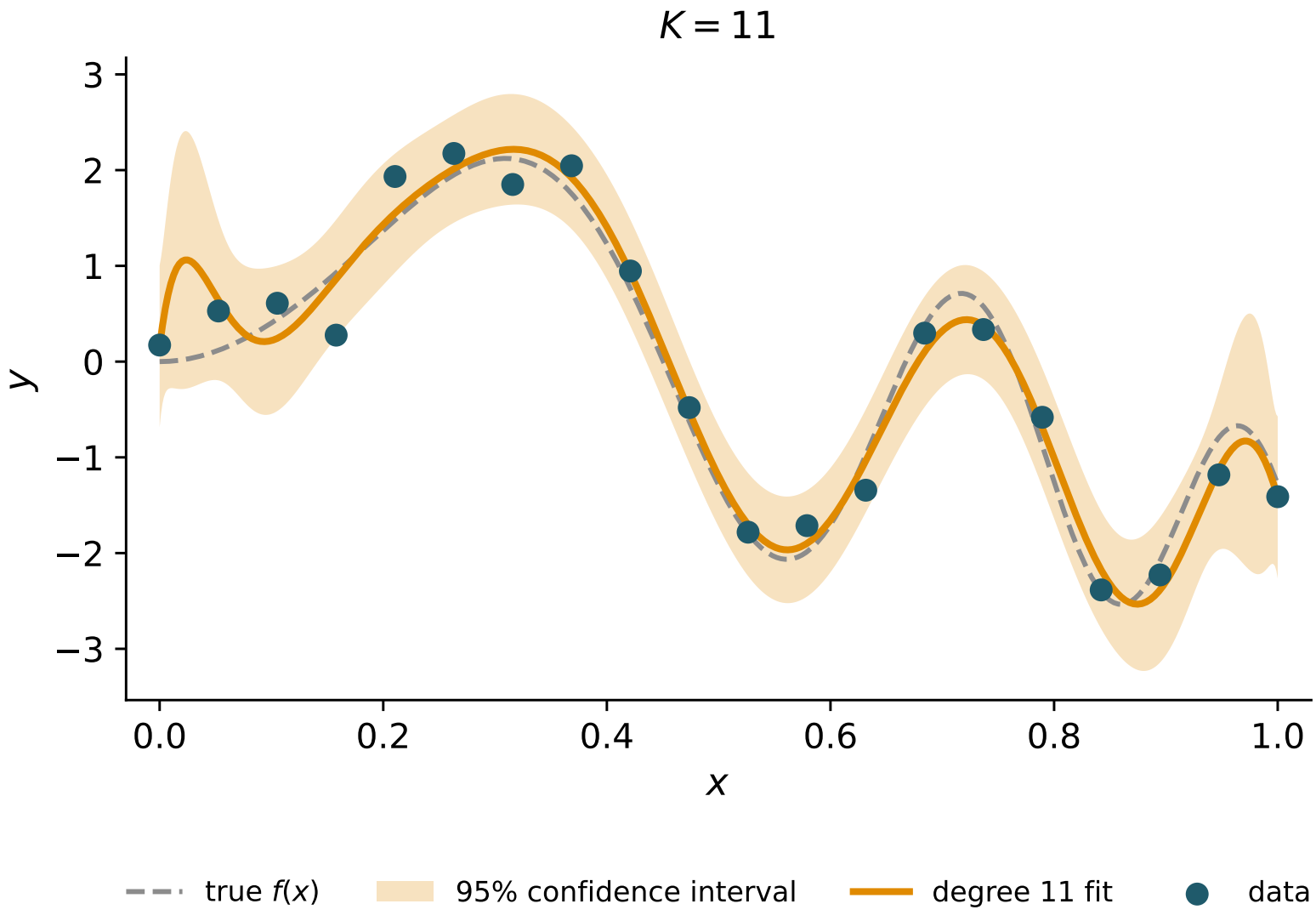
Polynomial fits for different K



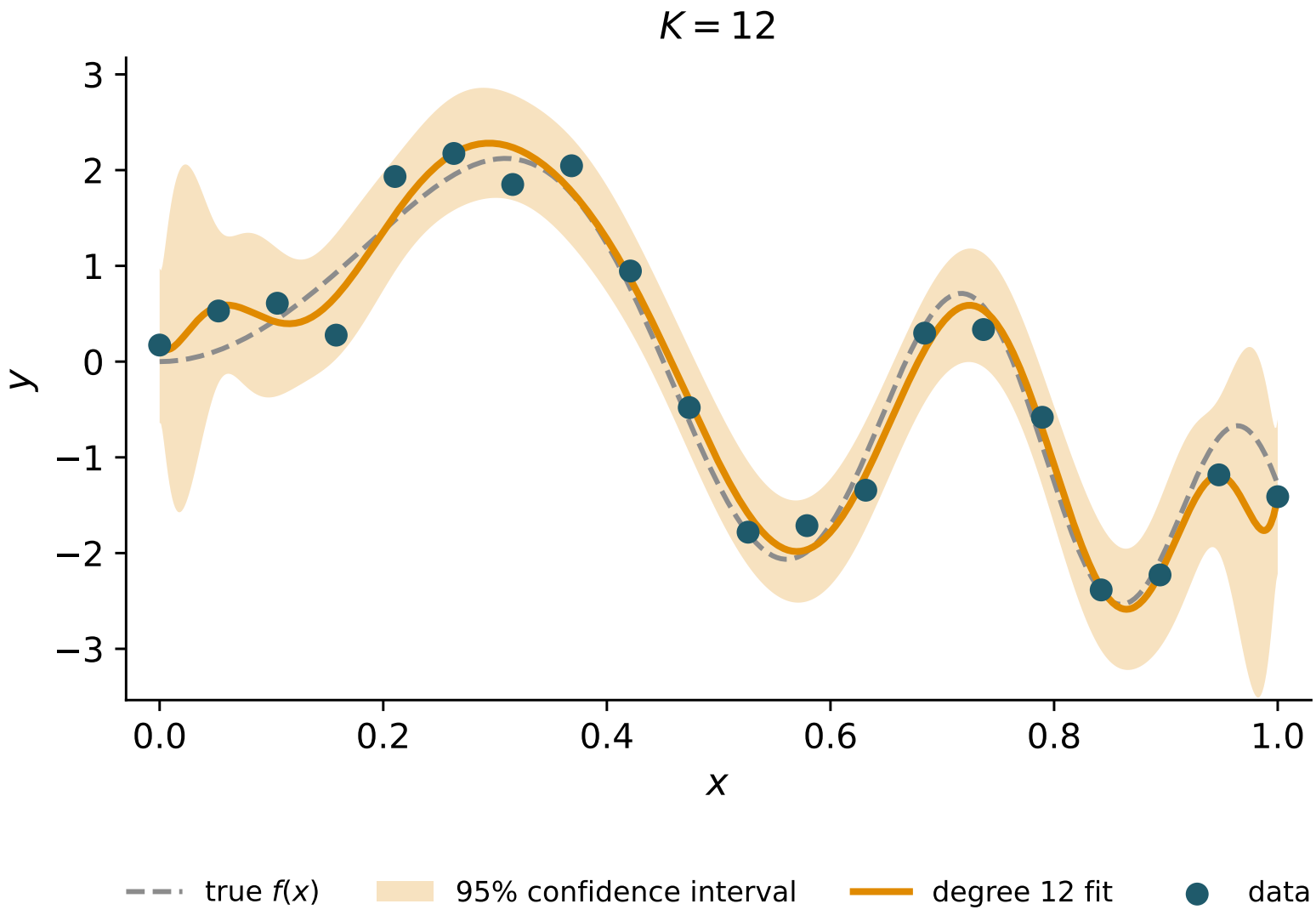
Polynomial fits for different K



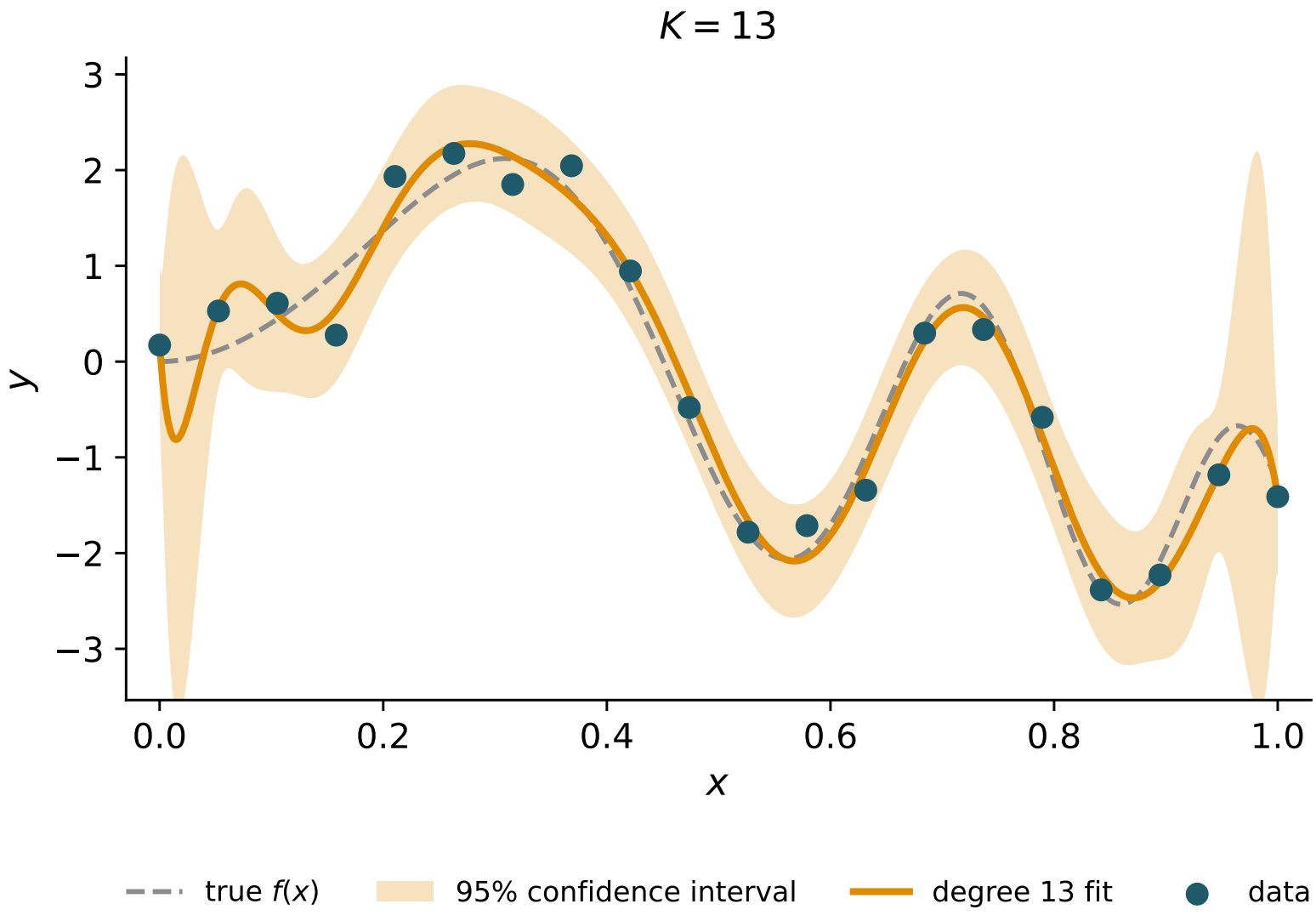
Polynomial fits for different K



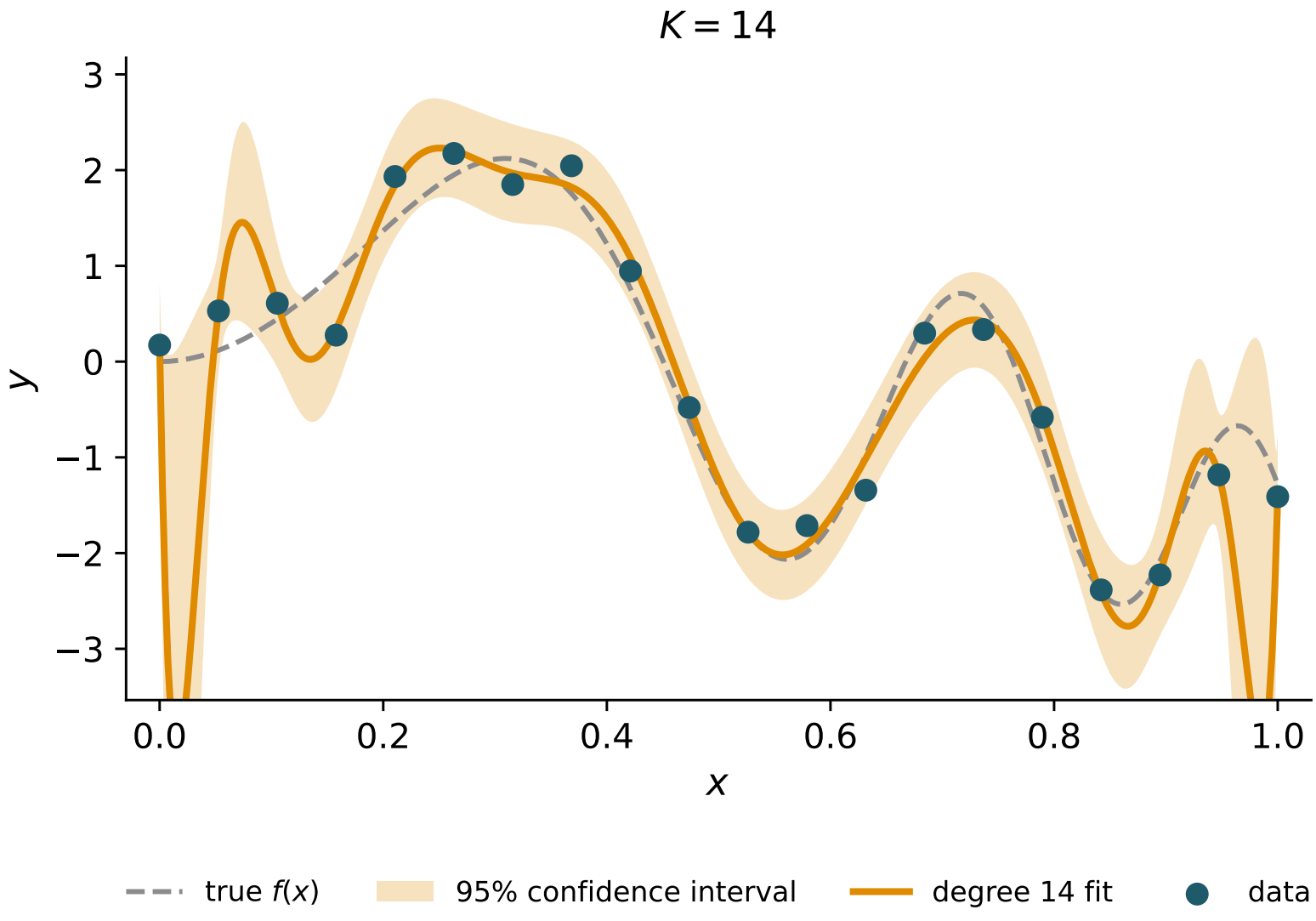
Polynomial fits for different K



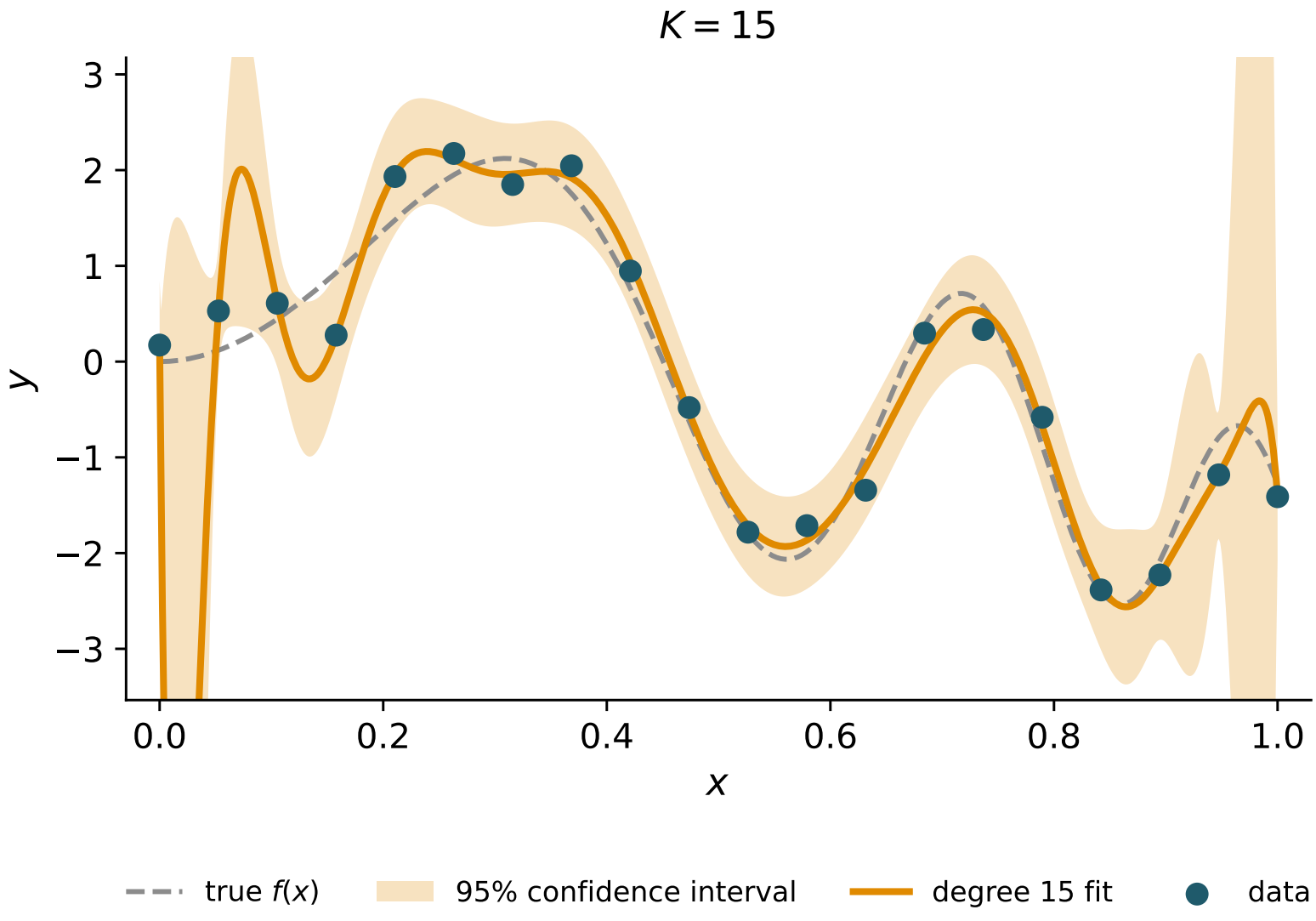
Polynomial fits for different K



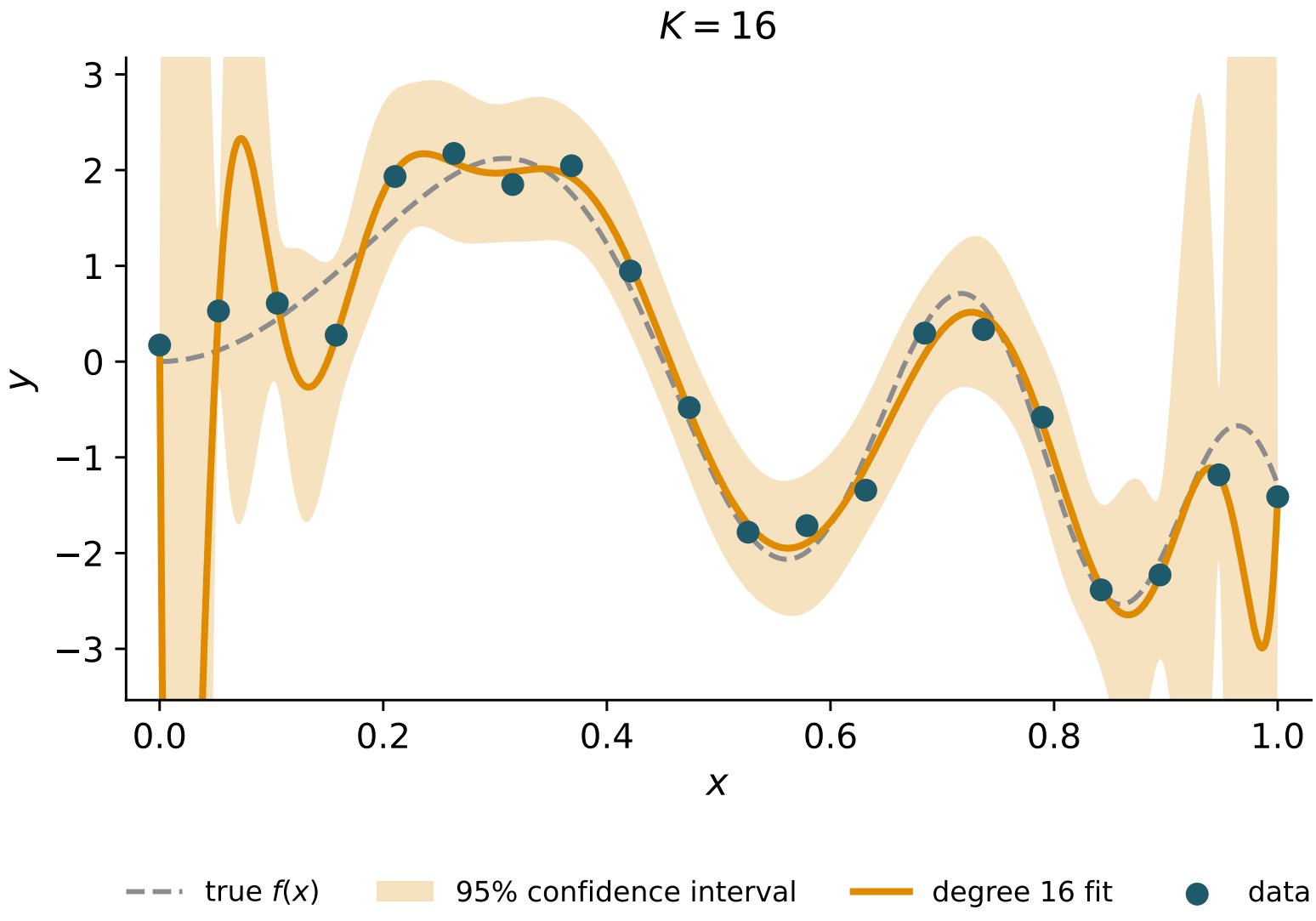
Polynomial fits for different K



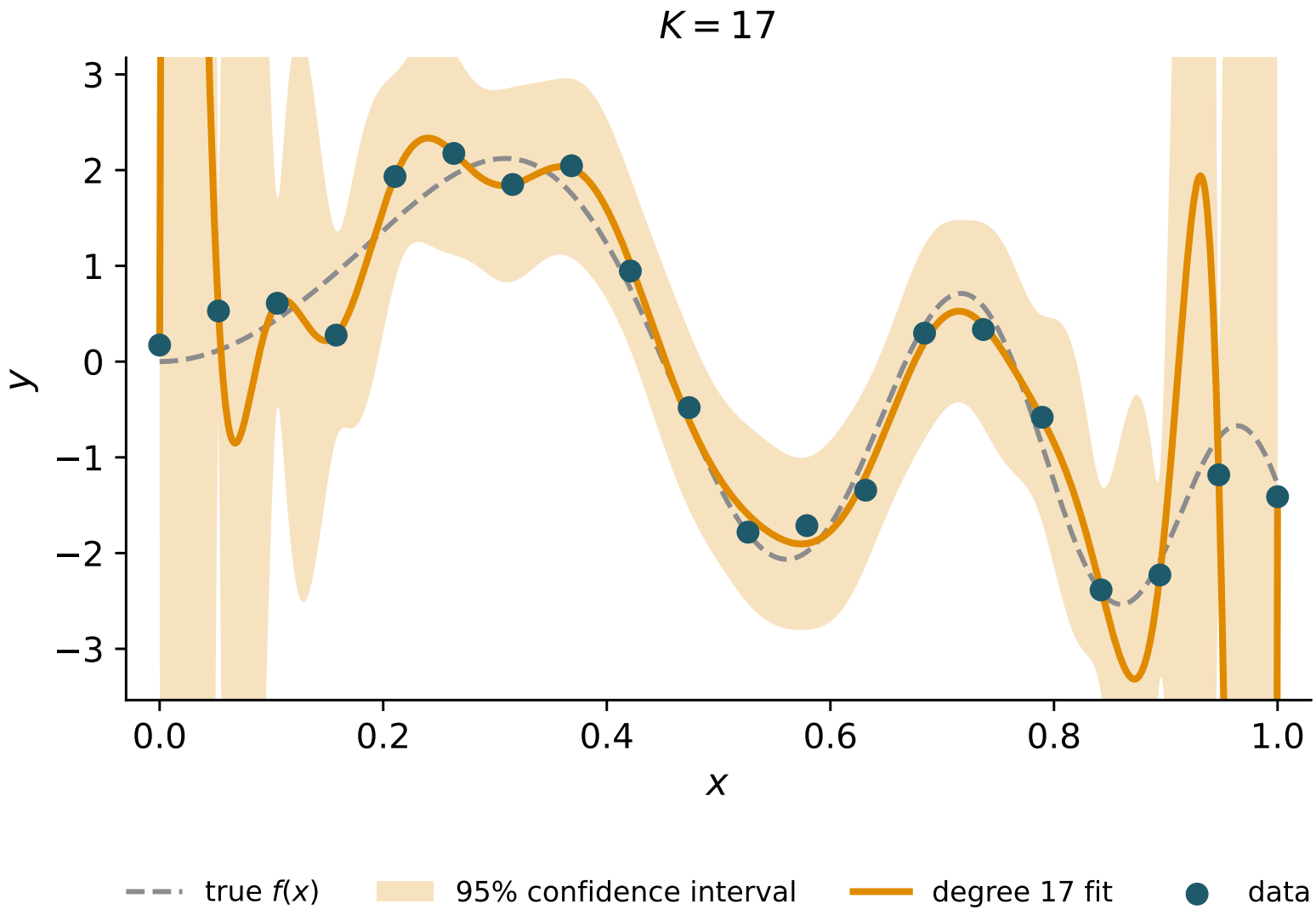
Polynomial fits for different K



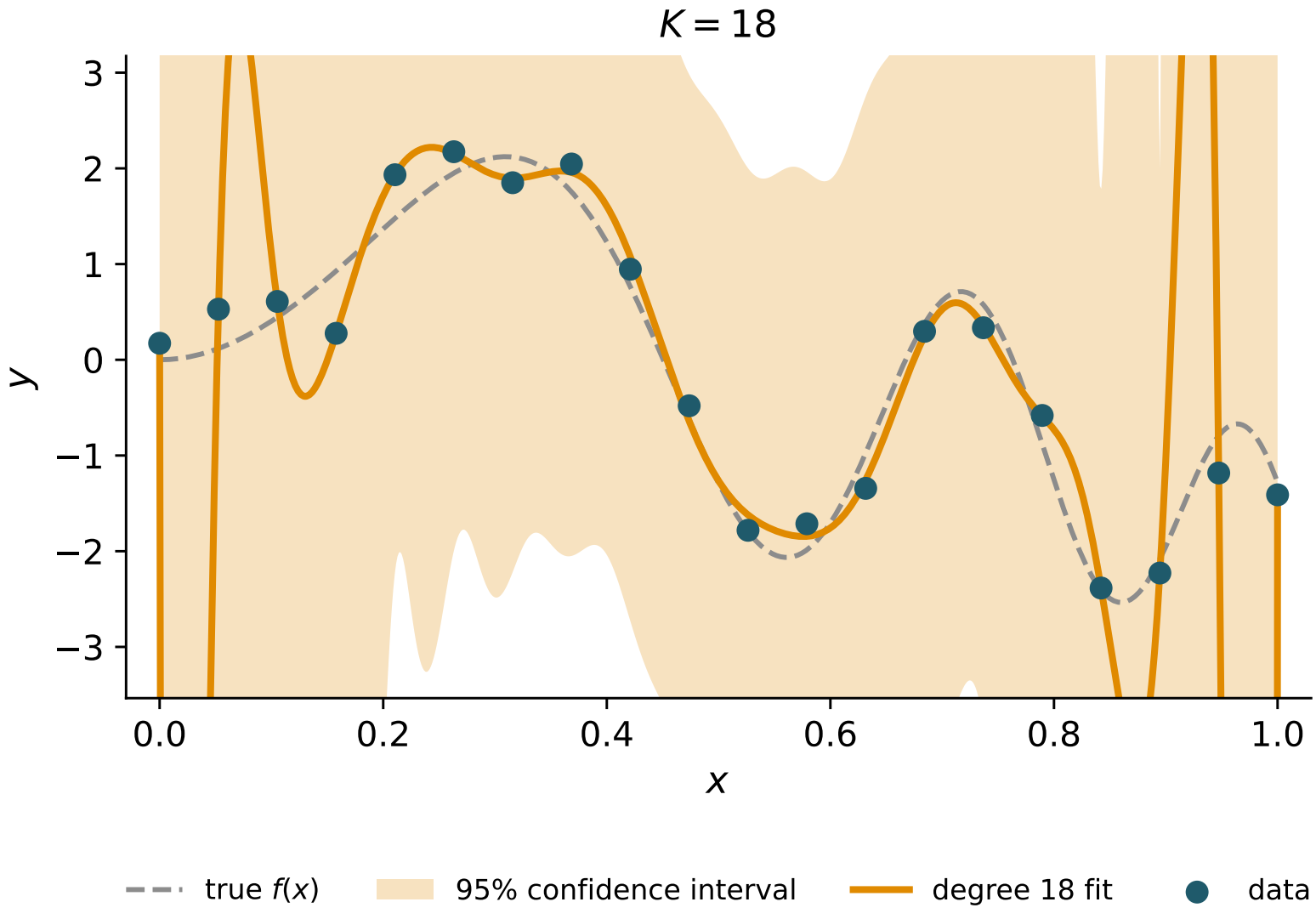
Polynomial fits for different K



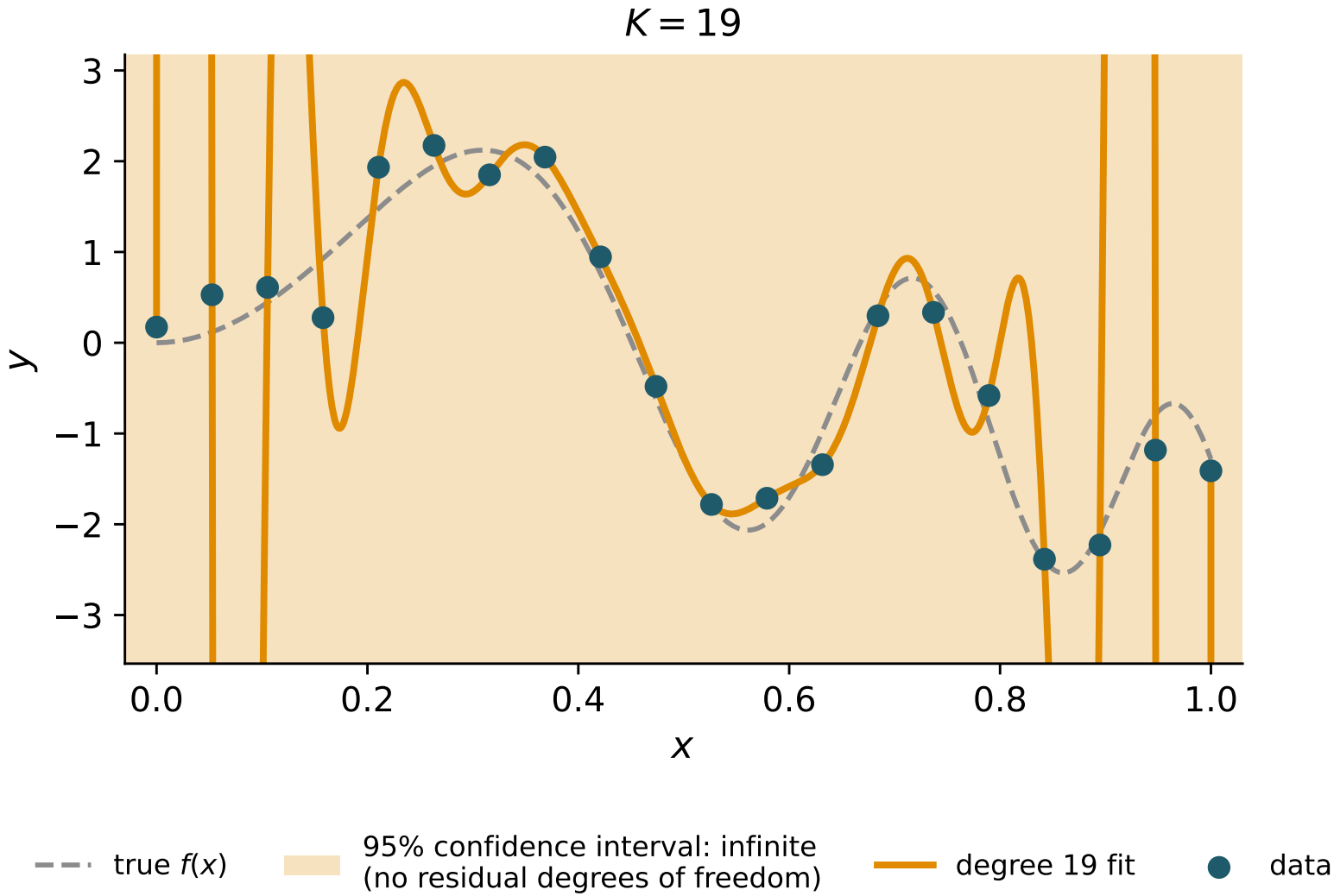
Polynomial fits for different K



Polynomial fits for different K

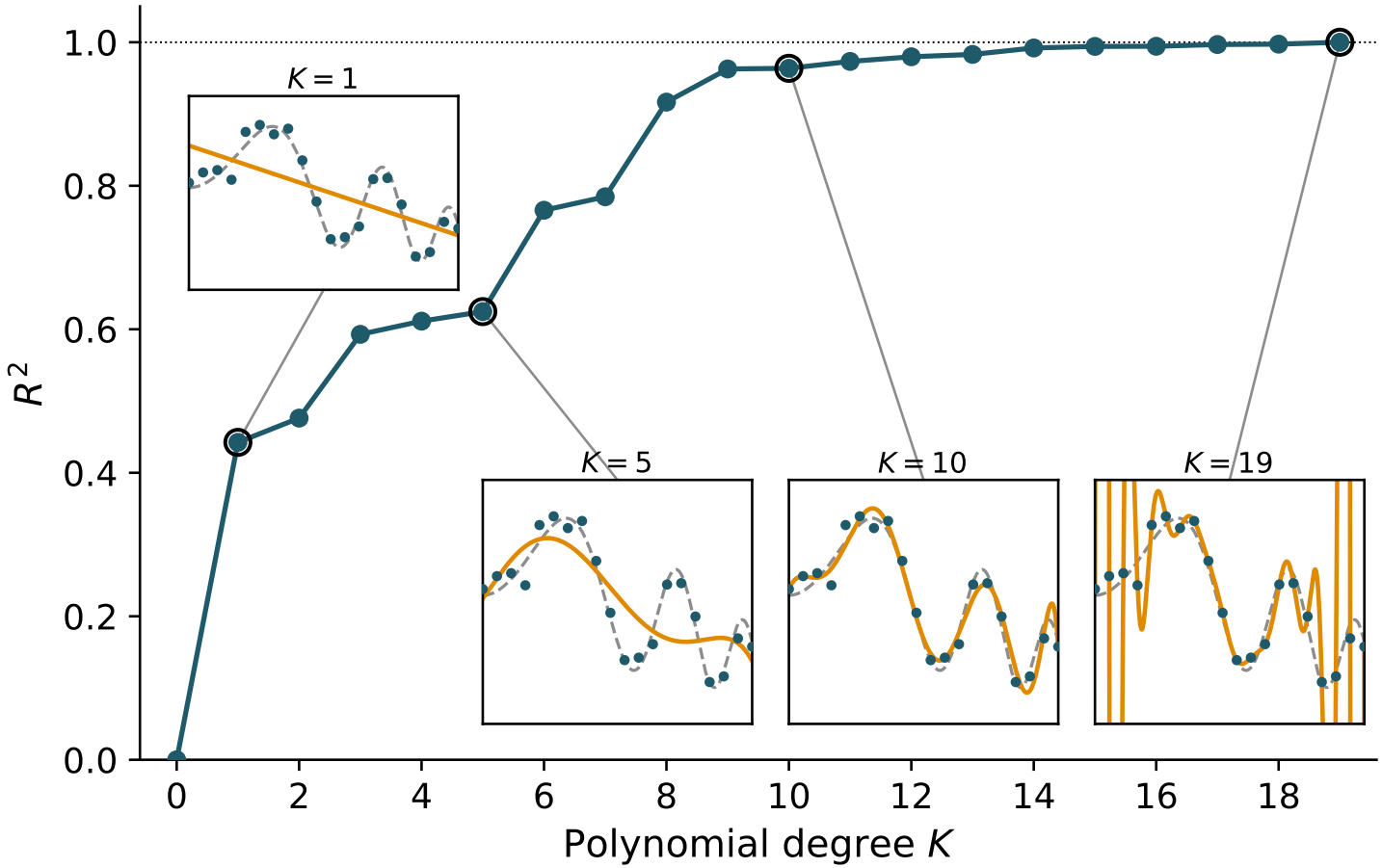


Polynomial fits for different K

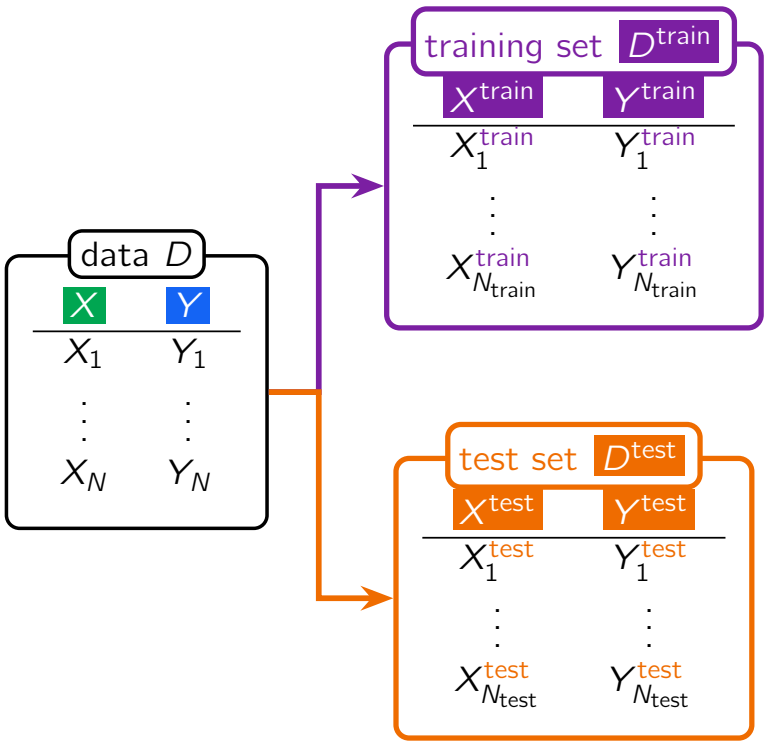


R^2 as the degree increases

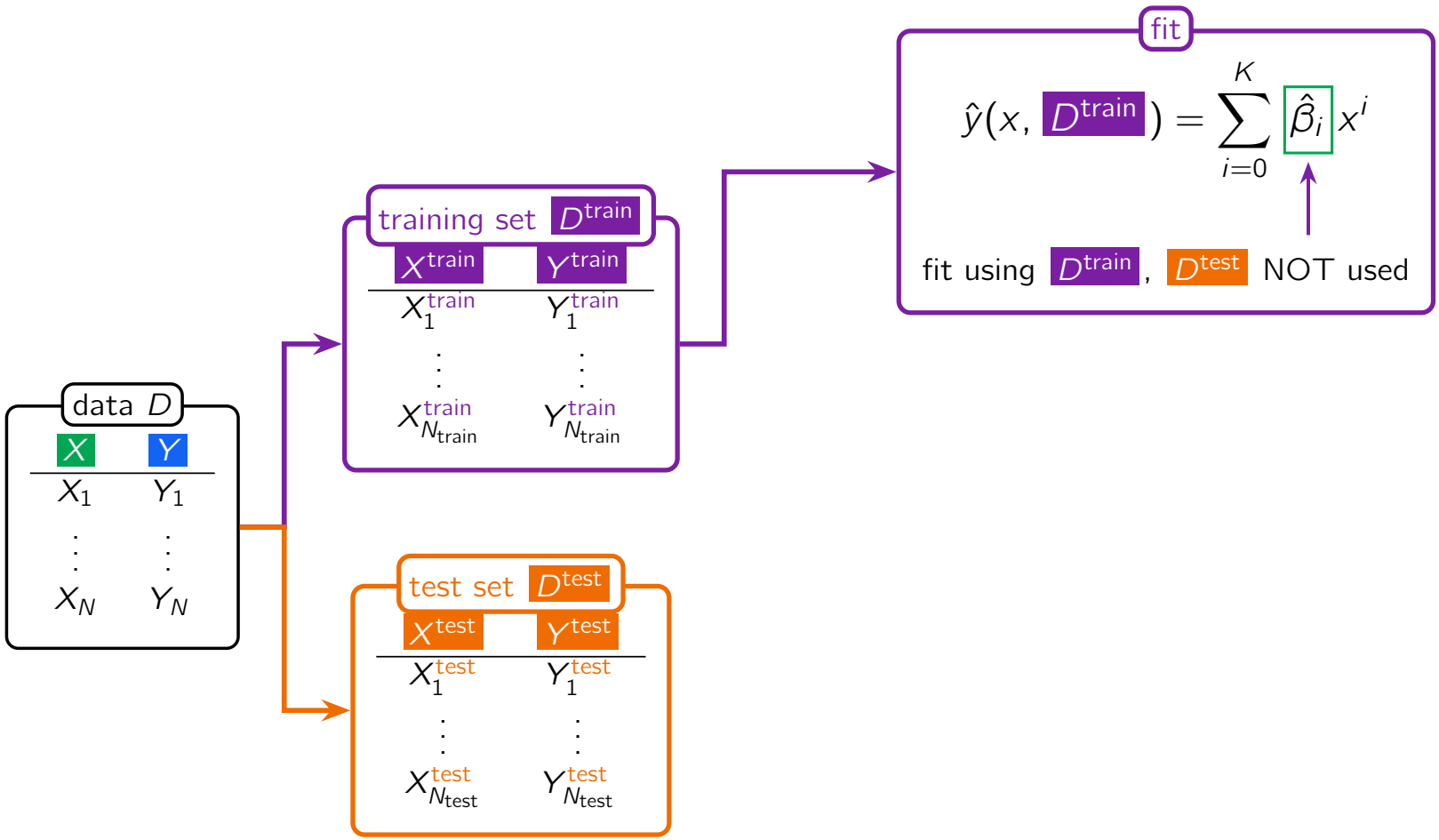
We already know that R^2 can only decrease as we add predictors, and since increasing K is adding predictors, this applies to our polynomial regression:



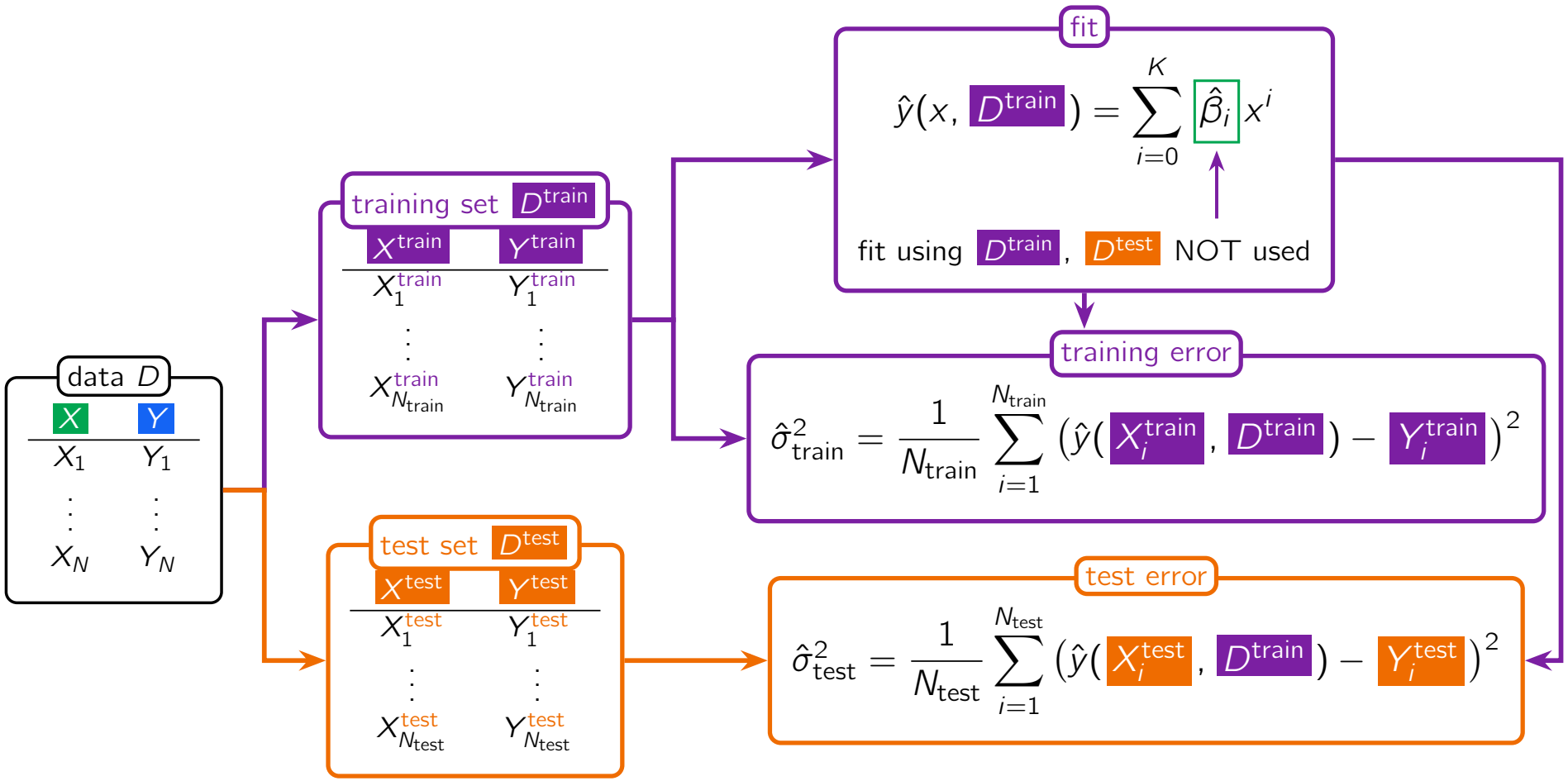
Test/training split



Test/training split



Test/training split



Relationship between training error and R^2

Consider the residuals $r_i^{\text{train}} = Y_i^{\text{train}} - \hat{y}(X_i^{\text{train}}, D^{\text{train}})$ for the training points. Then, when N_{train} is very large

$$R^2 = 1 - \frac{\sum_{i=1}^{N} r_i^2}{\sum_{i=1}^{N} \left(Y_i - \overline{Y}\right)^2}$$

Relationship between training error and R^2

Consider the residuals $r_i^{\text{train}} = Y_i^{\text{train}} - \hat{y}(X_i^{\text{train}}, D^{\text{train}})$ for the training points. Then, when N_{train} is very large

$$R^2 = 1 - \frac{\sum_{i=1}^{N} r_i^2}{\sum_{i=1}^{N} \left(Y_i - \overline{Y}\right)^2} \approx 1 - \frac{\sum_{i=1}^{N_{\text{train}}} (r_i^{\text{train}})^2}{\sum_{i=1}^{N_{\text{train}}} \left(Y_i^{\text{train}} - \overline{Y}\right)^2}$$

Relationship between training error and R^2

Consider the residuals $r_i^{\text{train}} = Y_i^{\text{train}} - \hat{y}(X_i^{\text{train}}, D^{\text{train}})$ for the training points. Then, when N_{train} is very large

$$
\begin{aligned}
R^2 &= 1 - \frac{\sum_{i=1}^{N} r_i^2}{\sum_{i=1}^{N} \left(Y_i - \overline{Y}\right)^2} \approx 1 - \frac{\sum_{i=1}^{N_{\text{train}}} (r_i^{\text{train}})^2}{\sum_{i=1}^{N_{\text{train}}} \left(Y_i^{\text{train}} - \overline{Y}\right)^2} \\
&= 1 - \frac{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (r_i^{\text{train}})^2}{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left(Y_i^{\text{train}} - \overline{Y}\right)^2}
\end{aligned}
$$

Relationship between training error and R^2

Consider the residuals $r_i^{\text{train}} = Y_i^{\text{train}} - \hat{y}(X_i^{\text{train}}, D^{\text{train}})$ for the training points. Then, when N_{train} is very large

$$
\begin{aligned}
R^2 &= 1 - \frac{\sum_{i=1}^{N} r_i^2}{\sum_{i=1}^{N} \left(Y_i - \overline{Y}\right)^2} \approx 1 - \frac{\sum_{i=1}^{N_{\text{train}}} (r_i^{\text{train}})^2}{\sum_{i=1}^{N_{\text{train}}} \left(Y_i^{\text{train}} - \overline{Y}\right)^2} \\
&= 1 - \frac{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (r_i^{\text{train}})^2}{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left(Y_i^{\text{train}} - \overline{Y}\right)^2} \\
&\approx 1 - \frac{\hat{\sigma}^2_{\text{train}}}{\hat{\sigma}^2_Y}.
\end{aligned}
$$

Relationship between training error and R^2

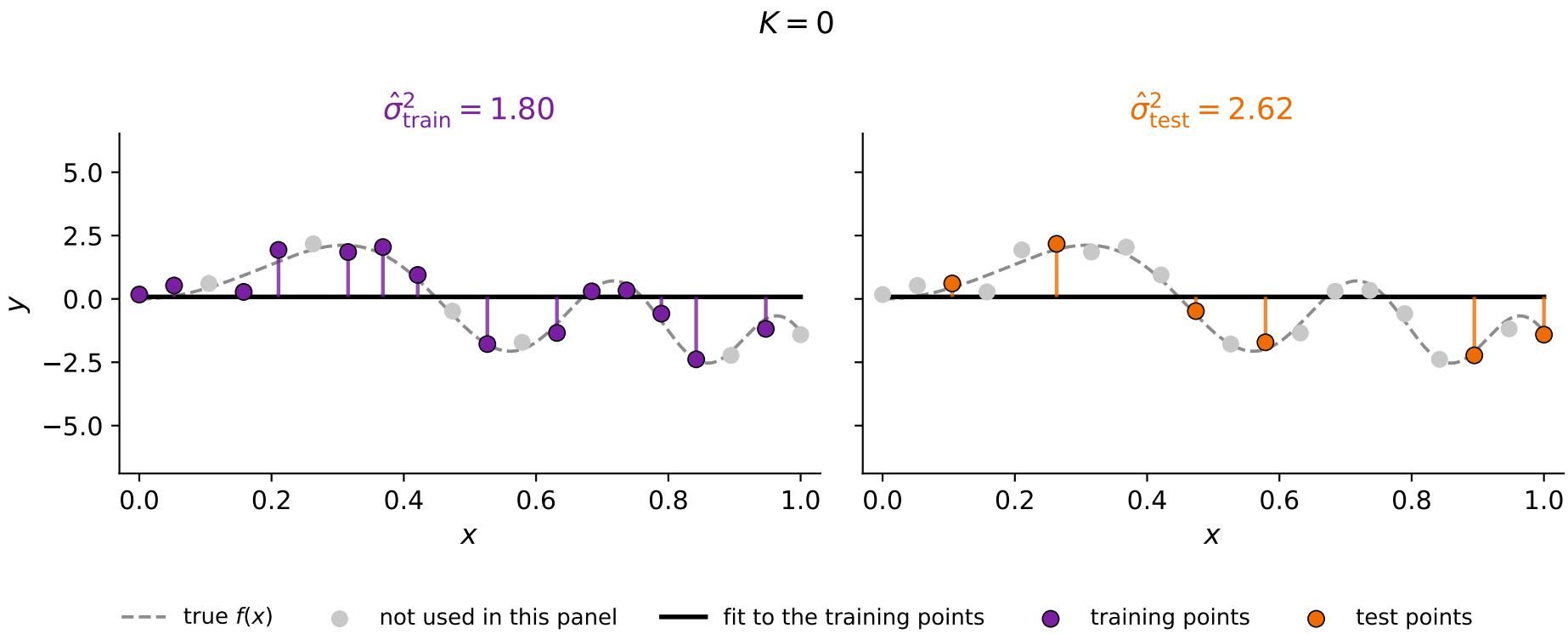
Consider the residuals $r_i^{\text{train}} = Y_i^{\text{train}} - \hat{y}(X_i^{\text{train}}, D^{\text{train}})$ for the training points. Then, when N_{train} is very large

$$\begin{aligned} R^2 &= 1 - \frac{\sum_{i=1}^{N} r_i^2}{\sum_{i=1}^{N} \left(Y_i - \overline{Y}\right)^2} \approx 1 - \frac{\sum_{i=1}^{N_{\text{train}}} (r_i^{\text{train}})^2}{\sum_{i=1}^{N_{\text{train}}} \left(Y_i^{\text{train}} - \overline{Y}\right)^2} \\ &= 1 - \frac{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (r_i^{\text{train}})^2}{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left(Y_i^{\text{train}} - \overline{Y}\right)^2} \\ &\approx 1 - \frac{\hat{\sigma}^2_{\text{train}}}{\hat{\sigma}^2_Y}. \end{aligned}$$

Since $\hat{\sigma}^2_Y$ is the same regardless of K (remember this does not depend on the model, only the data!), R^2 and $\hat{\sigma}^2_{\text{train}}$ give us approximately the same information, provided we have enough training points.

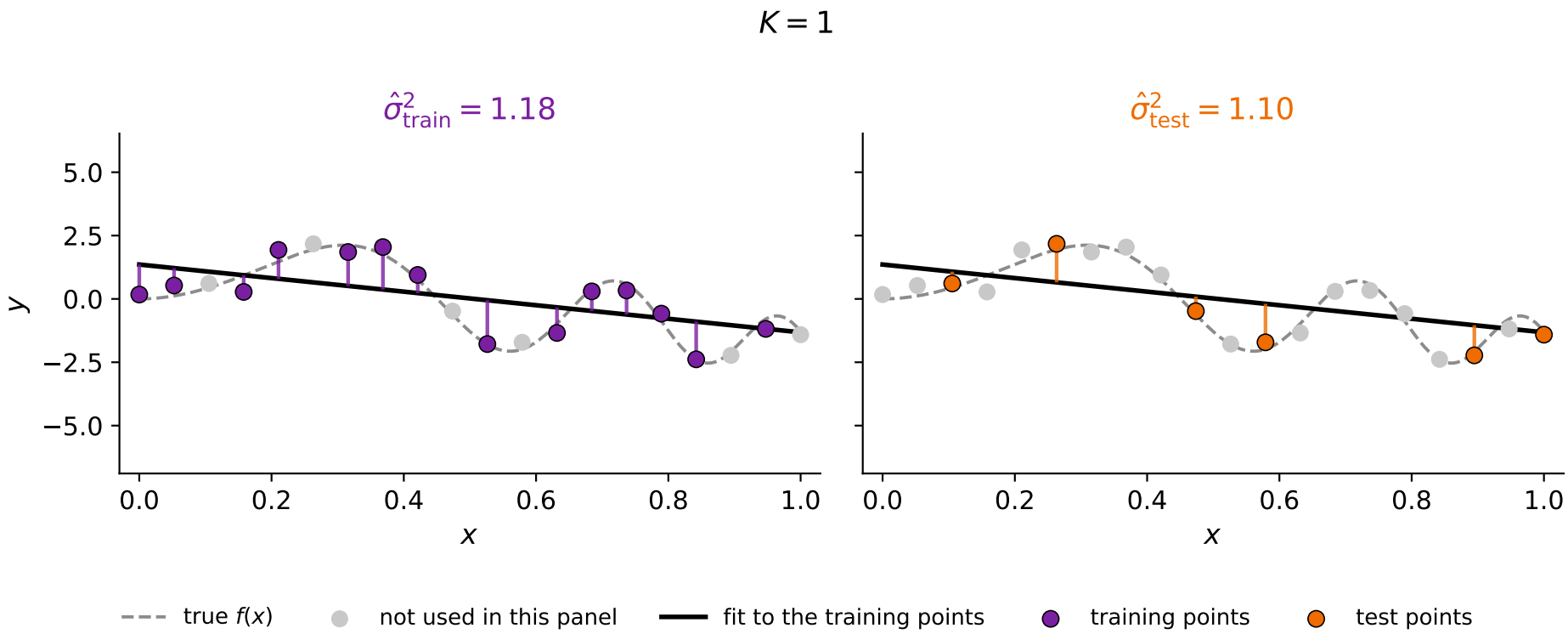
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



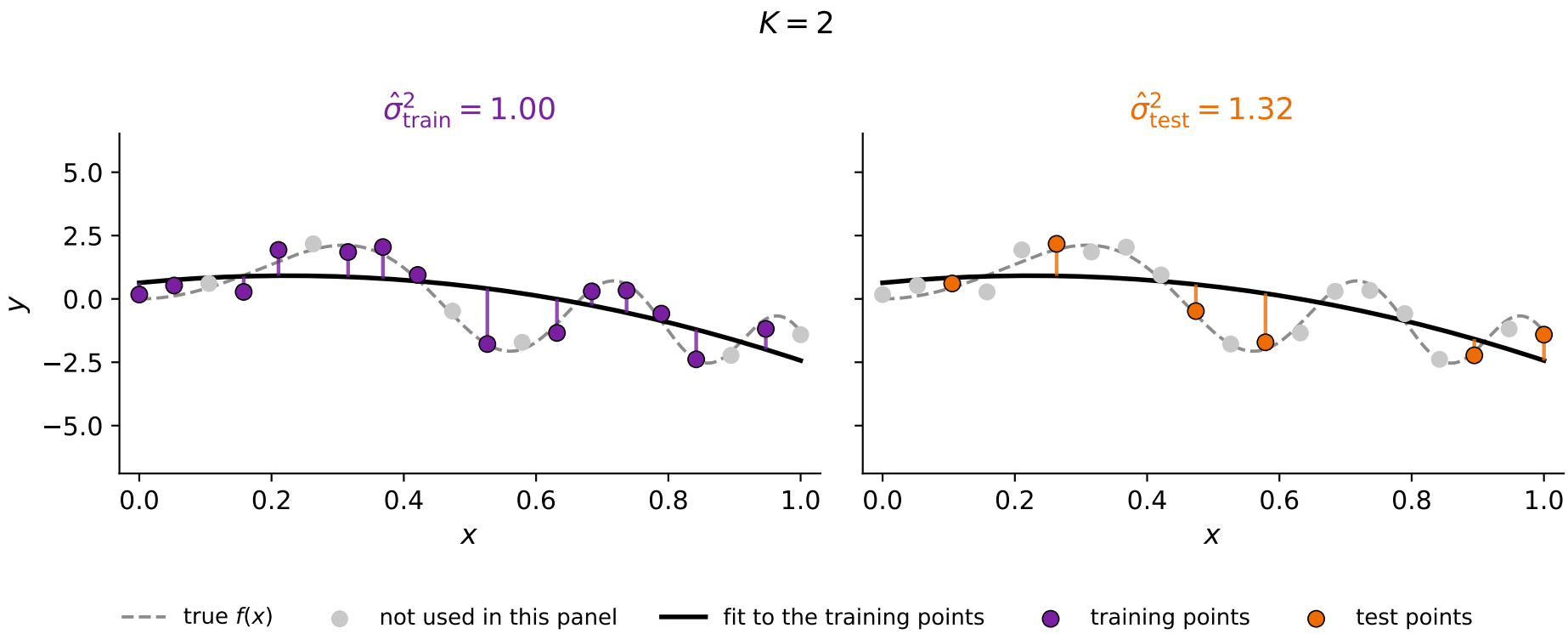
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



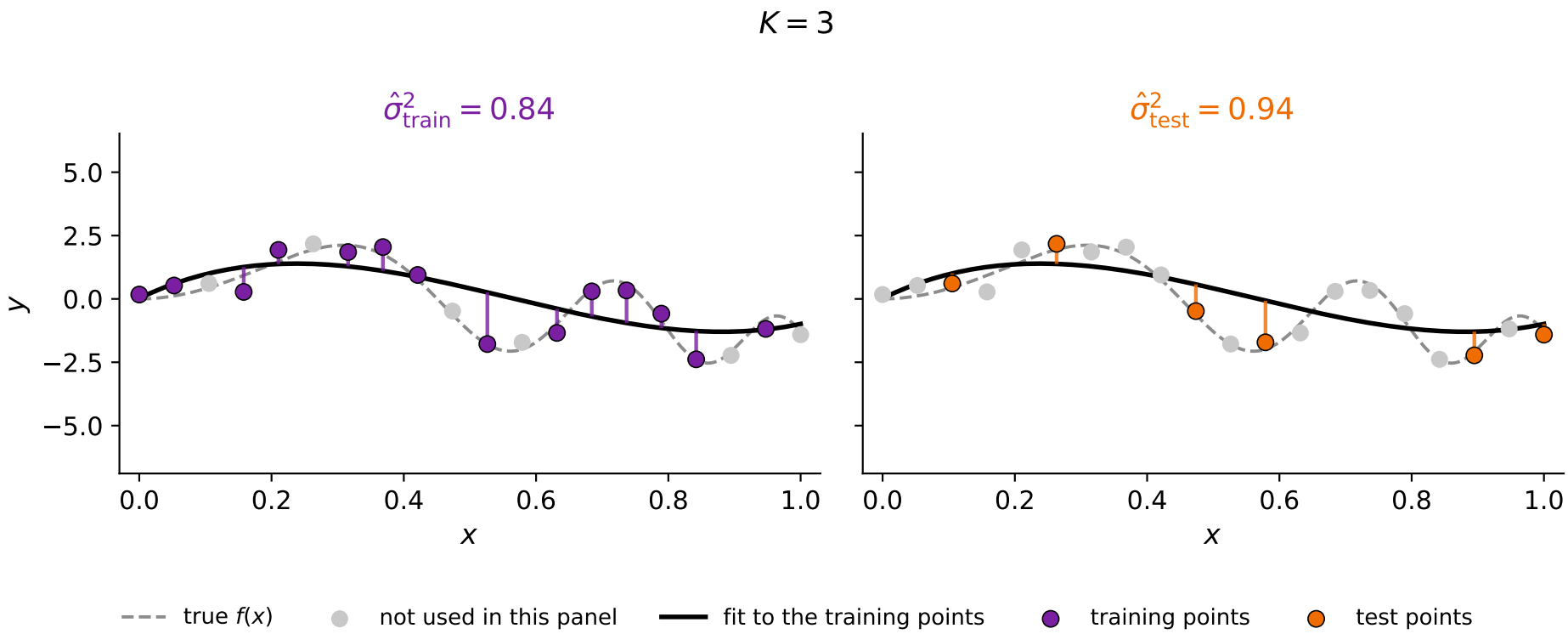
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



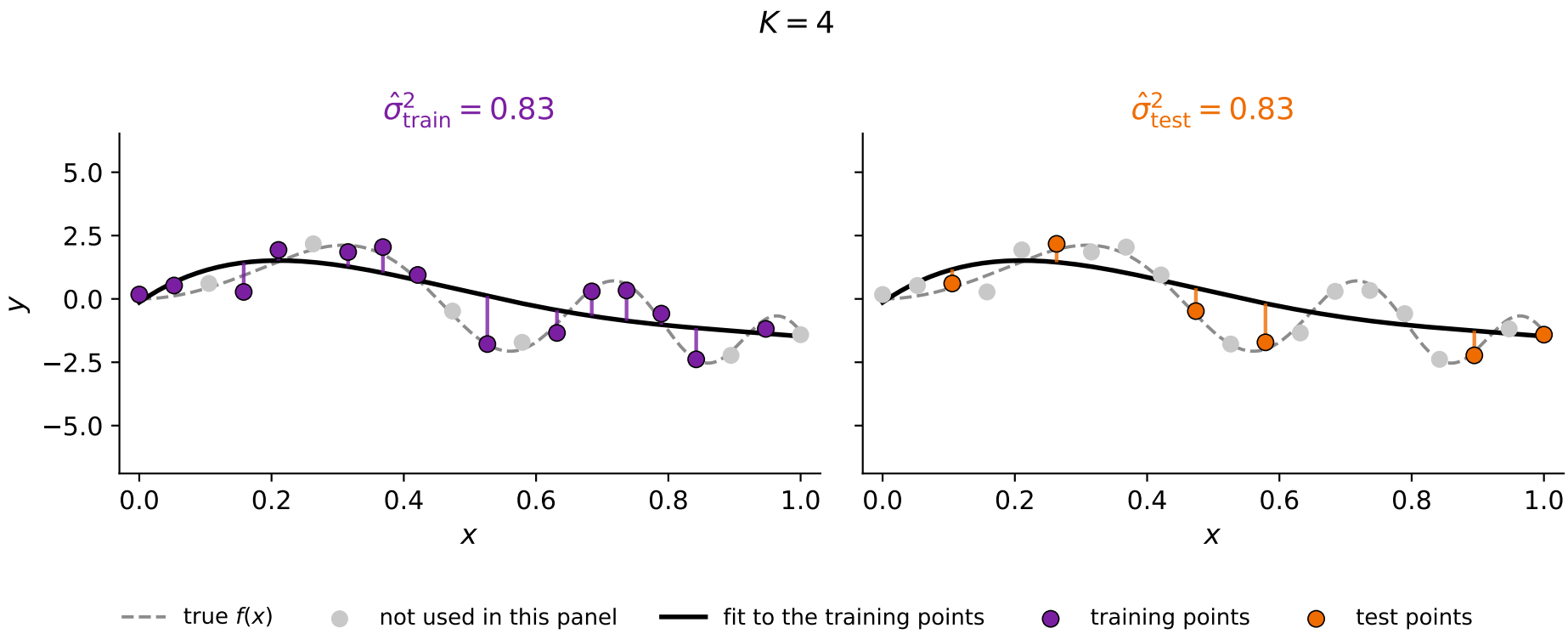
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



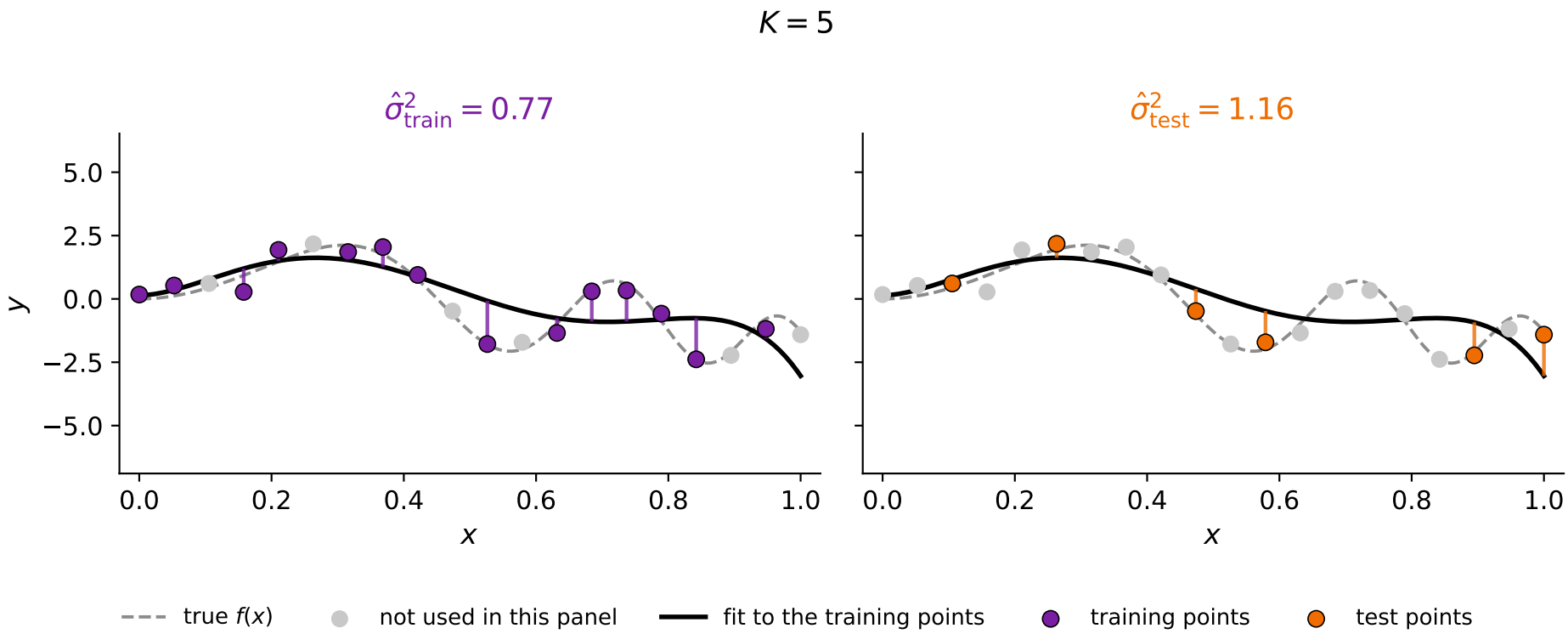
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



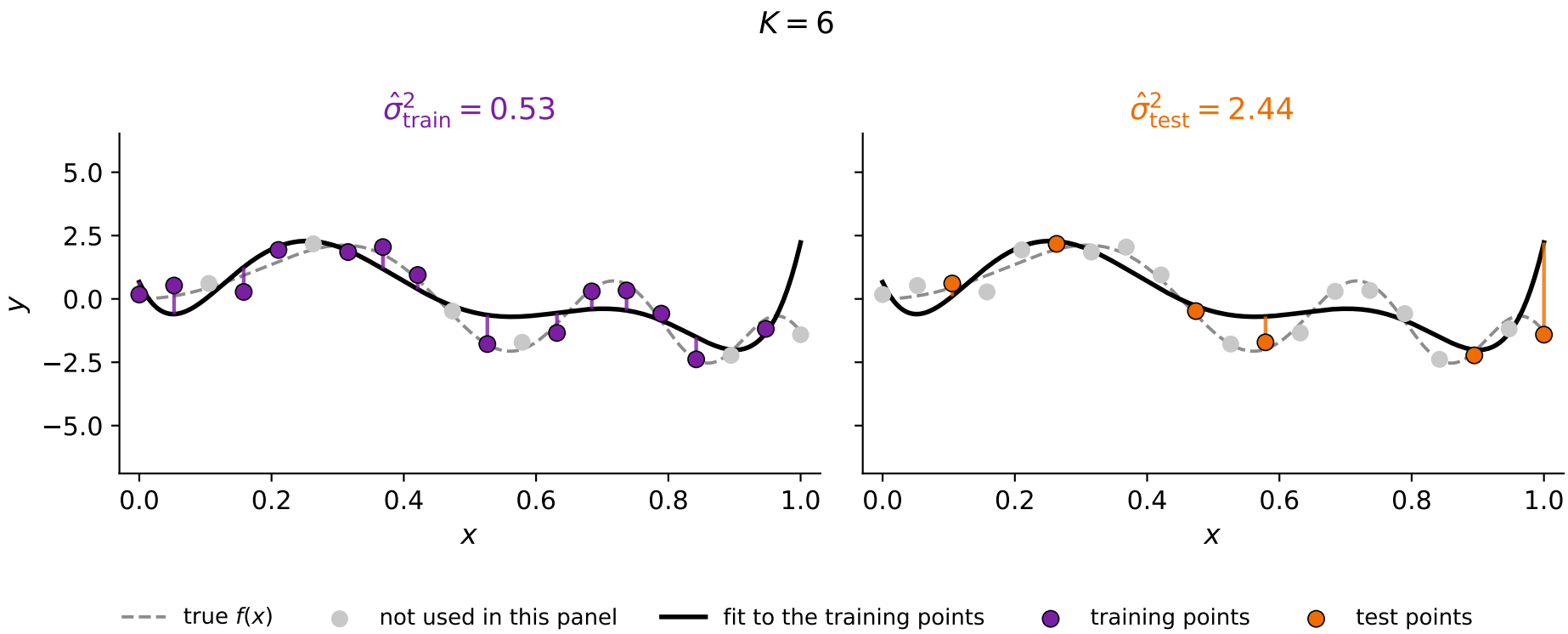
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



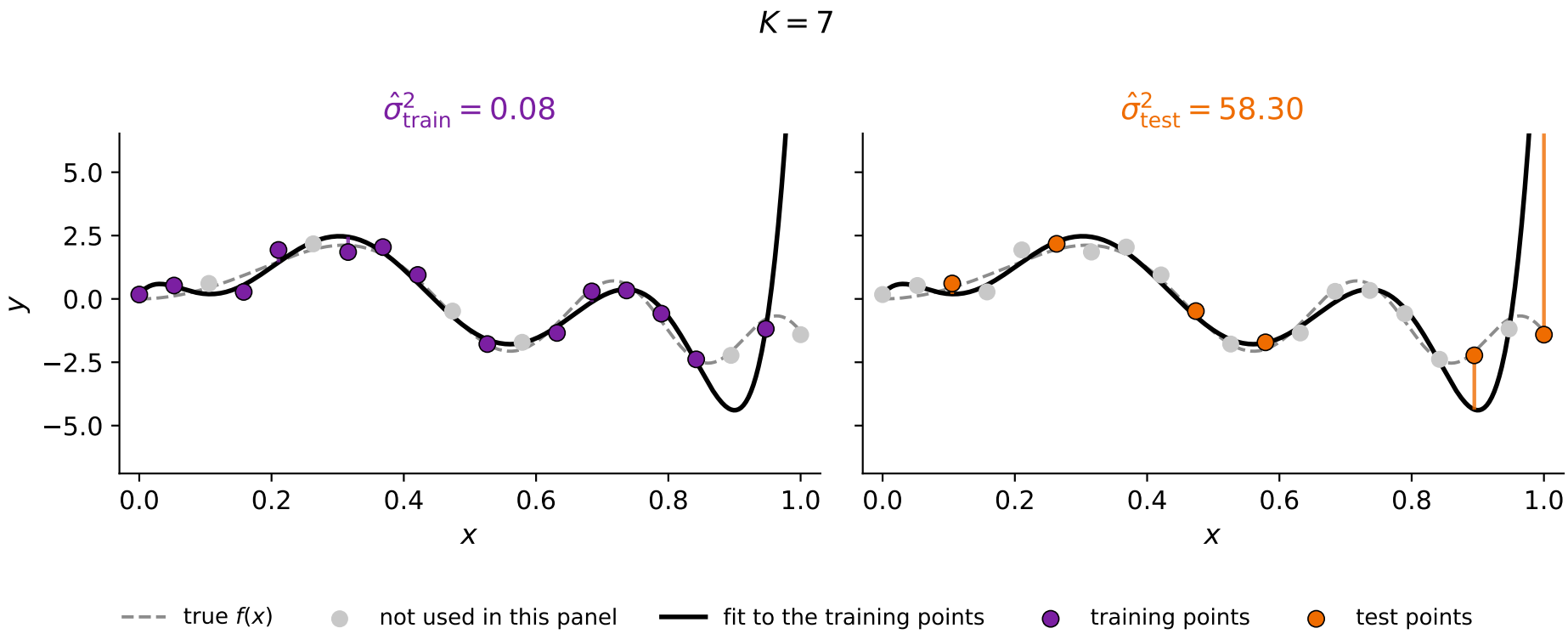
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



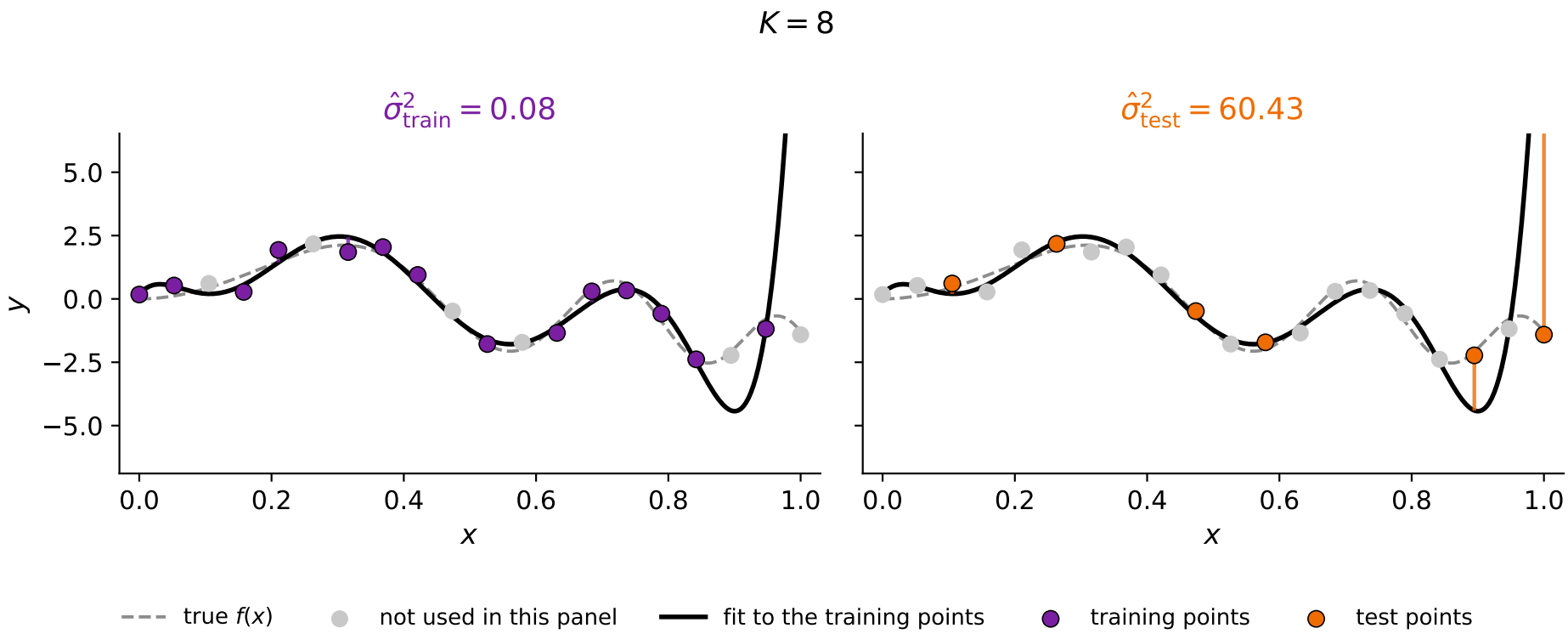
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



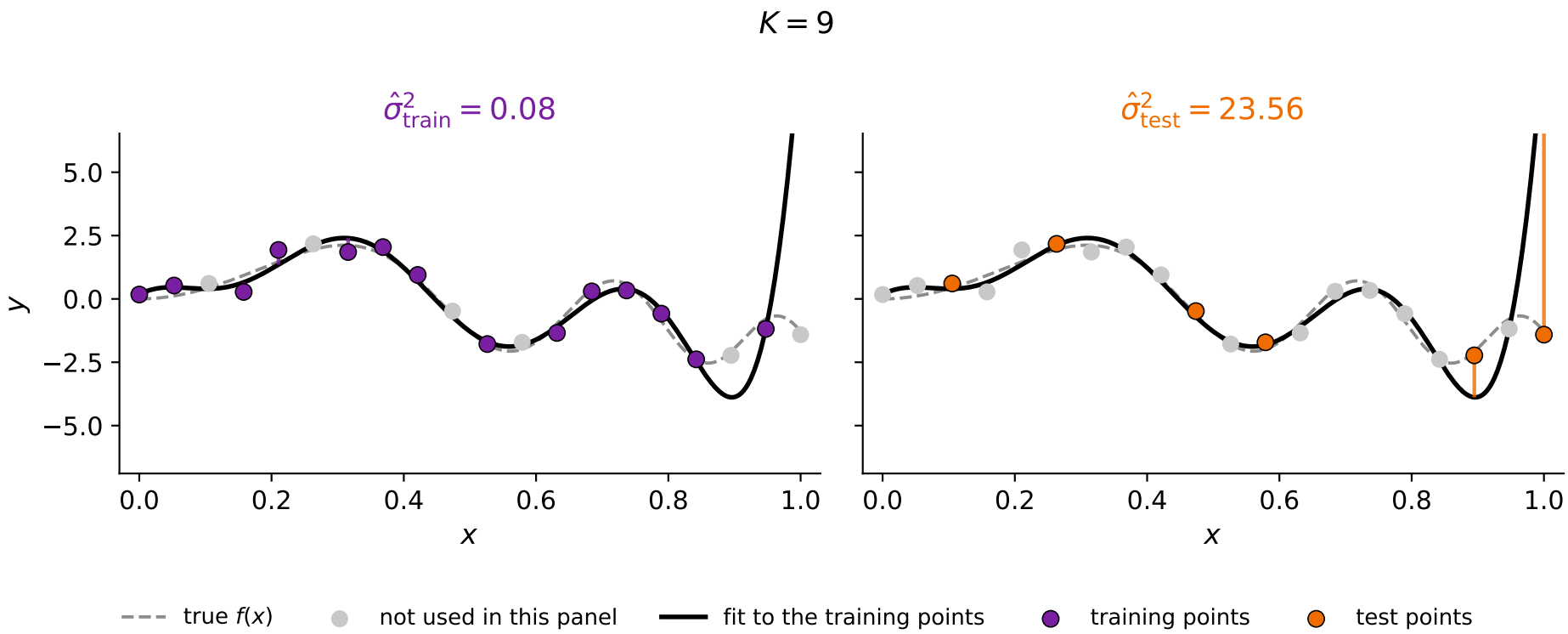
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



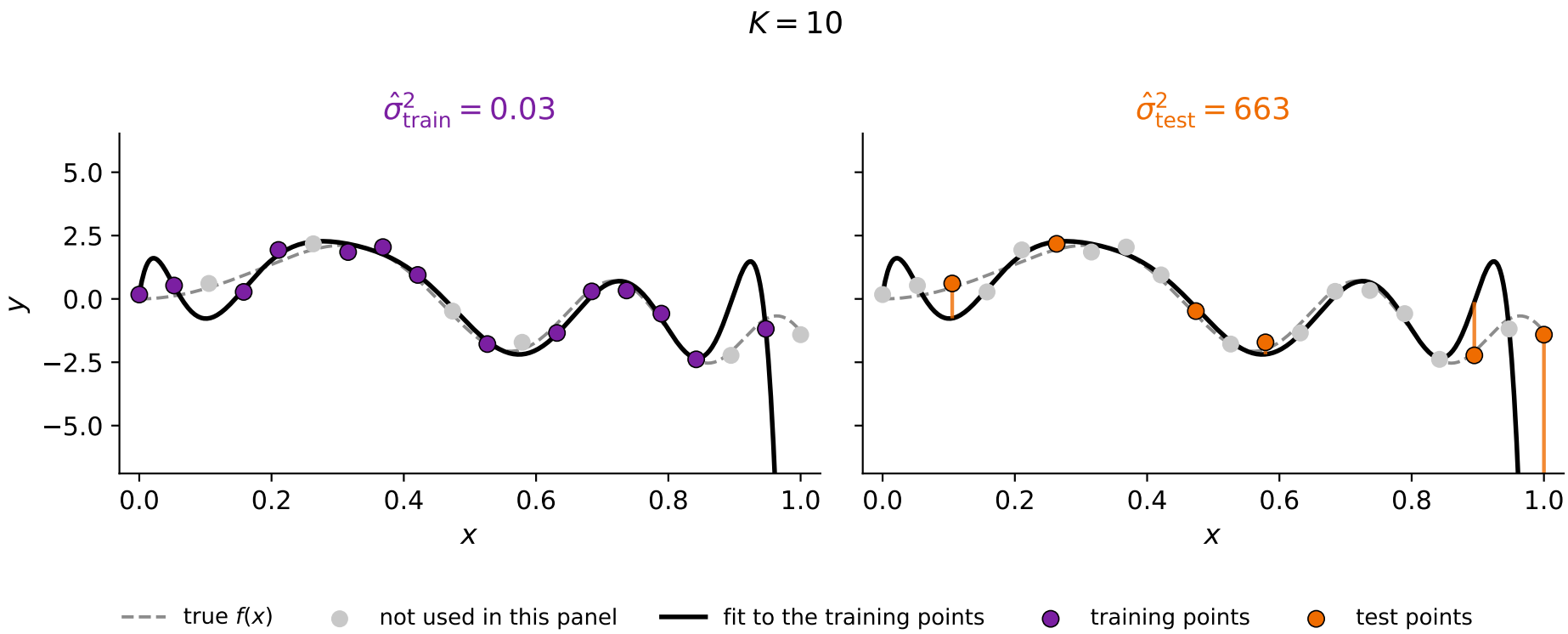
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



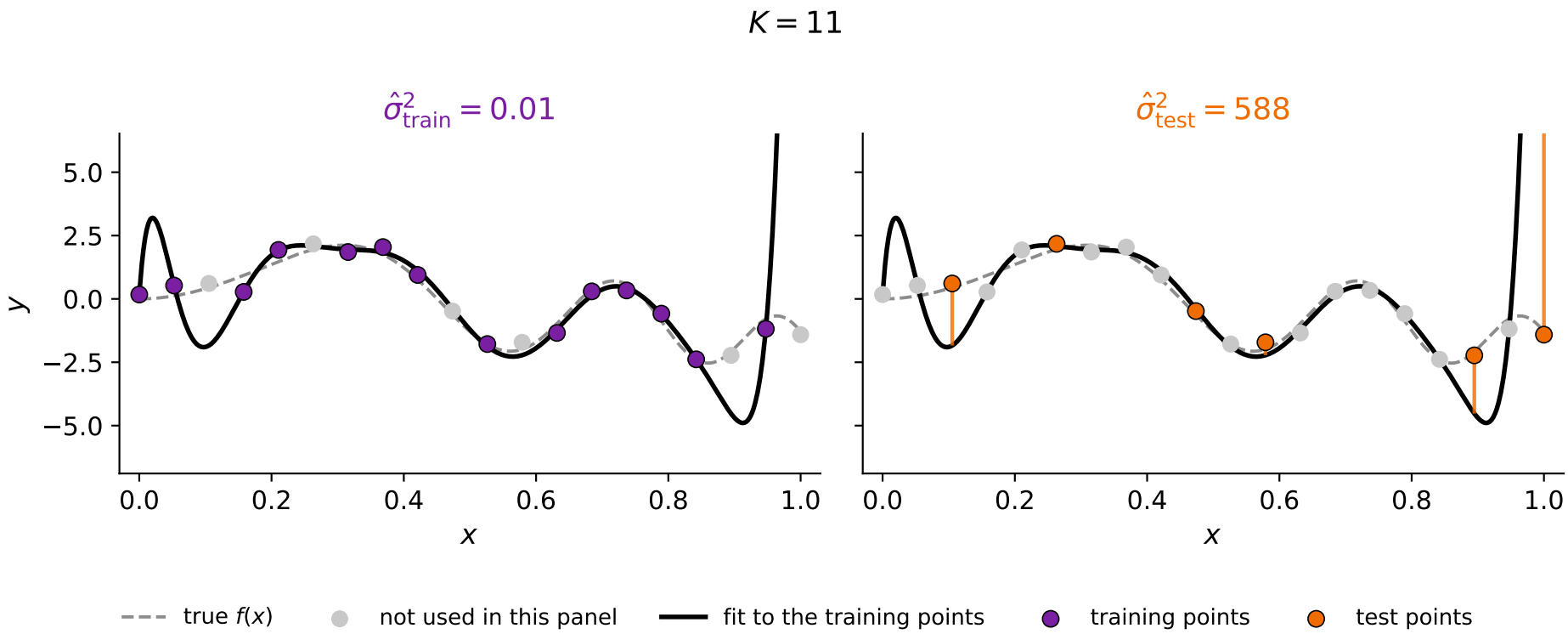
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



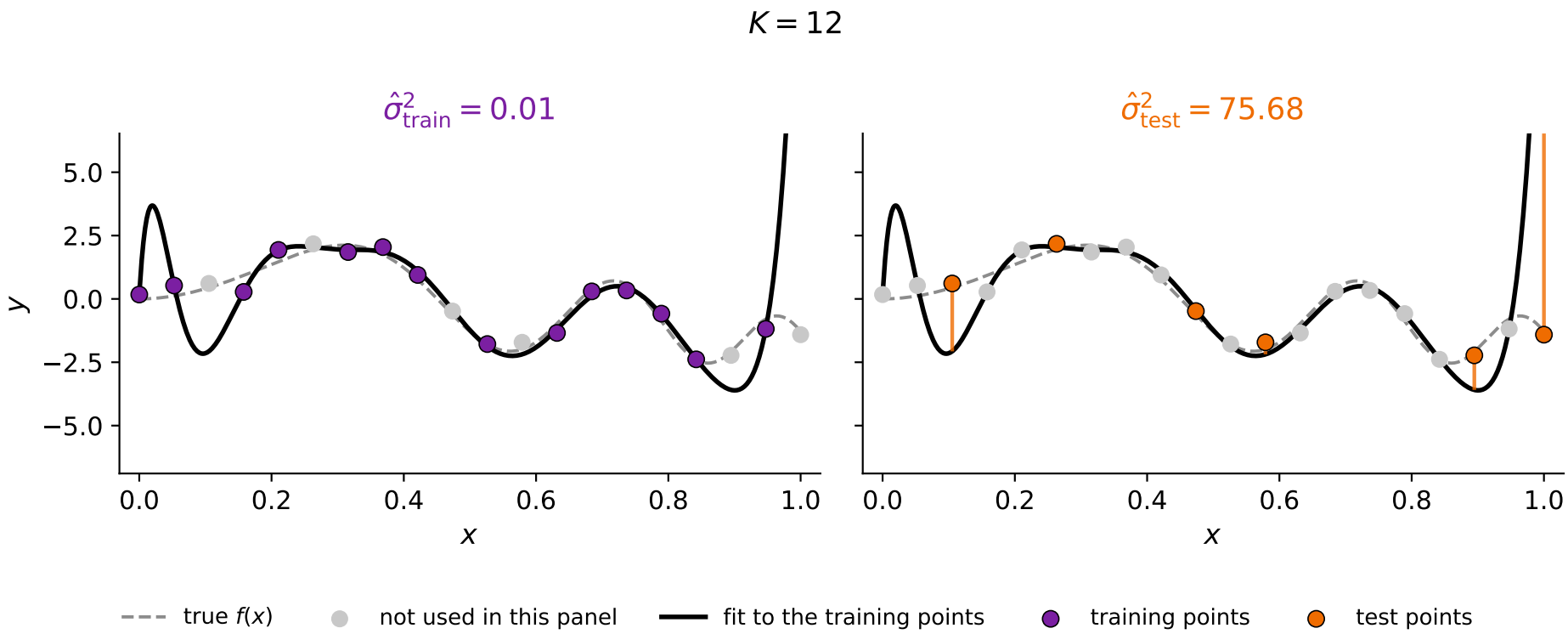
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



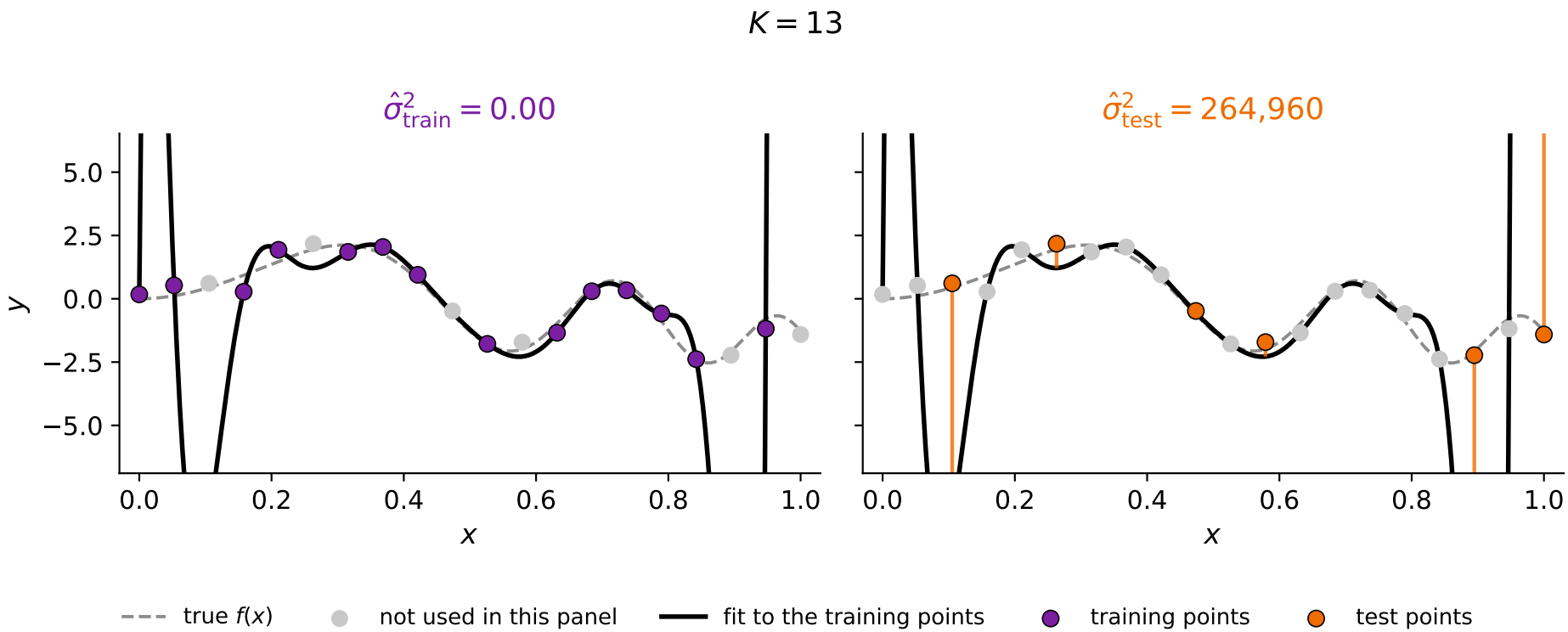
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



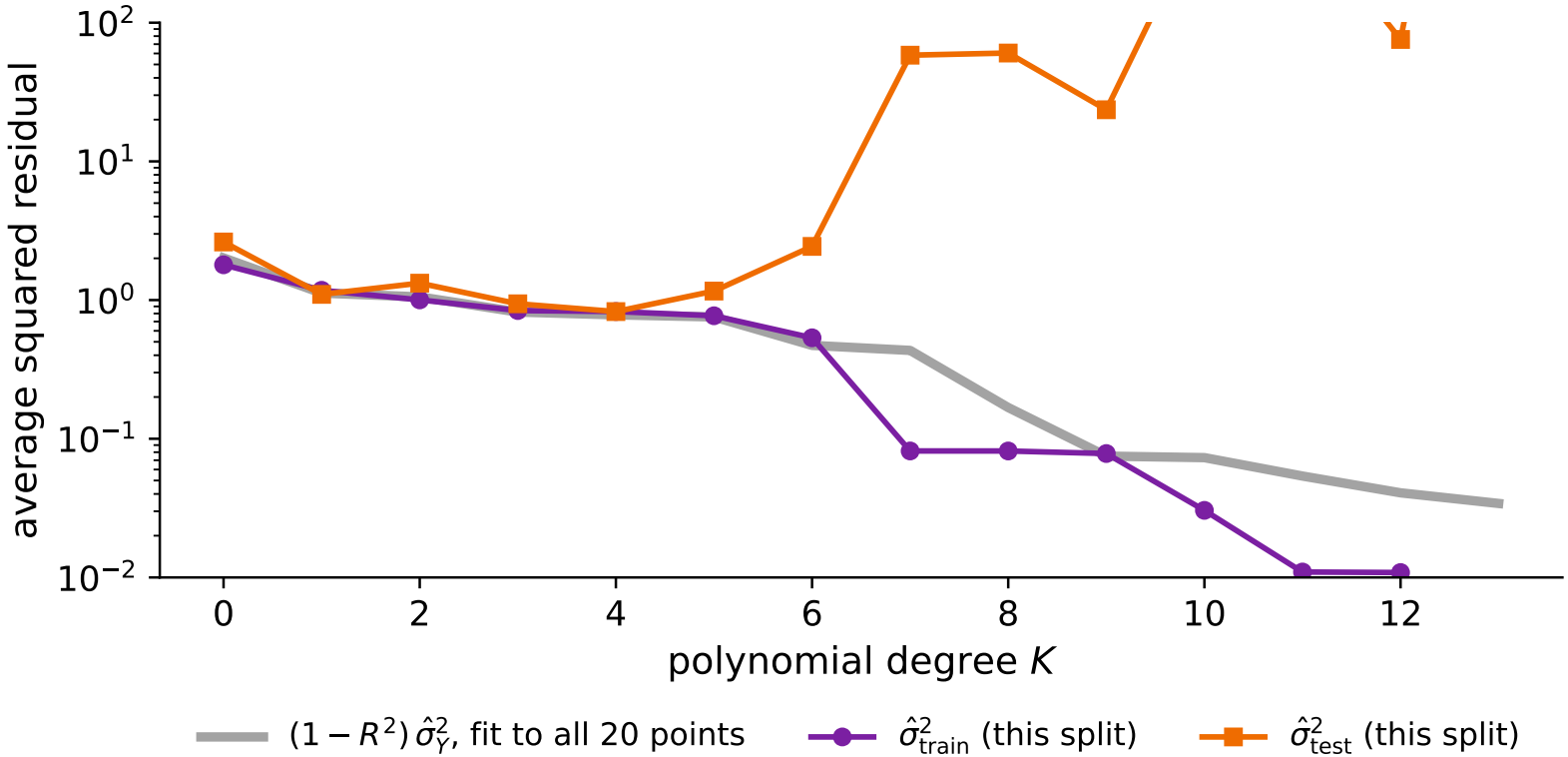
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



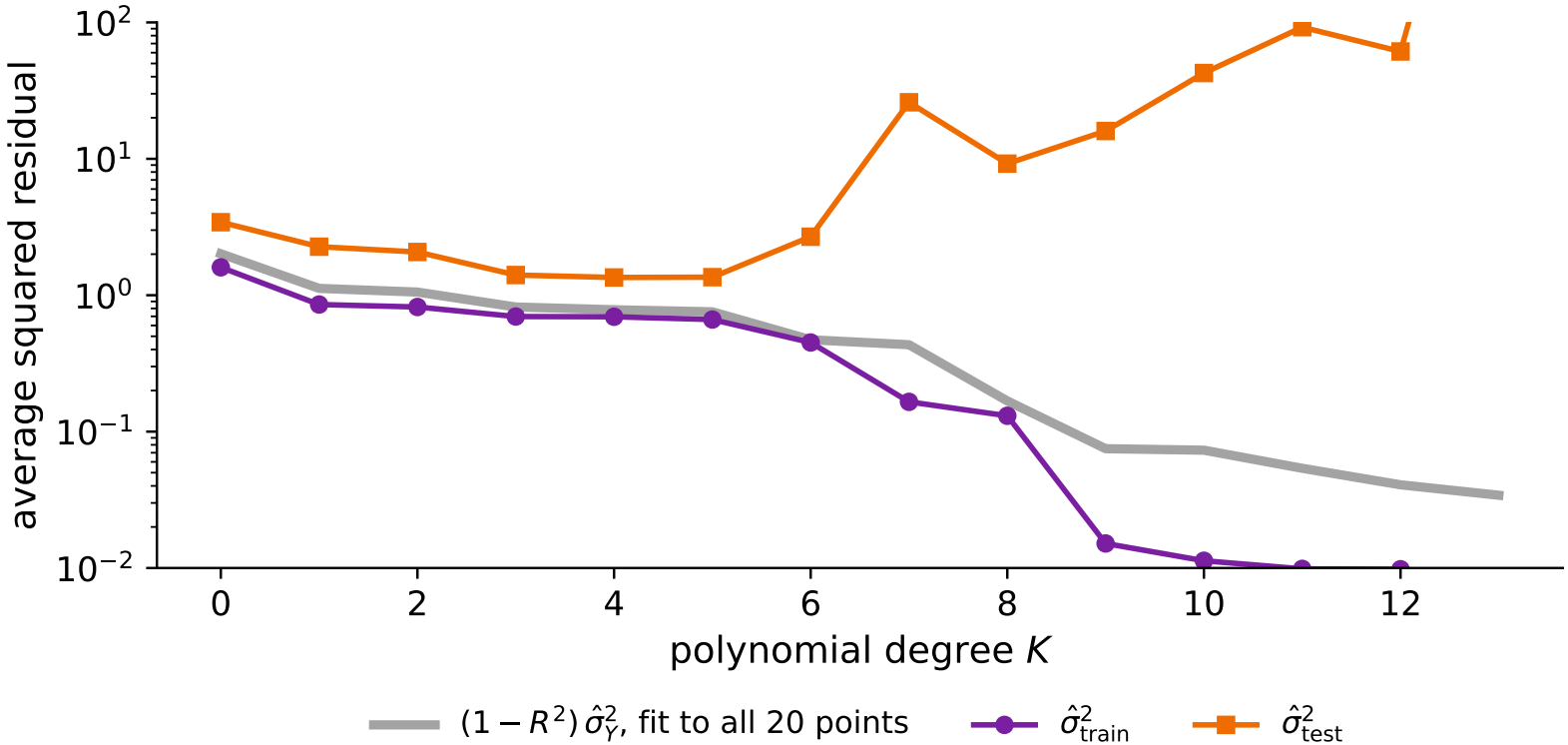
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



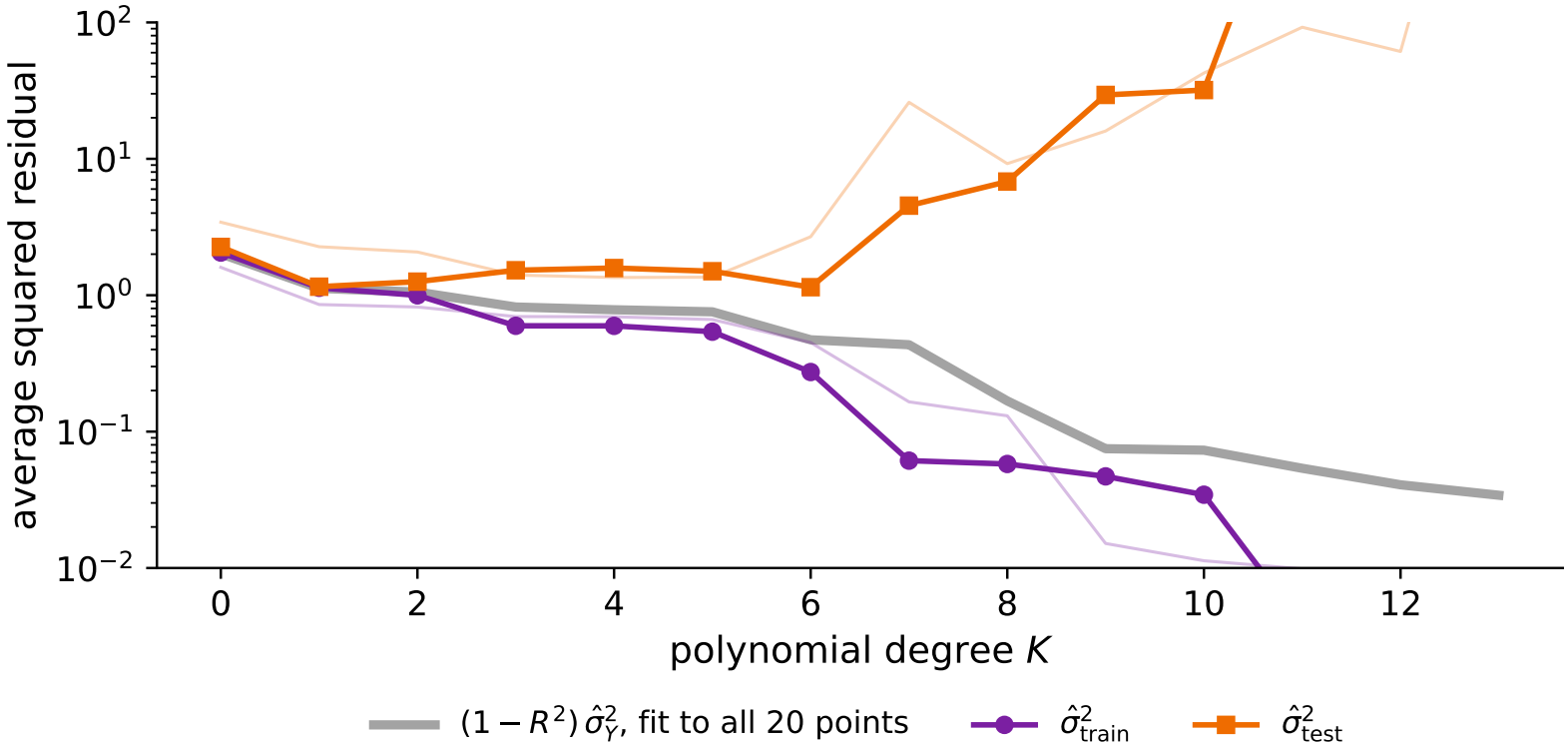
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



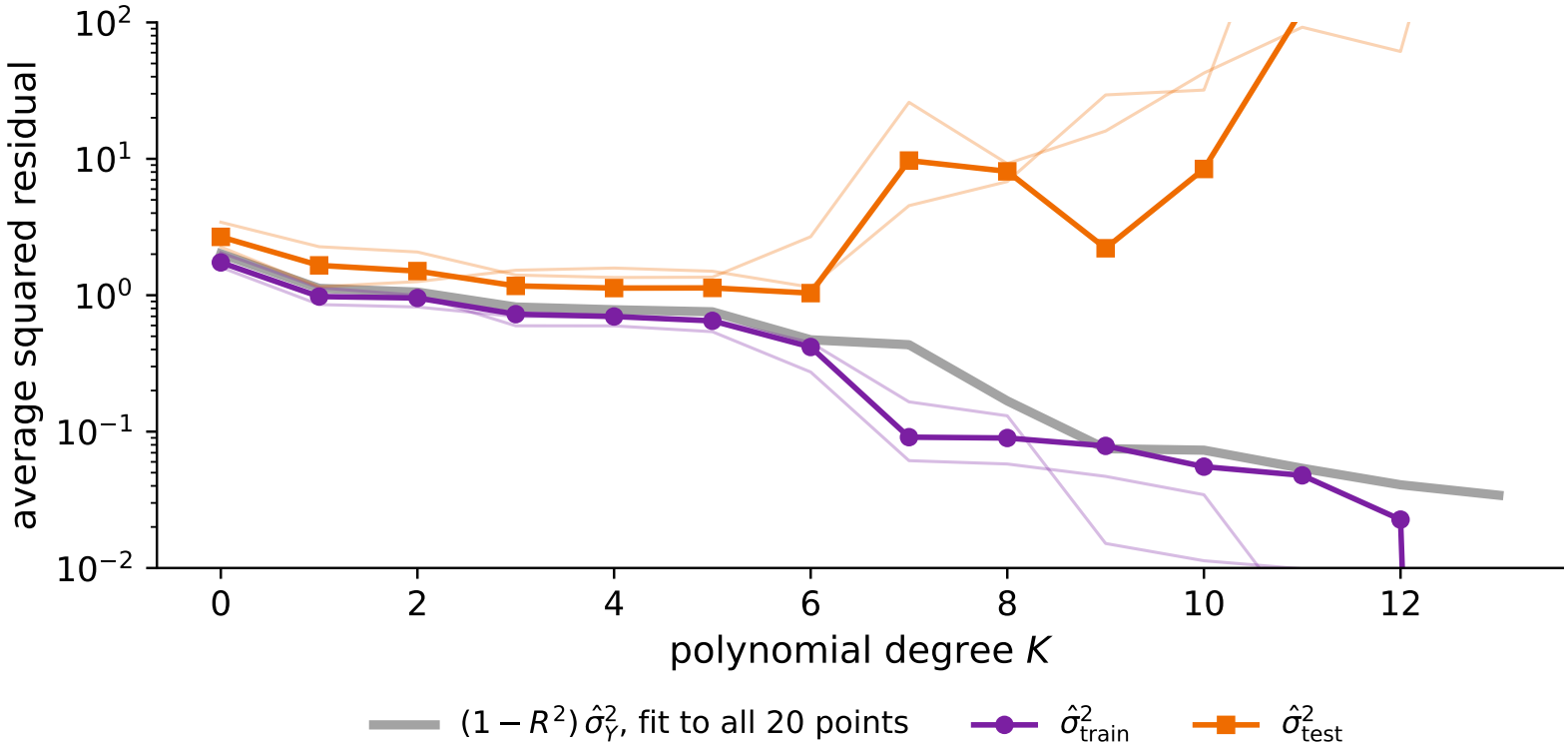
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



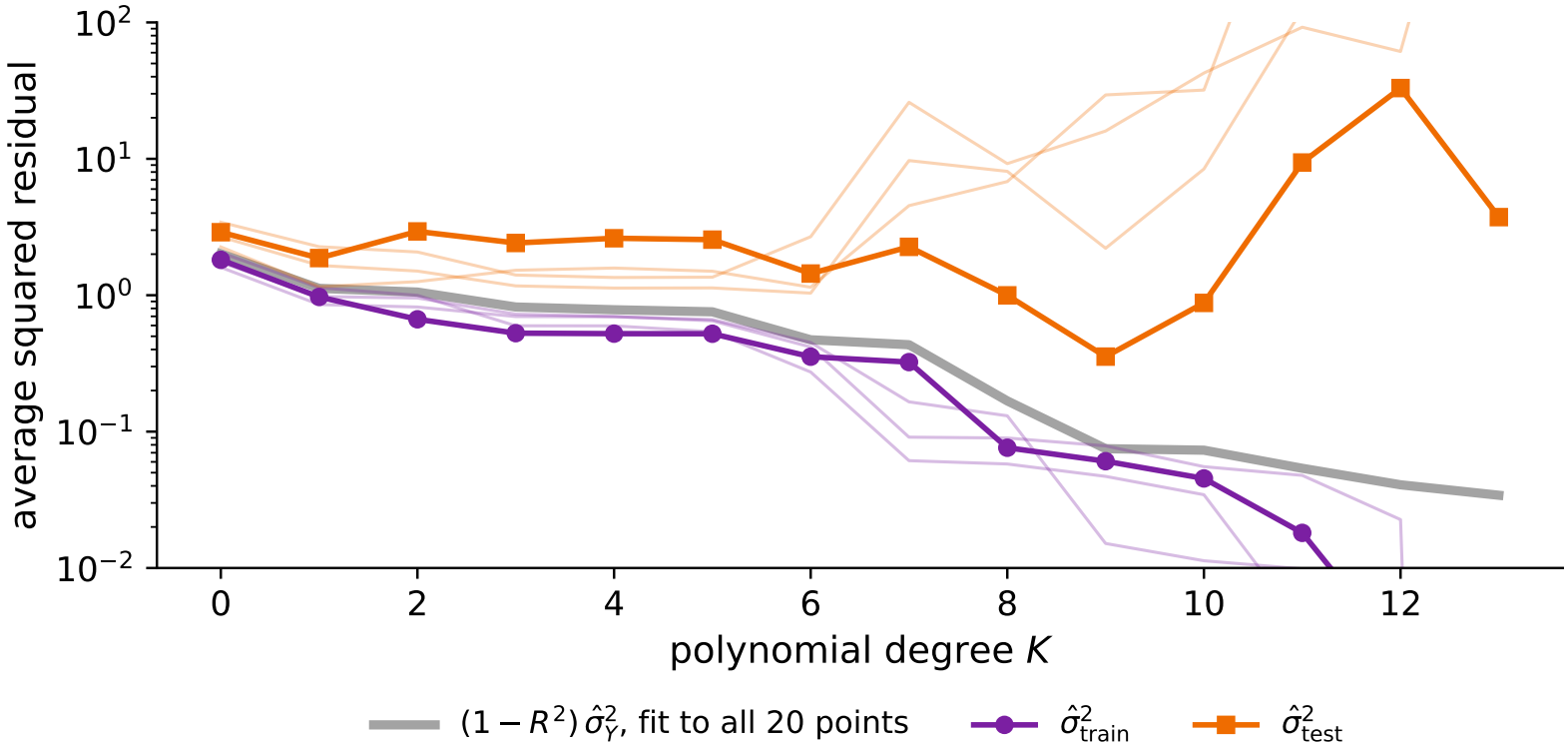
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



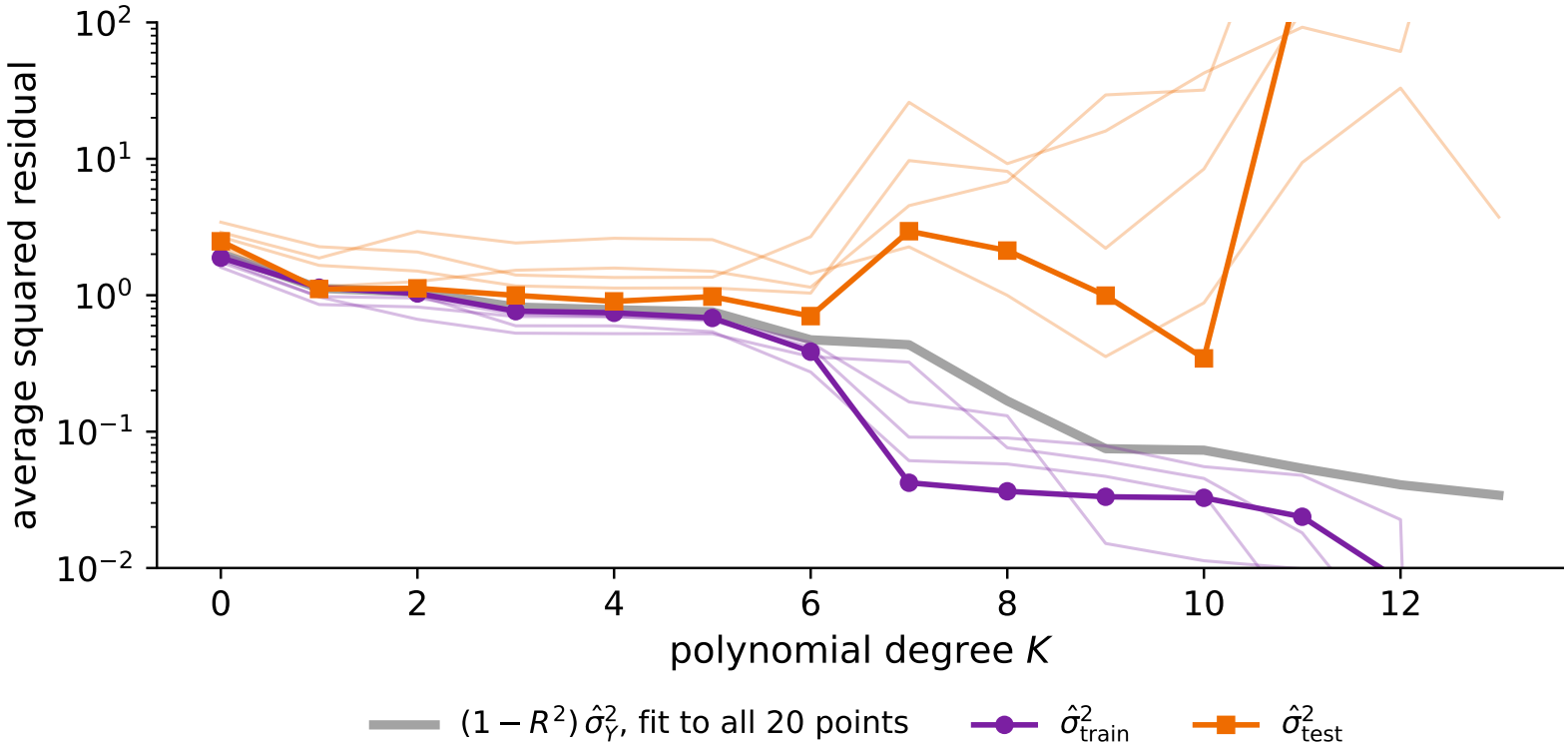
Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



Test/training split

The 20 points split at random into 14 training points and 6 test points. Each fit uses only the training points.



Test error and MSE

test error, large N_{test}

$$\hat{\sigma}^2_{\text{test}} \approx E_{X,Y}\left[(\hat{y}(X, D^{\text{train}}) - Y)^2\right]$$

$E_{X,Y}$: average over X (drawn at random from the evenly spaced points)
and $Y = f(X) + \epsilon$, with D^{train} fixed

Test error and MSE

test error, large N_{test}

$$\hat{\sigma}^2_{\text{test}} \approx E_{X,Y}\left[(\hat{y}(X, D^{\text{train}}) - Y)^2\right]$$

$E_{X,Y}$: average over X (drawn at random from the evenly spaced points) and $Y = f(X) + \epsilon$, with D^{train} fixed

MSE at x

$$\text{MSE}_{\hat{y}(x,D)} = E_D\left[(\hat{y}(X, D) - f(X))^2 \mid X = x\right]$$

E_D: average over the training set D, with $X = x$ fixed

Test error and MSE

With $Y = f(X) + \epsilon$, where ϵ has mean 0 and is independent of X and D^{train}, and writing $\hat{y} = \hat{y}(X, D^{\text{train}})$:

$$E_{X,Y}\left[(\hat{y} - Y)^2\right] = E_{X,Y}\left[(\hat{y} - f(X) - \epsilon)^2\right]$$

Test error and MSE

With $Y = f(X) + \epsilon$, where ϵ has mean 0 and is independent of X and D^{train}, and writing $\hat{y} = \hat{y}(X, D^{\text{train}})$:

$$\begin{aligned} E_{X,Y}\left[(\hat{y} - Y)^2\right] &= E_{X,Y}\left[(\hat{y} - f(X) - \epsilon)^2\right] \\ &= E_X\left[(\hat{y} - f(X))^2\right] - 2\,E_{X,Y}\left[(\hat{y} - f(X))\,\epsilon\right] + E[\epsilon^2] \end{aligned}$$

Test error and MSE

With $Y = f(X) + \epsilon$, where ϵ has mean 0 and is independent of X and D^{train}, and writing $\hat{y} = \hat{y}(X, D^{\text{train}})$:

$$\begin{aligned} E_{X,Y}\left[(\hat{y} - Y)^2\right] &= E_{X,Y}\left[(\hat{y} - f(X) - \epsilon)^2\right] \\ &= E_X\left[(\hat{y} - f(X))^2\right] - 2\,E_{X,Y}\left[(\hat{y} - f(X))\,\epsilon\right] + E[\epsilon^2] \\ &= E_X\left[(\hat{y} - f(X))^2\right] + \sigma_\epsilon^2 \qquad \text{(cross term 0)} \end{aligned}$$

Test error and MSE

With $Y = f(X) + \epsilon$, where ϵ has mean 0 and is independent of X and D^{train}, and writing $\hat{y} = \hat{y}(X, D^{\text{train}})$:

$$\begin{aligned} E_{X,Y}\left[(\hat{y} - Y)^2\right] &= E_{X,Y}\left[(\hat{y} - f(X) - \epsilon)^2\right] \\ &= E_X\left[(\hat{y} - f(X))^2\right] - 2\,E_{X,Y}\left[(\hat{y} - f(X))\,\epsilon\right] + E[\epsilon^2] \\ &= E_X\left[(\hat{y} - f(X))^2\right] + \sigma_\epsilon^2 \qquad \text{(cross term 0)} \end{aligned}$$

Now average over the training set, writing D for D^{train}:

$$E_D[\hat{\sigma}^2_{\text{test}}] \approx E_X\left[E_D\left[(\hat{y}(X, D) - f(X))^2 \mid X\right]\right] + \sigma_\epsilon^2$$

Test error and MSE

With $Y = f(X) + \epsilon$, where ϵ has mean 0 and is independent of X and D^{train}, and writing $\hat{y} = \hat{y}(X, D^{\text{train}})$:

$$\begin{aligned} E_{X,Y}\left[(\hat{y} - Y)^2\right] &= E_{X,Y}\left[(\hat{y} - f(X) - \epsilon)^2\right] \\ &= E_X\left[(\hat{y} - f(X))^2\right] - 2\,E_{X,Y}\left[(\hat{y} - f(X))\,\epsilon\right] + E[\epsilon^2] \\ &= E_X\left[(\hat{y} - f(X))^2\right] + \sigma_\epsilon^2 \qquad \text{(cross term 0)} \end{aligned}$$

Now average over the training set, writing D for D^{train}:

$$E_D[\hat{\sigma}^2_{\text{test}}] \approx E_X\left[E_D\left[(\hat{y}(X, D) - f(X))^2 \mid X\right]\right] + \sigma_\epsilon^2$$

test error and MSE

$$E_D[\hat{\sigma}^2_{\text{test}}] = E_X\left[\text{MSE}_{\hat{y}(X,D)}\right] + \sigma_\epsilon^2$$

Bias and variance of the MSE

By the bias-variance decomposition of the MSE,

$$\mathrm{MSE}_{\hat{y}(x,D)} = \underbrace{\mathrm{var}_D\left(\hat{y}(x,D)\right)}_{\text{variance}} + \underbrace{\left(E_D[\hat{y}(x,D)] - f(x)\right)^2}_{\text{bias}^2}.$$

Bias and variance of the MSE

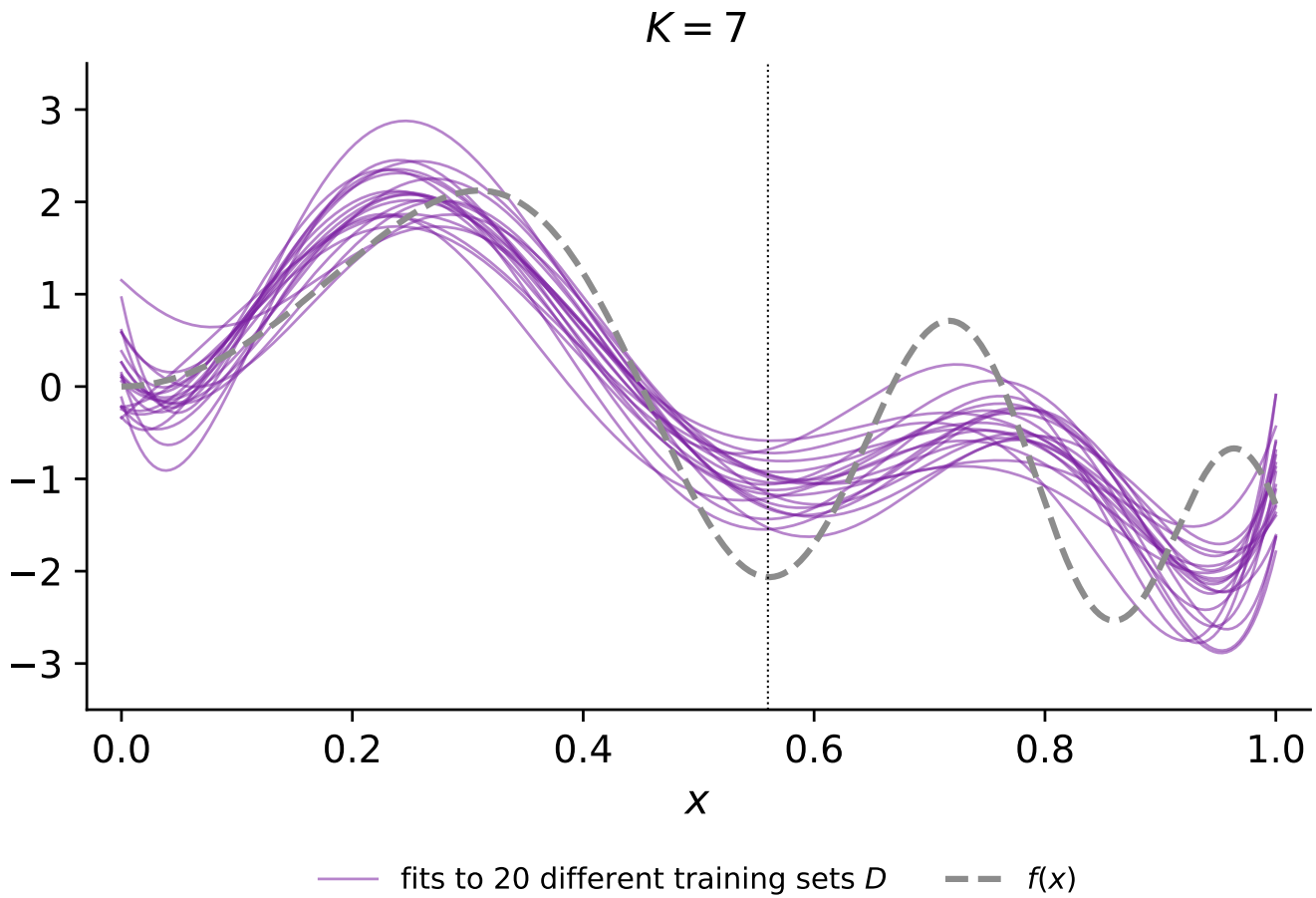
By the bias-variance decomposition of the MSE,

$$\mathrm{MSE}_{\hat{y}(x,D)} = \underbrace{\mathrm{var}_D\left(\hat{y}(x,D)\right)}_{\text{variance}} + \underbrace{\left(E_D[\hat{y}(x,D)] - f(x)\right)^2}_{\text{bias}^2}.$$

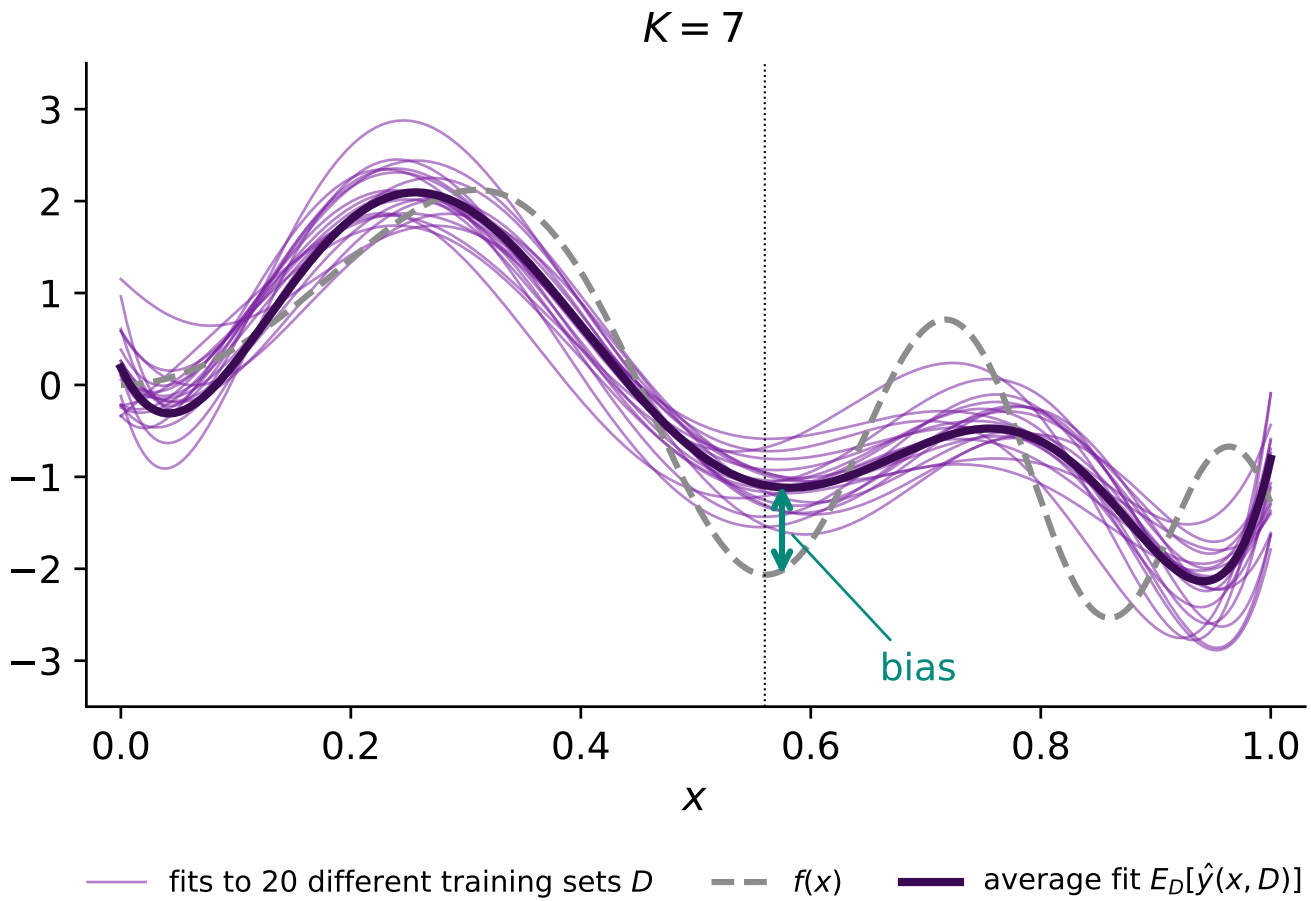
So the average test error is

$$\begin{aligned} E_D[\hat{\sigma}^2_{\text{test}}] &= E_X\left[\mathrm{MSE}_{\hat{y}(X,D)}\right] + \sigma^2_\epsilon \\ &= \underbrace{E_X\left[\mathrm{var}_D(\hat{y}(X,D))\right]}_{\text{variance, from } D} + \underbrace{E_X\left[(E_D[\hat{y}(X,D)] - f(X))^2\right]}_{\text{bias}^2} + \underbrace{\sigma^2_\epsilon}_{\text{noise, from } Y} \end{aligned}$$

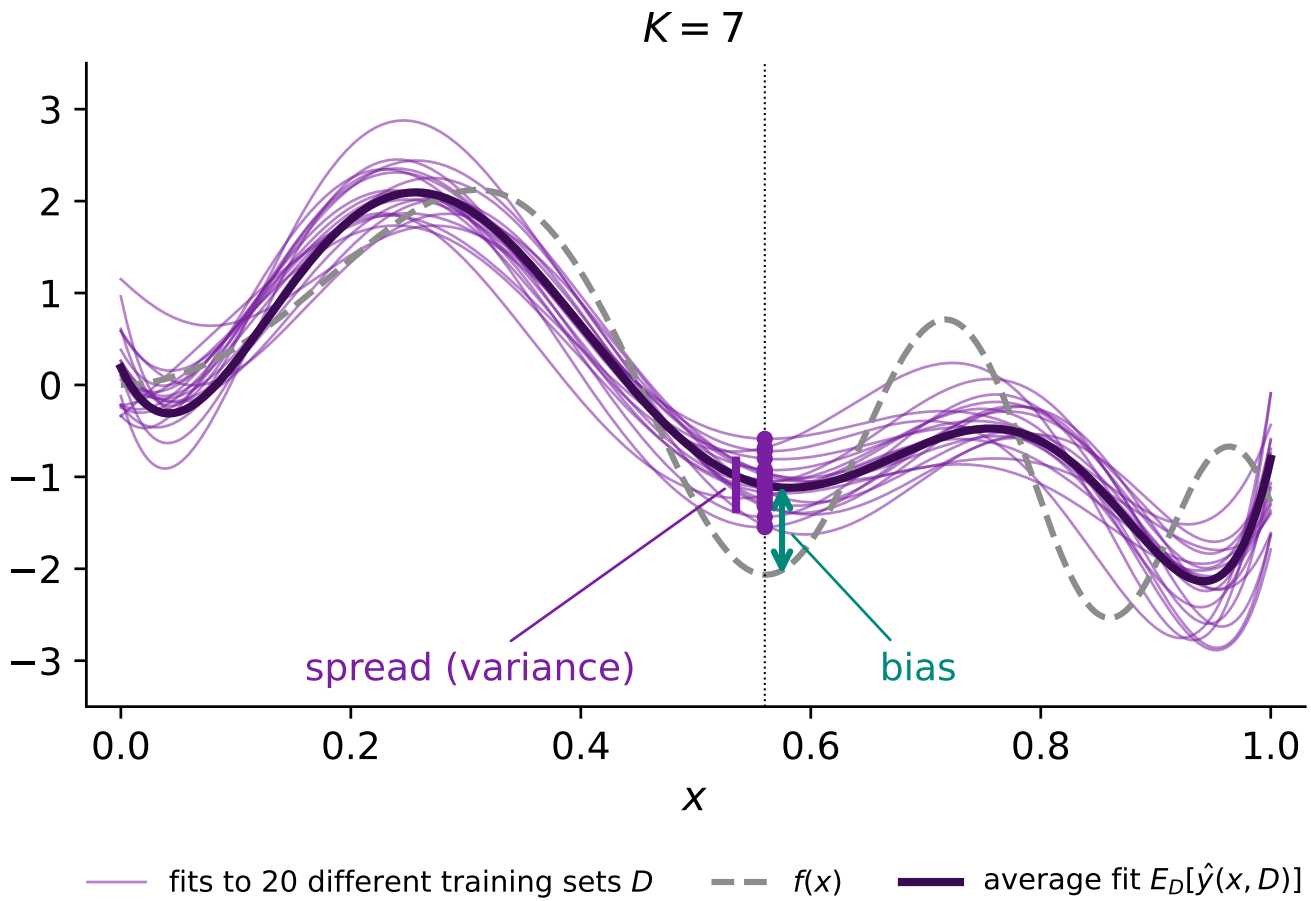
Bias and variance at one point



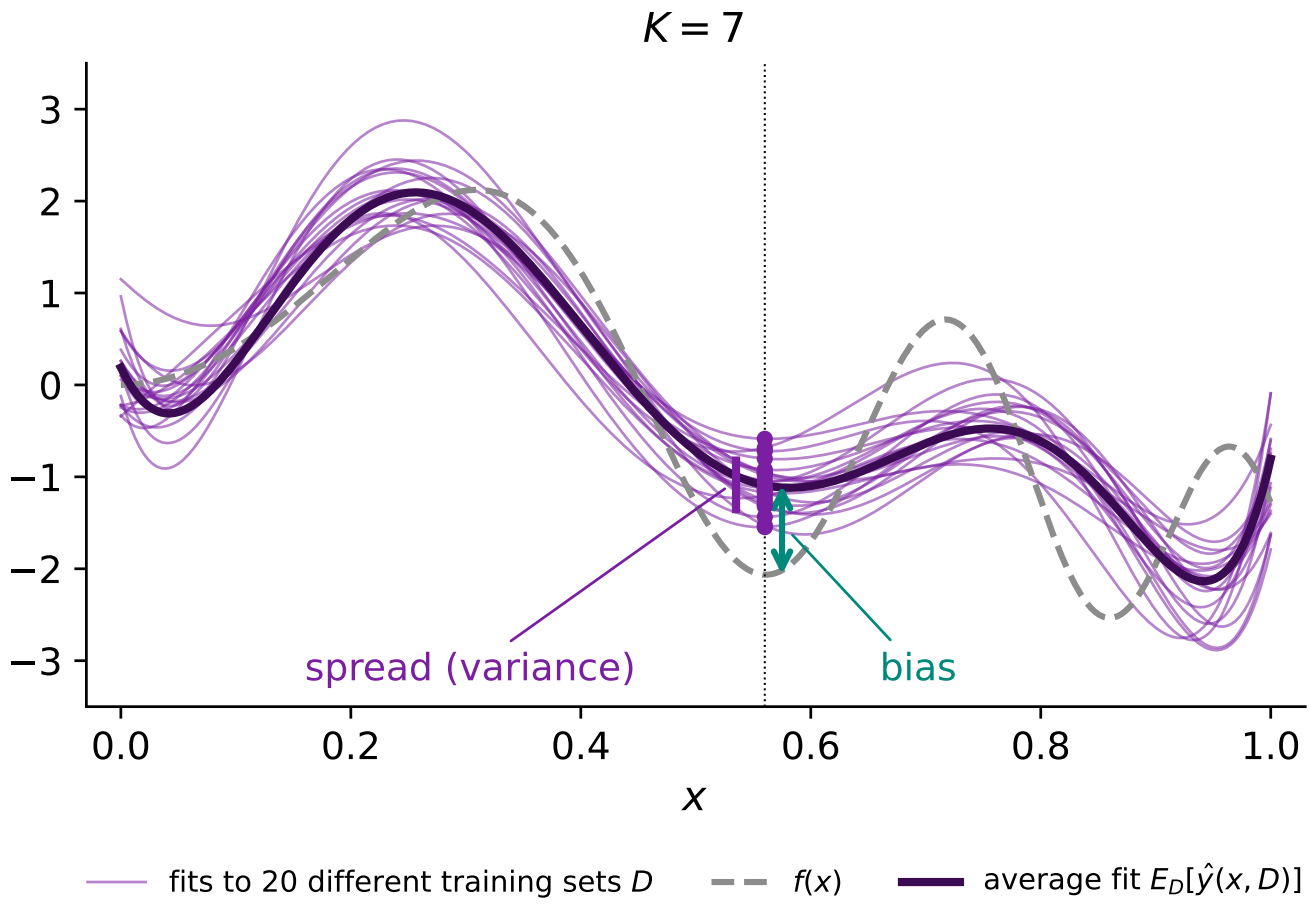
Bias and variance at one point



Bias and variance at one point

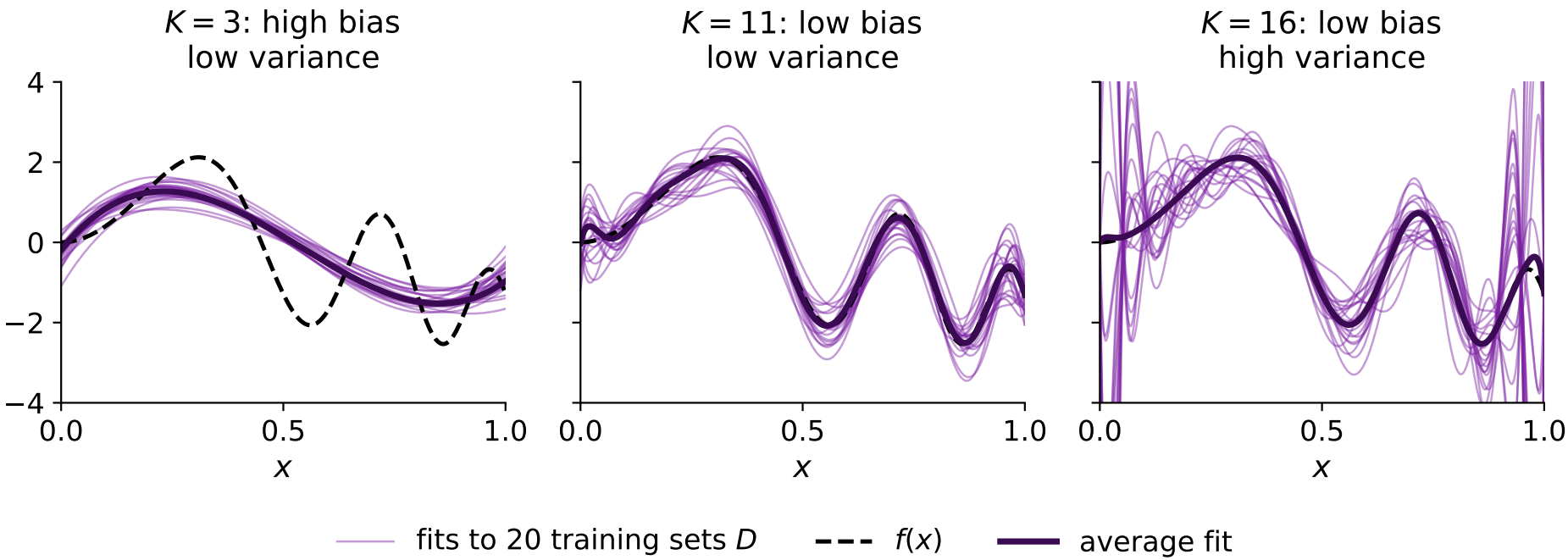


Bias and variance at one point

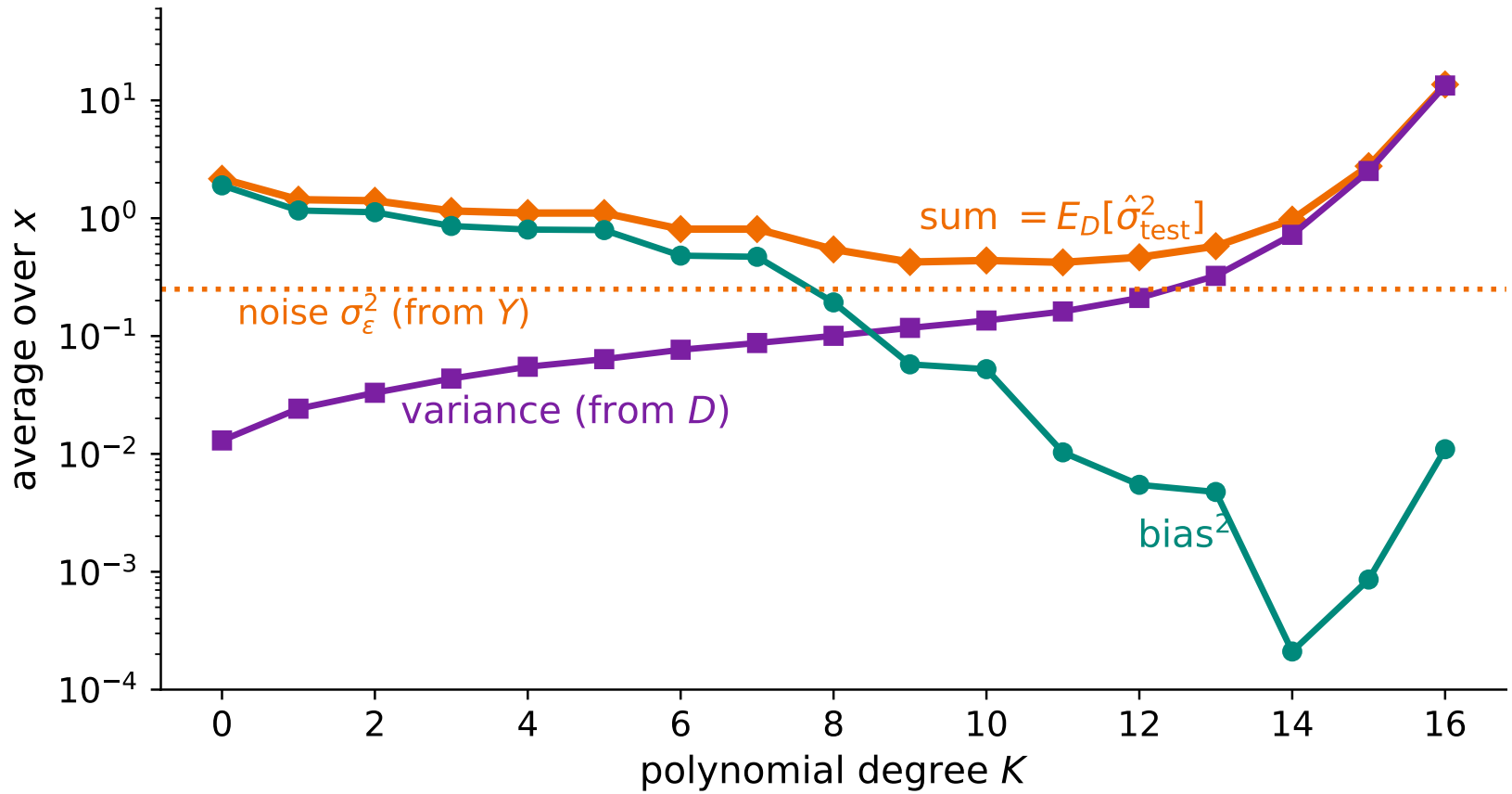


$$E\left[(\hat{y}(x, D) - Y)^2\right] = \underbrace{\sigma_\epsilon^2}_{\text{noise, from } Y} + \underbrace{\left(E_D[\hat{y}(x, D)] - f(x)\right)^2}_{\text{bias}^2} + \underbrace{\text{var}\left(\hat{y}(x, D)\right)}_{\text{variance, from } D}$$

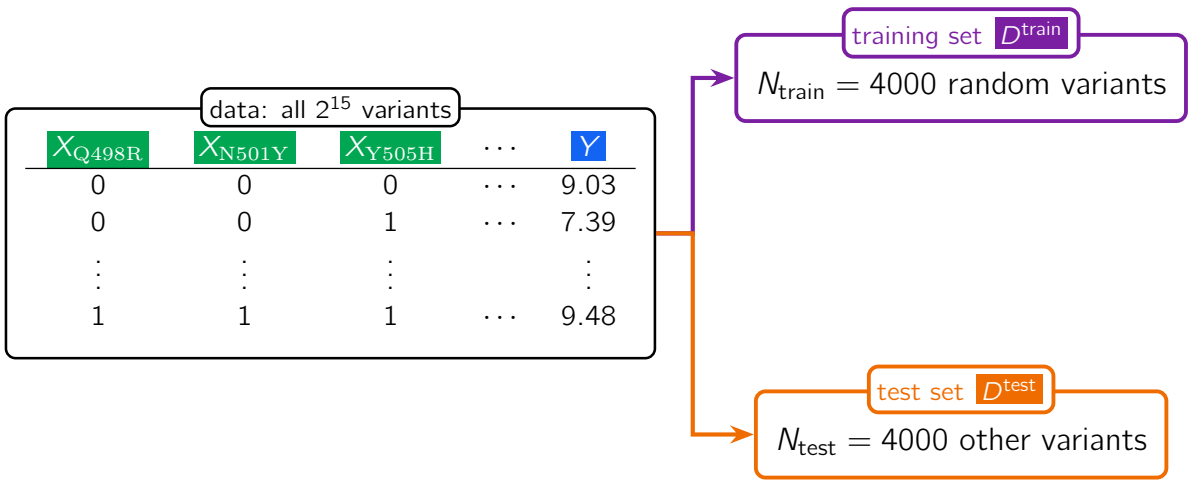
Bias and variance for the polynomial fits



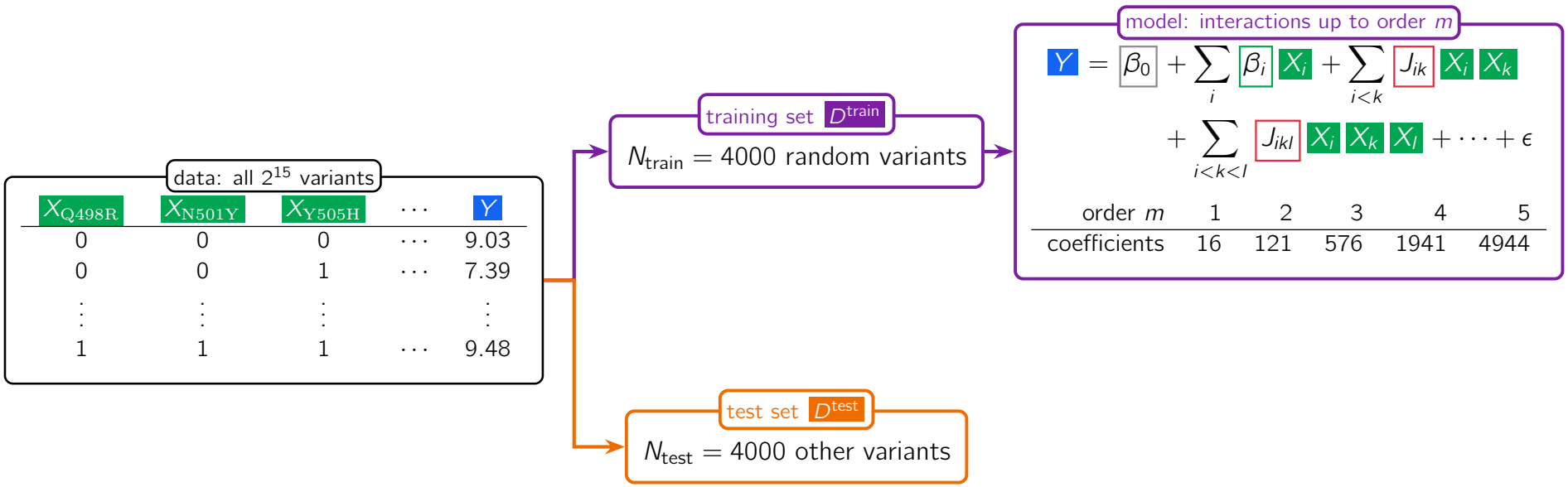
Bias and variance for the polynomial fits



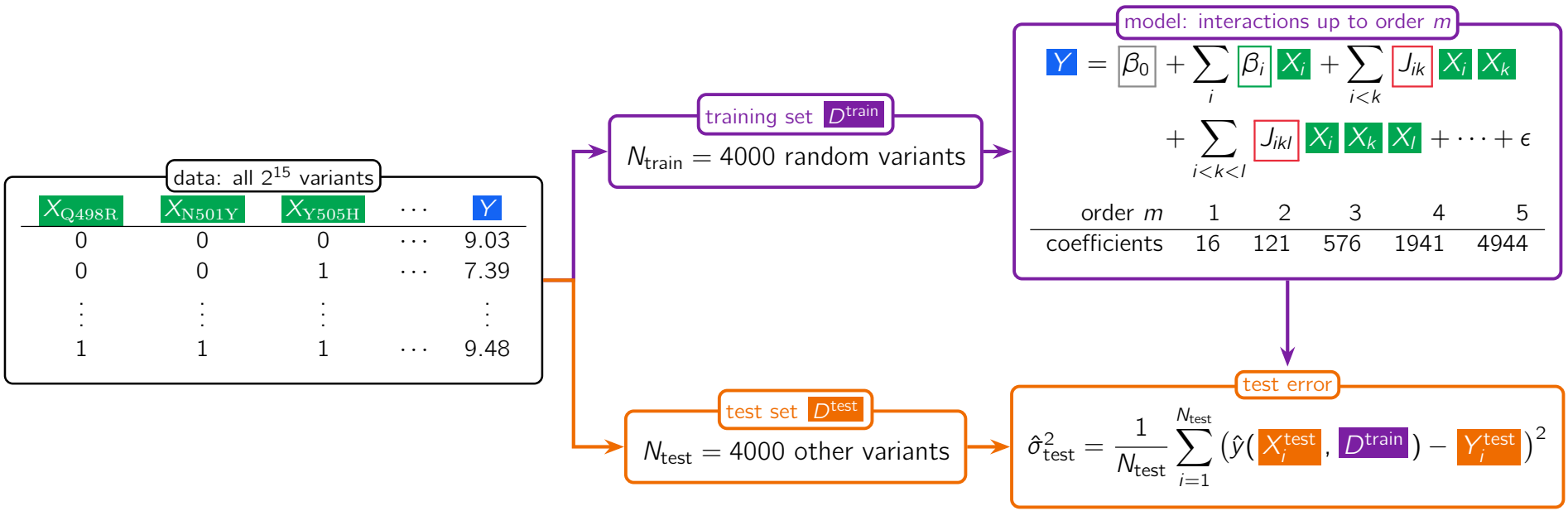
Back to the spike protein



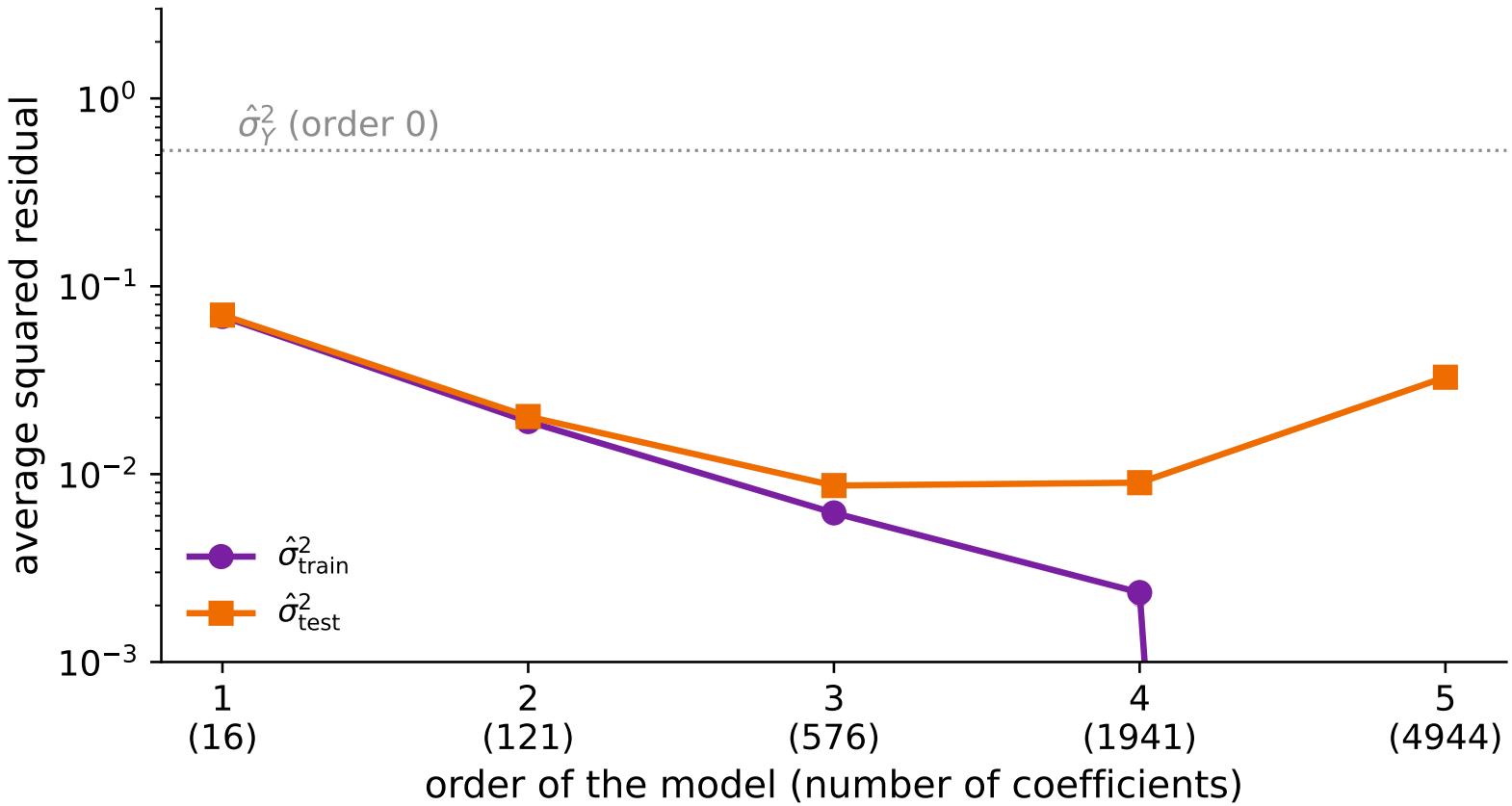
Back to the spike protein



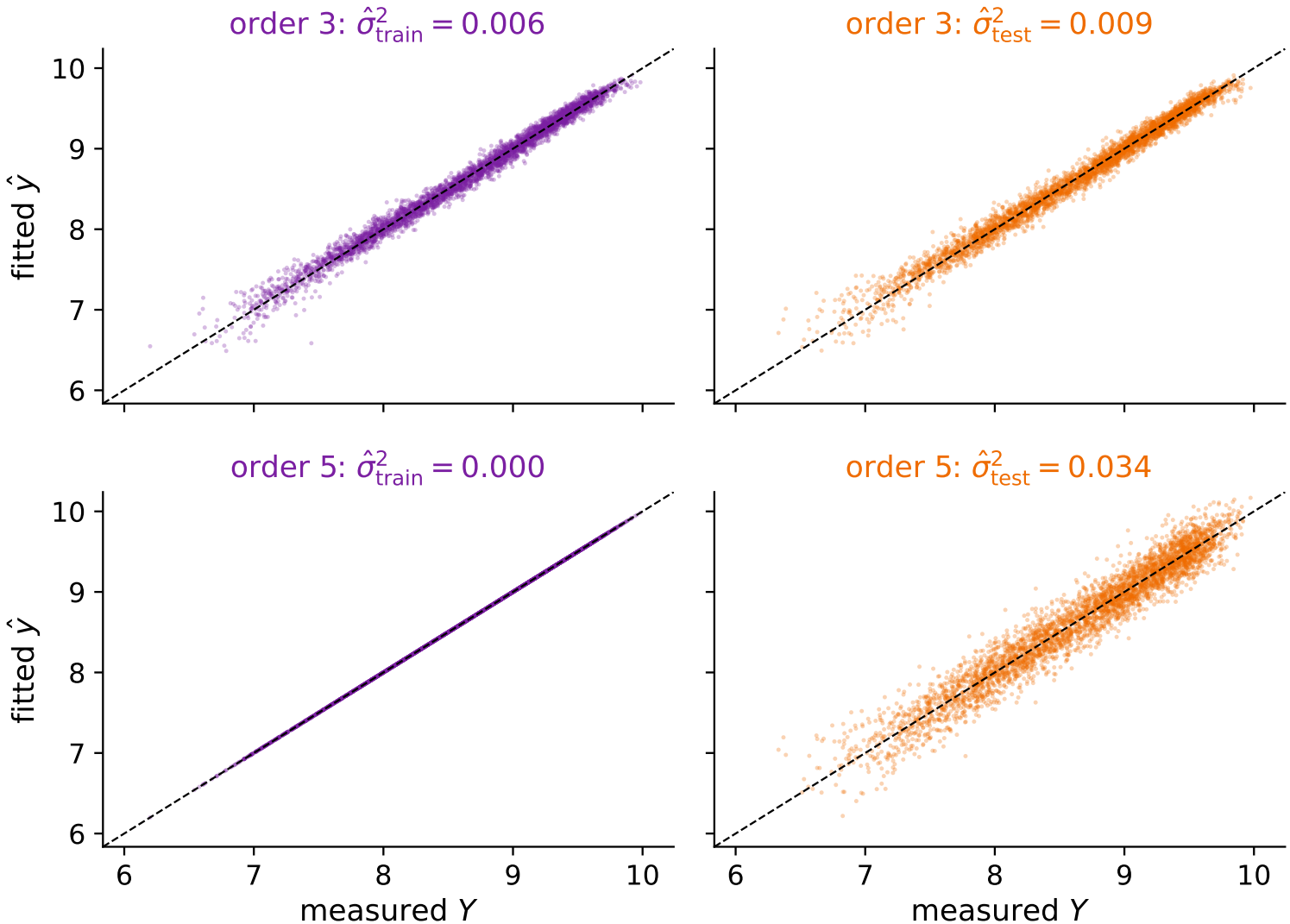
Back to the spike protein



Training and test error for the spike protein



Fitted vs. measured binding



Bias and variance for the spike protein

20 fits per order, each to a different random training set of 4000 variants, scored on the same 4000 test variants. f is unknown, so the squared bias and the noise cannot be separated.

